

Report on Descriptive, Predictive and Text Analytics for Airline Customer Reviews

Contents

Introduction.....	2
Methodology	2
Data Pre-Processing.....	2
<i>Numeric</i>	<i>3</i>
<i>Text</i>	<i>4</i>
Data Analytics.....	5
<i>Descriptive Analysis (Numerical)</i>	<i>5</i>
<i>Predictive Analysis (Numerical)</i>	<i>8</i>
<i>Descriptive Analysis (Text).....</i>	<i>10</i>
<i>Sentiment Approaches</i>	<i>10</i>
<i>Manual Analysis of Reviews</i>	<i>11</i>
<i>Visualisations</i>	<i>12</i>
<i>Predictive Analysis</i>	<i>14</i>
Appendix.....	25
Appendix A: Full Python Code.....	25
Appendix B: Manual Validation of Sentiment Classification	107
Appendix C: Illustrative Review Examples	108
Appendix D: Manual Review Coding and Classification Notes	110
Appendix E: Full Model Outputs.....	111

Introduction

The airline industry is highly competitive, with service quality playing a central role in shaping customer satisfaction, loyalty, and profitability (Han, Yu and Kim, 2019). Customer recommendation is a critical behavioural outcome, reflecting prior experience and willingness to endorse an airline. Although airlines increasingly rely on customer review data to inform decision-making, extracting actionable insights remains analytically challenging, particularly when combining structured ratings with unstructured textual feedback (Lucini et al., 2020; Yang et al., 2023).

Service quality in air transport is inherently multidimensional, spanning tangible elements such as seat comfort and onboard services, alongside intangible aspects including staff behaviour and the overall travel experience (Parasuraman et al., 1988; Bellizzi et al., 2020). Prior research suggests that structured ratings are effective predictors of customer outcomes, particularly where perceived value and service quality exhibit strong correlations with behavioural intentions (PwC, 2018). However, this literature also highlights a limitation: high predictive performance does not necessarily imply a comprehensive understanding of the evaluative process, as aggregated scores may mask the relative importance of specific service encounters (Keiningham et al., 2020).

A further strand of research emphasises the role of textual reviews in capturing contextual and experiential detail that structured ratings cannot fully represent (Filieri et al., 2018; Guo, Barnes and Jia, 2017). In particular, textual data reveals discrepancies between expressed sentiment and assigned ratings, suggesting that customer evaluations are shaped by salient experiences rather than uniform assessments across service dimensions (Liu et al., 2025). This aligns with evidence of asymmetric evaluation processes, where negative experiences exert a disproportionate influence on overall judgement.

Building on this literature, the present study examines whether numerical ratings alone are sufficient to explain recommendation behaviour, or whether textual features provide additional explanatory value. By integrating numerical and textual data within a unified analytical framework, the study evaluates the extent to which predictive performance reflects behavioural insight versus structural redundancy, providing a more robust basis for airline decision-making.

Methodology

This study is structured using the CRISP-DM framework (Shearer, 2000), progressing from business understanding to evaluation. The objective is to predict whether a passenger recommends an airline while identifying the service factors most strongly associated with that outcome. The task is not purely predictive but also decision-oriented, as airlines require evidence on which aspects of the customer journey warrant attention. This justifies combining numerical ratings with textual reviews to capture both structured evaluations and contextual service experiences.

During data understanding, exploratory analysis identified three key challenges: class imbalance, substantial systematic missingness in optional service variables, and inconsistencies between Overall Rating and recommendation. These patterns are consistent with airline review data, where selective reporting and uneven service provision are common (Mariani et al., 2018). Data preparation therefore aimed to preserve information while reducing distortion. Numerical preprocessing distinguished between service non-availability and poor performance through availability indicators and imputation, while textual preprocessing used tokenisation, lemmatisation, and negation preservation to retain sentiment-bearing language relevant to delays, cancellations, and service recovery.

Descriptive statistics and visualisations were selected to reflect the cleaned data structure and support managerial interpretation. Distributional and comparative visualisations were used to identify concentration, threshold effects, and segment-level differences, while a management-oriented visualisation translated outputs into actionable service priorities.

For predictive modelling, Logistic Regression and Random Forest were selected as complementary supervised methods. Logistic Regression offers interpretability through transparent coefficients and probabilities, whereas Random Forest captures non-linear relationships and interaction effects with fewer assumptions (Breiman, 2001; James et al., 2021). More complex models were not prioritised because the assignment requires not only predictive accuracy but also methodological transparency and managerial usefulness (Zhang and Yang, 2022). Generative AI was used only in an assistive role, with outputs validated through manual review and domain-based judgement.

Data Pre-Processing

Numeric

Data pre-processing addressed non-trivial quality issues to ensure reliable modelling of customer recommendation. As shown in Table 1, several service variables exhibit substantial missingness, particularly Wifi & Connectivity (74.45%) and Inflight Entertainment (53.26%). This reflects systematic missingness driven by service availability, which carries informational value rather than representing random data loss. Logical inconsistencies, such as divergence between Overall Rating and recommendation, were retained, as they capture genuine complexity in customer evaluation rather than error.

Table 1 Missing Values Before Cleaning

<i>Variable</i>	<i>Missing Before</i>	<i>Missing %</i>
<i>Wifi & Connectivity</i>	17251	74.45 %
<i>Inflight Entertainment</i>	12342	53.26 %
<i>Food & Beverages</i>	8671	37.42 %
<i>Ground Service</i>	4793	20.69 %
<i>Cabin Staff Service</i>	4260	18.39 %
<i>Seat Comfort</i>	4155	17.93 %
<i>Value For Money</i>	1066	4.60 %
<i>Overall Rating</i>	0	0.00 %

Two approaches to missing data were considered. A strict deletion strategy (Table 2) was rejected, as it would remove 76.8% of observations and distort class balance, reducing model generalisability. Instead, an availability-aware approach was implemented: missing values for optional services were recoded to zero and supplemented with binary indicators, allowing models to distinguish between absence of service and poor performance. For core service variables, median imputation was applied (Table 3), as zero imputation introduces downward bias and weakens relationships with recommendation.

Table 2 Impact of Dropping Optional-Service Missingness

<i>Feature</i>	<i>Original Dataset</i>	<i>After Dropping Optional-Service Missingness</i>
<i>Total Rows</i>	23,171	5,376
<i>Rows Removed</i>	0	17,795
<i>Percent Rows Removed</i>	0.00	76.80
<i>Not Recommended Percent</i>	66.31	78.27
<i>Recommended Percent</i>	33.69	21.73

Table 3 Zero Versus Median Imputation For Core Services

Feature	Original $\bar{X}$	Zero Impute $\bar{X}$	Median Impute $\bar{X}$	Original M	Zero Impute M	Median Impute M
Seat Comfort	2.62	2.15	2.69	3.0	2.0	3.0
Cabin Staff Service	2.87	2.34	2.90	3.0	2.0	3.0
Ground Service	2.35	1.87	2.07	1.0	1.0	1.0

Following preprocessing, a combination of row removal, imputation, and recoding produced a consistent dataset for modelling (Table 4). While sample size was reduced, these decisions improve data integrity and ensure that model outputs reflect meaningful service variation rather than artefacts of missingness, directly enhancing interpretability for airline decision-making.

Table 4 Before and After Cleaning

Feature	Rows Before	Cleaning Method	Rows Removed	Rows After
Overall Rating	23171	Drop missing rows	842	22329
Value For Money	22329	Drop missing rows	541	21788
Wifi and Connectivity	21788	Add availability indicator and fill missing with 0	0	21788
Inflight Entertainment	21788	Add availability indicator and fill missing with 0	0	21788
Food & Beverages	21788	Add availability indicator and fill missing with 0	0	21788
Seat Comfort	21788	Fill missing with median (3.0)	0	21788
Cabin Staff Service	21788	Fill missing with median (3.0)	0	21788
Ground Service	21788	Fill missing with median (1.0)	0	21788

Text

A context-specific preprocessing pipeline was applied to textual reviews, including tokenisation, lemmatisation, and negation preservation, to retain sentiment-relevant information while enabling structured analysis. This approach is appropriate for airline data, where dissatisfaction is often expressed through specific service failures (delays or cancellations) (Khan et al., 2023). However, preprocessing may reduce nuance, particularly in cases of sarcasm or mixed sentiment (Cambria et al., 2020).

Exploratory analysis (Table 5) reveals dataset-specific patterns: operational failures such as customer support, cancellations, refunds, and delays are strongly associated with non-recommended reviews, while positive themes (staff, seat, food) are more prevalent in recommended reviews. This indicates that dissatisfaction is driven by discrete service failures rather than overall experience quality, supporting the inclusion of text-derived features and reinforcing the need to prioritise failure reduction in airline operations.

Table 5 Theme Prevalence By Recommendation

Theme	Not Recommended Percent	Recommended Percent	Difference pp
Customer Support Theme	41.57 %	10.03 %	31.54
Cancellation Theme	21.56 %	2.53 %	19.03
Refund Theme	17.24 %	1.53 %	15.71
Delay Theme	34.69 %	19.48 %	15.22
Baggage Theme	29.74 %	22.96 %	6.78
Boarding Theme	36.37 %	38.15 %	-1.77
Wifi Theme	2.31 %	4.72 %	-2.40
Seat Theme	26.27 %	43.25 %	-16.98
Staff Theme	59.25 %	80.51 %	-21.26
Food Theme	21.32 %	45.84 %	-24.52

Data Analytics

Descriptive Analysis (Numerical)

Descriptive statistics (Table 6) indicate low customer satisfaction, with a mean Overall Rating of 3.14 and a median of 1, reflecting a strong concentration of negative evaluations. This is consistent with recommendation outcomes (Figure 1), where 64.3% of customers do not recommend the airline. This imbalance supports an asymmetric evaluation process in which negative experiences exert a disproportionate influence on recommendation behaviour (Hollebeek et al., 2019), implying that preventing service failure is more impactful than incremental improvements.

Table 6 Descriptive Statistics for Numerical Variables

Variable	Count	Mean	Std	Min	25%	50%	75%	Max	Missing count	Missing %
Overall Rating	21788.0	3.14	2.90	1.0	1.0	1.0	5.0	9.0	0	0.0
Seat Comfort	21788.0	2.68	1.37	0.0	1.0	3.0	4.0	5.0	0	0.0
Cabin Staff	21788.0	2.90	1.49	0.0	1.0	3.0	4.0	5.0	0	0.0
Food & Beverages	21788.0	1.69	1.74	0.0	0.0	1.0	3.0	5.0	0	0.0
Ground Service	21788.0	2.14	1.55	1.0	1.0	1.0	4.0	5.0	0	0.0
Inflight Entertainment	21788.0	1.08	1.51	0.0	0.0	0.0	1.0	5.0	0	0.0
Wifi	21788.0	0.48	1.05	0.0	0.0	0.0	1.0	5.0	0	0.0
Value For Money	21788.0	2.47	1.60	0.0	1.0	2.0	4.0	5.0	0	0.0

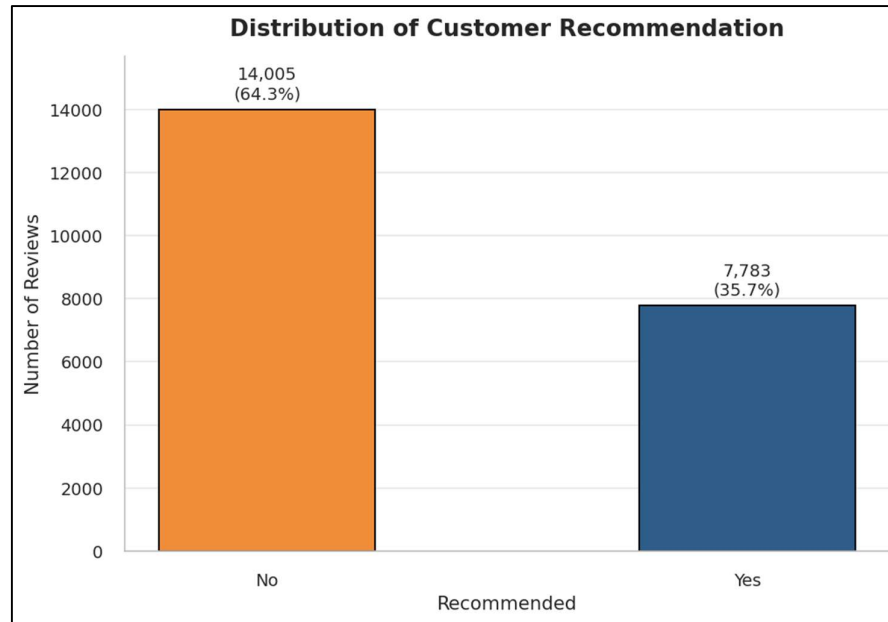


Figure 1 Distribution of Customer Recommendation

Two non-obvious patterns emerge. First, 13.6% of observations exhibit inconsistencies between Overall Rating and recommendation (Figure 2), indicating that recommendation is not determined solely by aggregate scores but by specific service encounters (De Keyser et al., 2020). This limits the usefulness of summary metrics, as they can obscure critical failure points in the customer journey.

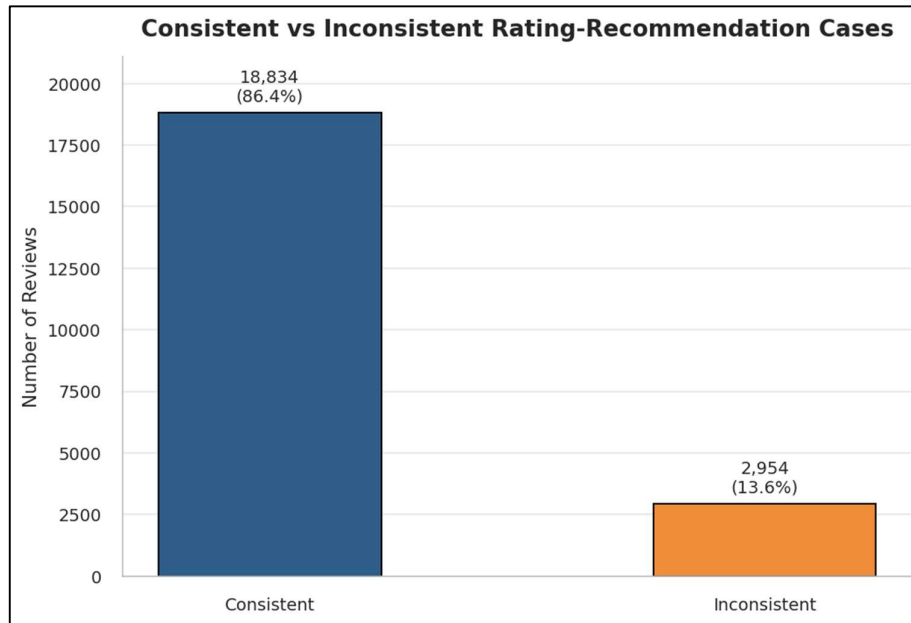


Figure 2 Consistent vs Inconsistent Rating-Recommendation Cases

Second, recommendation behaviour exhibits a clear threshold effect. As shown in Figure 3, recommendation rates remain minimal at lower rating levels before increasing sharply beyond a critical point. This is reinforced by the managerial visualisation of Value for Money (Figure 4), where recommendation rises from near zero to over 90% as perceived value improves. This visual is particularly effective for non-technical stakeholders, as it highlights the relationship between perceived value and recommendation.

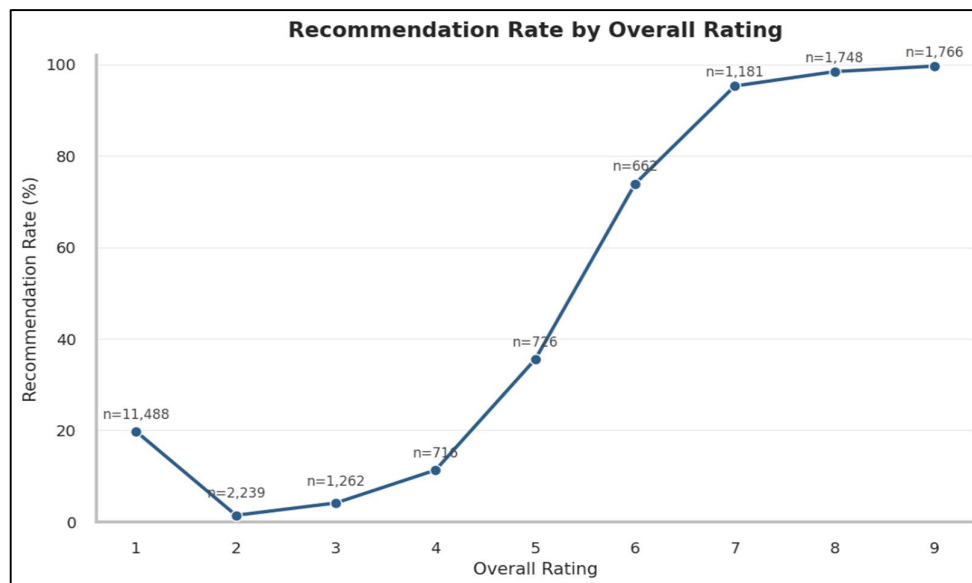


Figure 3 Recommendation Rate by Overall Rating

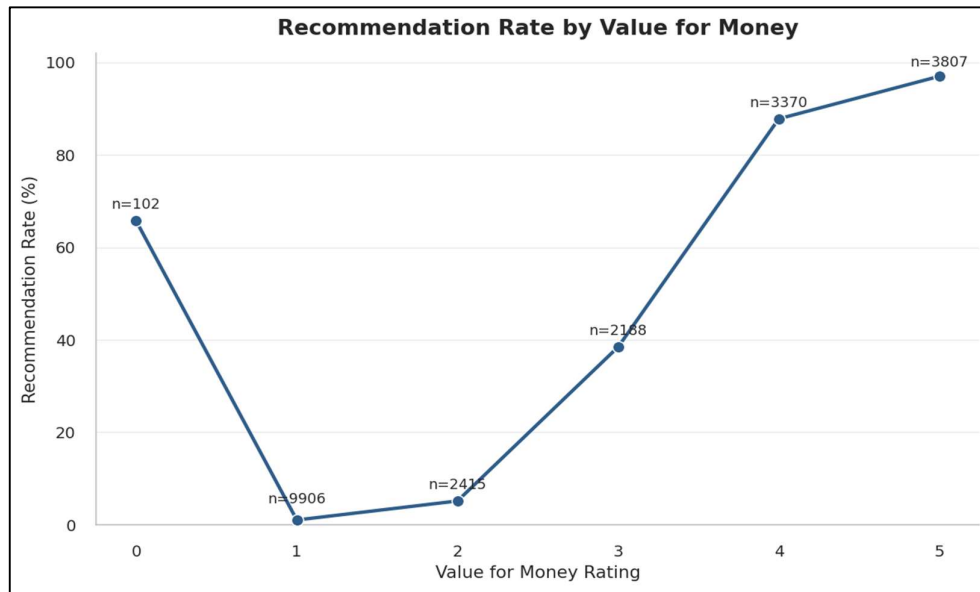


Figure 4 Recommendation Rate by Value for Money

Comparative visualisations further support these findings. Recommendation rates across seat types (Figure 5) show only moderate gains for premium passengers, suggesting that higher service tiers do not offset elevated expectations. Correlation analysis (Figure 6) identifies Value for Money as the dominant driver of recommendation, while features such as Wifi & Connectivity have minimal influence. This suggests that customer evaluation is more strongly associated with core service delivery than peripheral features (Lemon and Verhoef, 2016), implying that investment in reliability and value perception yields greater returns than improvements in secondary services.

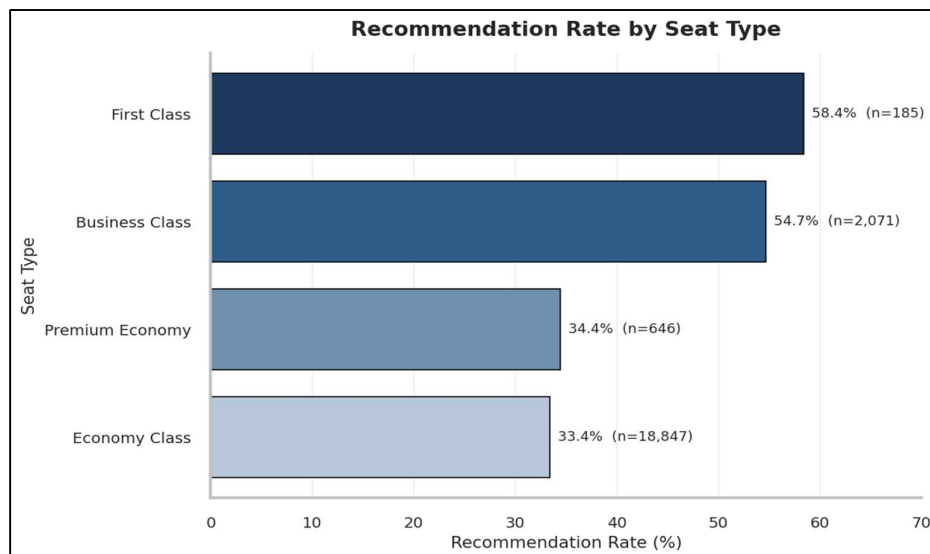


Figure 5 Recommendation Rate by Seat Type

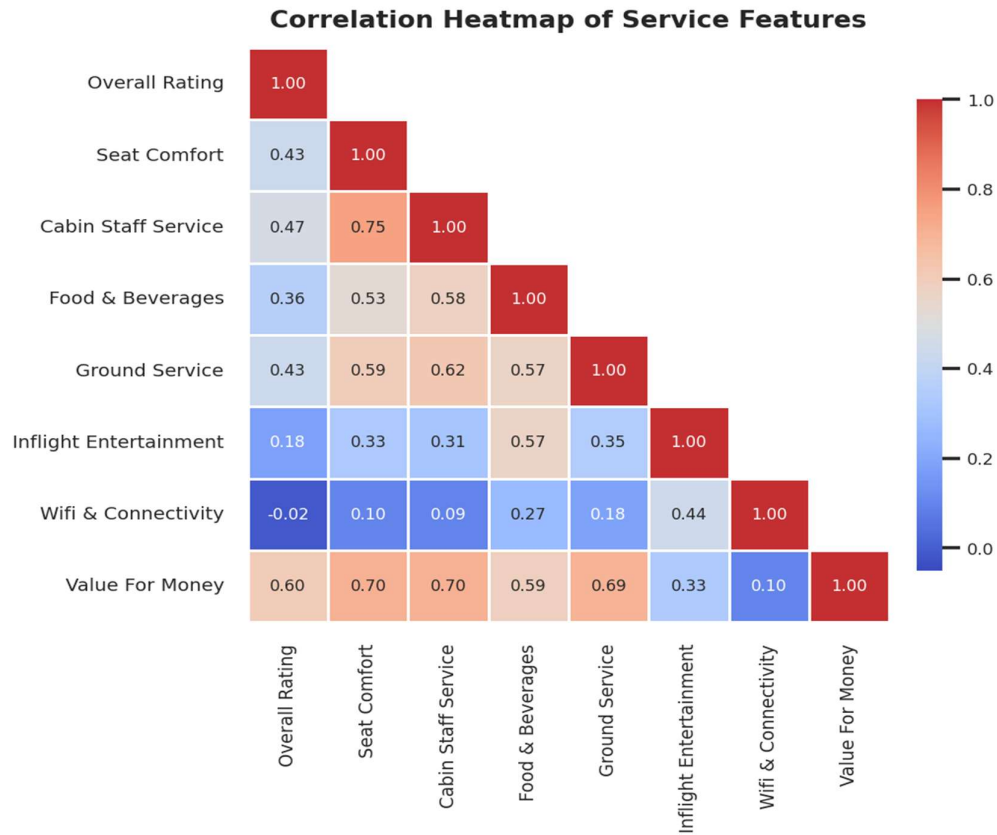


Figure 6 Correlation Heatmap of Service Features

Predictive Analysis (Numerical)

Predictive modelling was conducted using Logistic Regression and Random Forest, selected for their complementary strengths in interpretability and non-linear modelling. A stratified train-test split (80:20) preserved class proportions, ensuring reliable evaluation. Both models achieved strong performance (ROC-AUC > 0.97; Table 7), indicating that numerical service attributes provide a stable signal of recommendation behaviour. However, Random Forest consistently outperformed Logistic Regression (Table 7), suggesting that non-linear dependencies are present in the relationship between service attributes and recommendation behaviour. This performance difference is reflected in the ROC curve (Figure 8), where Random Forest maintains a higher true positive rate across thresholds. This suggests that combinations of service failures may disproportionately reduce recommendation likelihood, consistent with evidence that airline choice is shaped by multiple service dimensions acting jointly (McKinsey, 2024).

Model	Accuracy	Precision	Recall	F1	ROC_AUC
Logistic Regression (0.5)	0.9449	0.9290	0.9159	0.9224	0.9795
Random Forest (0.5)	0.9631	0.9532	0.9428	0.9480	0.9884
Lasso Logistic Regression (0.5)	0.9449	0.9290	0.9159	0.9224	0.9797
Logistic Regression (0.3)	0.9440	0.8919	0.9595	0.9245	-
Random Forest (0.3)	0.9578	0.9225	0.9627	0.9422	-
Lasso Logistic Regression (0.3)	0.9442	0.8920	0.9602	0.9248	-

Table 7 Model Comparisons

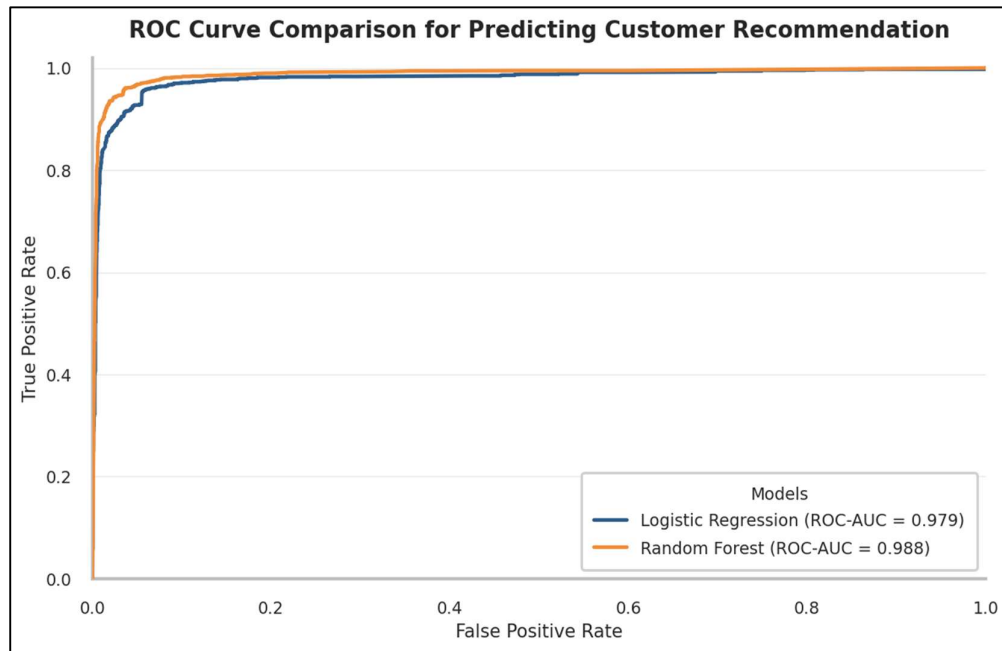


Figure 8 ROC Curve

LASSO regularisation confirms strong feature concentration, with Value for Money and Overall Rating dominating all predictors (Table 8). This suggests that model performance reflects a compressed representation of customer sentiment, where a small number of variables act as proxies for overall experience, potentially overstating behavioural understanding.

<i>Feature</i>	<i>Coefficient</i>
<i>Value For Money</i>	1.988729
<i>Overall Rating</i>	1.445538
<i>Food & Beverages_available</i>	-0.687045
<i>Wifi & Connectivity_available</i>	-0.641395
<i>Cabin Staff Service</i>	0.564465
<i>Seat Comfort</i>	0.557228
<i>Food & Beverages</i>	0.507841
<i>Ground Service</i>	0.404878
<i>Wifi & Connectivity</i>	0.367257
<i>Inflight Entertainment</i>	-0.264105
<i>Inflight Entertainment_available</i>	0.250507

Table 8 Features Selected by Lasso

Threshold adjustment (Table 7) highlights a trade-off between precision and recall. Lower thresholds improve recall, supporting applications such as loyalty targeting, but increase false positives, requiring alignment with strategic priorities.

Hyperparameter tuning produced marginal improvements (Table 9), indicating that baseline models capture the dominant structure of the data. While this suggests behavioural stability, it may also reflect the strong correlation between Value for Money and recommendation, potentially inflating predictive performance without capturing deeper behavioural drivers. Robustness testing using a time-based split

(Table 10) confirms consistent performance across periods. However, airline environments are inherently dynamic, with shifting expectations and external disruptions, meaning model performance may not fully generalise without ongoing recalibration.

<i>Model</i>	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1</i>	<i>ROC_AUC</i>
<i>Baseline Logistic Regression</i>	0.9449	0.9290	0.9159	0.9224	0.9795
<i>Baseline Random Forest</i>	0.9631	0.9532	0.9428	0.9480	0.9884
<i>Tuned Logistic Regression</i>	0.9403	0.9030	0.9332	0.9179	0.9802
<i>Tuned Random Forest</i>	0.9626	0.9417	0.9544	0.9480	0.9912

Table 9 Baseline Versus Tuned Models

<i>Model</i>	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1</i>	<i>ROC_AUC</i>
<i>Logistic Regression (Random Split)</i>	0.9449	0.9290	0.9159	0.9224	0.9795
<i>Random Forest (Random Split)</i>	0.9631	0.9532	0.9428	0.9480	0.9884
<i>LASSO Logistic Regression (Random Split)</i>	0.9449	0.9290	0.9159	0.9224	0.9797
<i>Logistic Regression (Time-Based Split)</i>	0.9617	0.8981	0.9431	0.9201	0.9864
<i>Random Forest (Time-Based Split)</i>	0.9670	0.9327	0.9254	0.9291	0.9832
<i>LASSO Logistic Regression (Time-Based Split)</i>	0.9617	0.8981	0.9431	0.9201	0.9864

Table 10 Random Split Versus Time Based Split

The confusion matrix (Table 11) shows balanced classification with low misclassification. Overall, Random Forest provides superior predictive accuracy, while LASSO enhances interpretability, supporting targeted strategies that prioritise perceived value and consistent service delivery as the primary drivers of customer recommendation (Molnar, 2022).

	<i>Predicted 0</i>	<i>Predicted 1</i>
<i>Actual 0</i>	2729	72
<i>Actual 1</i>	89	1468

Table 11 Confusion Matrix Random Forest (0.5)

Descriptive Analysis (Text)

Sentiment Approaches

Two sentiment approaches were applied: VADER (lexicon-based) and RoBERTa (contextual transformer). While VADER enables rapid polarity detection, its reliance on fixed lexical rules limits its ability to interpret multi-stage airline experiences (Hutto and Gilbert, 2014). RoBERTa provides contextual understanding but remains sensitive to surface-level negativity rather than decision-relevant judgement (Cambria et al., 2020).

Comparison (Tables 12–13) highlights these limitations. VADER produces highly polarised outputs, overstating positivity (70.6% recommendation for “positive” sentiment), while RoBERTa assigns a larger share of neutral sentiment, reflecting mixed language. Although RoBERTa aligns more closely with recommendation (93.4% for positive sentiment), this reflects correlation rather than behavioural understanding. Both models fail to distinguish minor service issues from critical operational failures, limiting their standalone value.

<i>Sentiment</i>	<i>VADER Count</i>	<i>RoBERTa Count</i>
<i>Negative</i>	11,377	12,649
<i>Neutral</i>	323	2,816
<i>Positive</i>	10,088	6,323

Table 12 Sentiment Distribution (Model Comparison)

<i>Model</i>	<i>Sentiment</i>	<i>Total Reviews</i>	<i>Recommended</i>	<i>Not Recommended</i>	<i>Recommendation Rate</i>
<i>VADER</i>	<i>Negative</i>	11,377	593	10,784	5.2 %
<i>VADER</i>	<i>Neutral</i>	323	70	253	21.7 %
<i>VADER</i>	<i>Positive</i>	10088	7120	2968	70.6 %
<i>RoBERTa</i>	<i>Negative</i>	12649	829	11820	6.6 %
<i>RoBERTa</i>	<i>Neutral</i>	2816	1050	1766	37.3 %
<i>RoBERTa</i>	<i>Positive</i>	6323	5904	419	93.4 %

Table 13 Sentiment vs Recommendation (Model Comparison)

This is problematic in airlines, where recommendation is driven by a small number of decisive failures rather than overall tone. Sentiment models therefore risk oversimplifying customer evaluation by treating all expressions as equally informative (Liu et al., 2019).

Manual Review Analysis

A total of 15 ambiguous or misclassified reviews were manually examined. Table 14 presents 10 illustrative cases, while the full set of reviews is provided in Appendix B–D.

<i>Group</i>	<i>Review ID</i>	<i>Rating</i>	<i>Recommended</i>	<i>VADER</i>	<i>RoBERTa</i>
<i>VADER Misclass</i>	17581	4	No	Positive	Negative
<i>VADER Misclass</i>	4675	4	No	Positive	Negative
<i>VADER Misclass</i>	12207	1	No	Positive	Negative
<i>RoBERTa Misclass</i>	8077	6	Yes	Positive	Negative
<i>RoBERTa Misclass</i>	16961	7	Yes	Positive	Negative
<i>RoBERTa Misclass</i>	6093	5	No	Negative	Positive
<i>RoBERTa Misclass</i>	17207	7	Yes	Positive	Neutral
<i>Ambiguous</i>	21101	7	Yes	Positive	Neutral
<i>Ambiguous</i>	6929	7	Yes	Positive	Neutral
<i>Ambiguous</i>	6118	1	Yes	Positive	Positive

Table 14 Sample of Ambiguous and Misclassified Reviews

First, mixed-valence narratives distort polarity signals. In Review 17581, VADER assigns positive sentiment due to favourable comments on food, yet the review is dominated by delays, a broken seat, and poor staff response, resulting in non-recommendation. This shows that lexicon-based models aggregate sentiment, whereas customers weight severe failures more heavily.

Second, models fail to capture attribution of responsibility. In Review 8077, disruption is described, yet the airline is recommended because delays are attributed externally. RoBERTa classifies this as negative, indicating that it captures negative content but not how passengers interpret its cause.

Third, airline reviews are contrastive and outcome-driven. Final judgements depend on whether a minimum acceptable standard is met, explaining persistent misalignment between sentiment and recommendation and reinforcing threshold-based evaluation.

Visualisations

Figures 9–10 show overall alignment between sentiment and recommendation, with positive sentiment concentrated among recommended reviews. However, RoBERTa’s higher neutral share reflects mixed experiences rather than clearer interpretation.

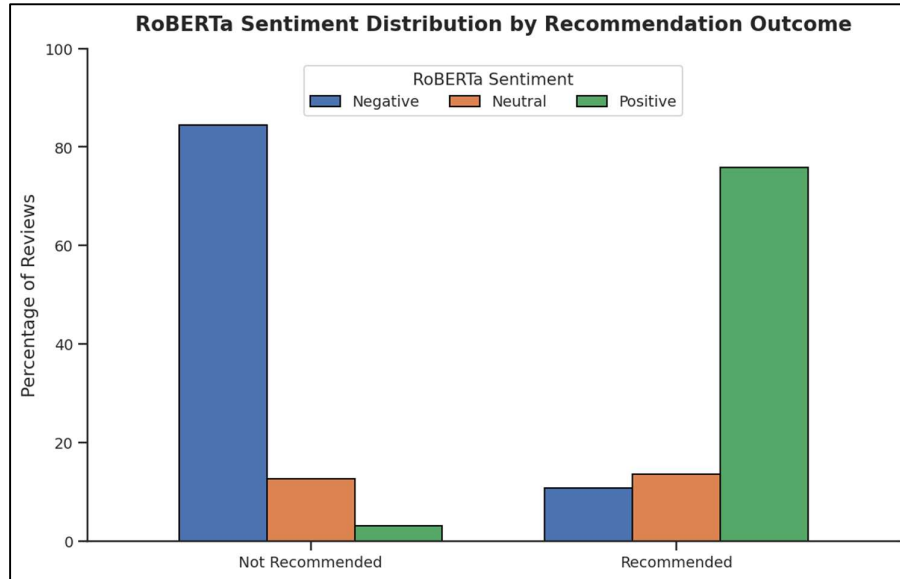


Figure 9: VADER Sentiment Distribution

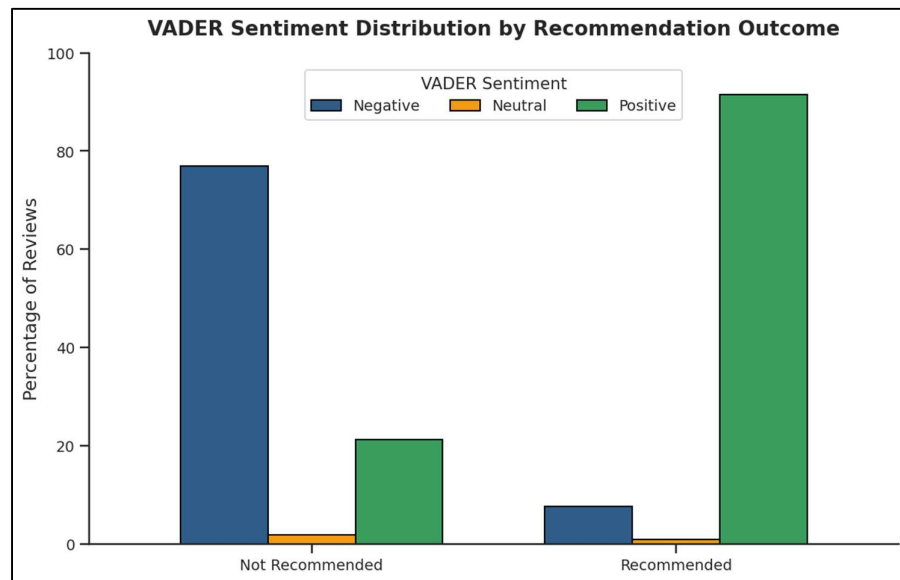


Figure 10: RoBERTa Sentiment Distribution

Contradiction analysis (Figure 11) exposes key limitations. VADER frequently assigns positive sentiment to non-recommended reviews, misinterpreting balanced complaints. RoBERTa performs better but still misclassifies cases where negative events do not translate into dissatisfaction. This confirms that sentiment alone is insufficient for understanding behaviour (Kunić, 2019).

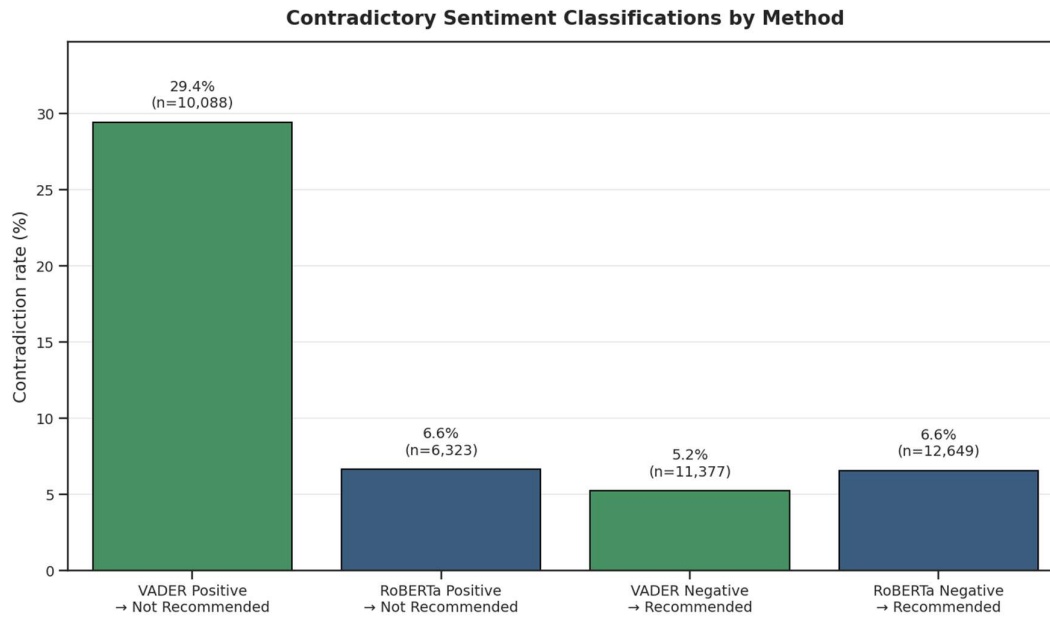


Figure 11: Contradictory Sentiment Classifications

Integration with numerical ratings (Figures 12–13) shows that structured ratings dominate decision-making. High ratings consistently lead to recommendation regardless of sentiment, while low ratings result in non-recommendation even when sentiment appears positive. This indicates that sentiment refines rather than replaces structured evaluation.

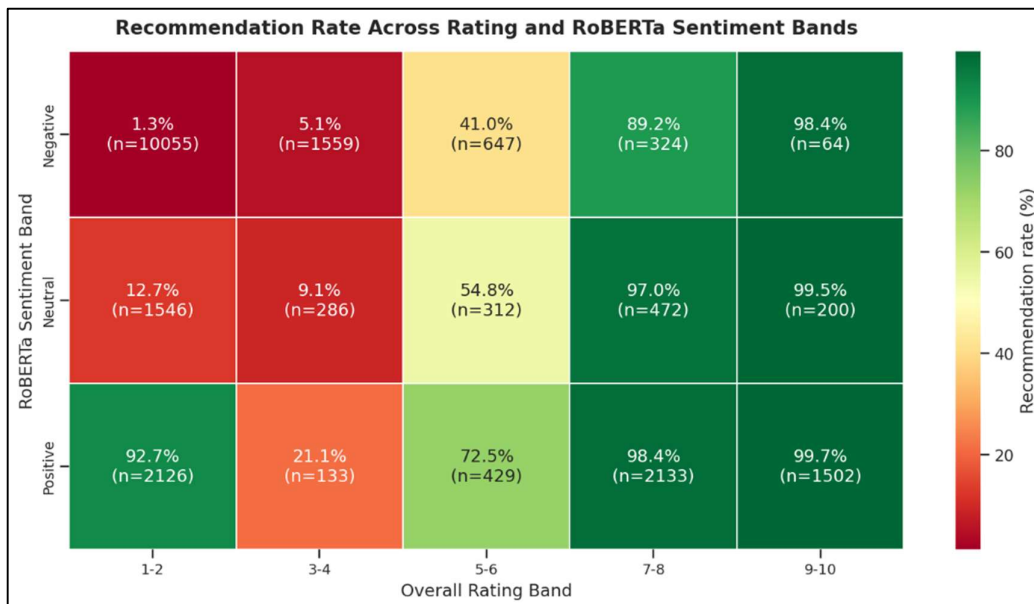


Figure 12: VADER Sentiment vs Rating Heatmap

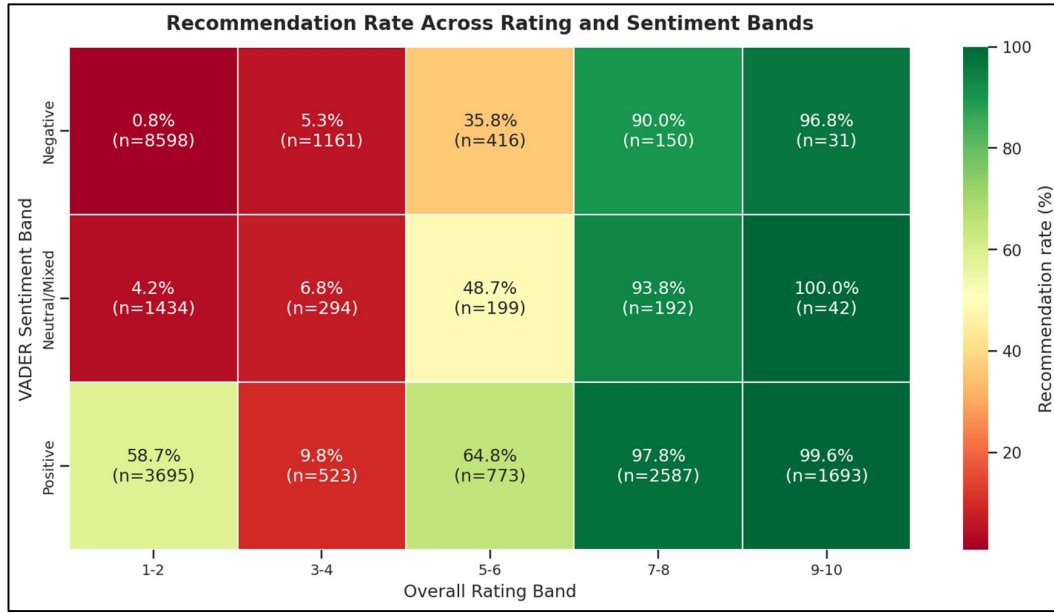


Figure 13: RoBERTa Sentiment vs Rating Heatmap

Theme analysis (Figure 14) reinforces this. Operational issues such as refunds, cancellations, and delays correspond to extremely low recommendation rates, while service elements such as staff and food perform better. For airlines, this confirms that operational reliability, not incremental service quality is the primary driver of recommendation (McKinsey, 2023).

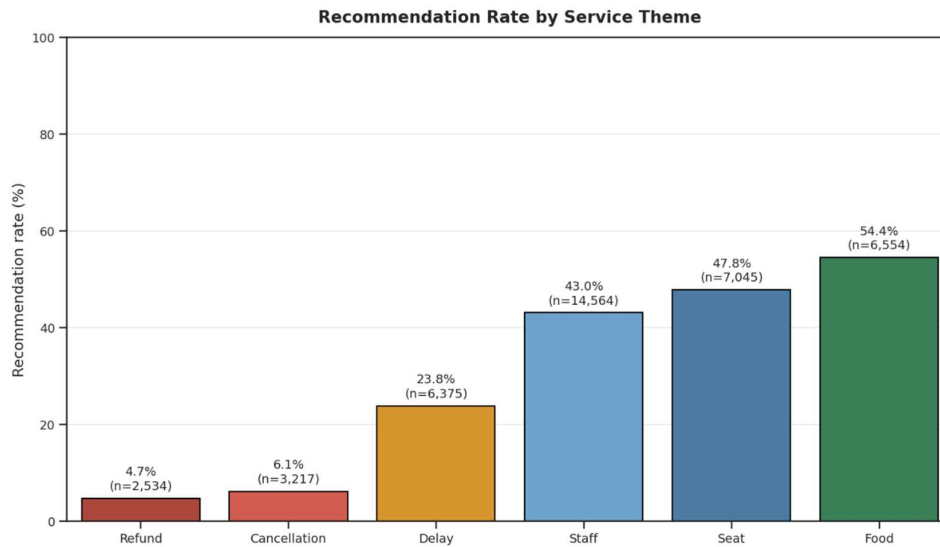


Figure 14: Recommendation Rate by Service Theme

Predictive Analysis

Text models were trained using TF-IDF features (39,277 terms) with Logistic Regression, Random Forest, and Linear SVM (Table 15; Figure 15). All models achieve strong performance (ROC-AUC $\approx$ 0.97–0.98), confirming that text contains predictive signal.

<i>Model</i>	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1 Score</i>	<i>ROC-AUC</i>
<i>Text Logistic Regression</i>	0.9371	0.9332	0.8876	0.9098	0.9771
<i>Text Random Forest</i>	0.9158	0.9053	0.8536	0.8787	0.9677
<i>Text Linear SVM</i>	0.9406	0.9315	0.8998	0.9154	0.9781

Table 15: Text Model Performance Comparison

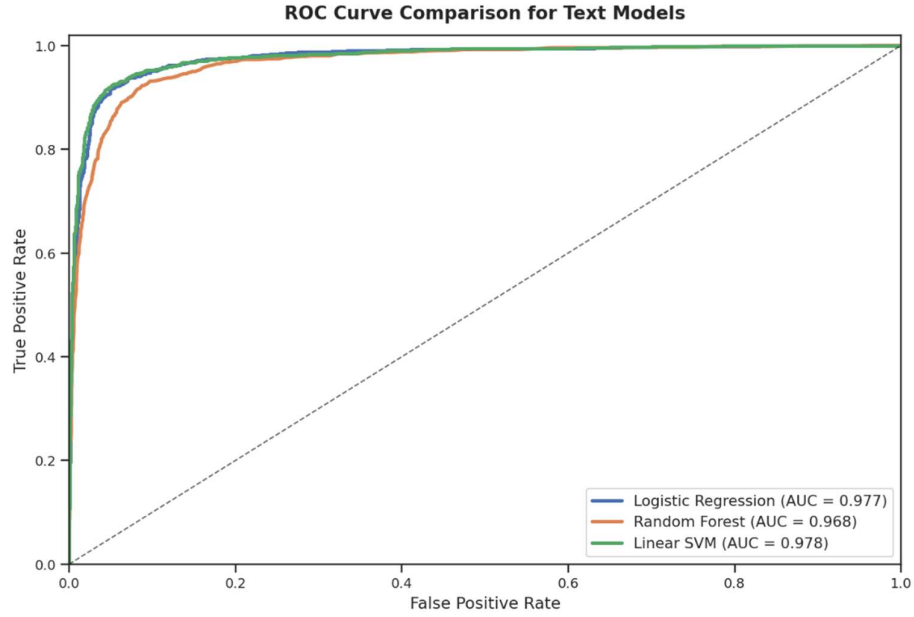


Figure 15: ROC Curve Comparison for Text Models

Linear SVM performs best (Table 15), reflecting its suitability for high-dimensional text (Joachims, 1998), while Random Forest underperforms, indicating limited value from modelling non-linear relationships in sparse data. Logistic Regression performs similarly, suggesting prediction is driven by dominant keywords rather than deeper linguistic structure.

<i>Model</i>	<i>True Negatives</i>	<i>False Positives</i>	<i>False Negatives</i>	<i>True Positives</i>
<i>Logistic Regression</i>	2,702	99	175	1,382
<i>Random Forest</i>	2,662	139	228	1,329
<i>Linear SVM</i>	2,698	103	156	1,401

Table 16 Confusion Matrix Summary

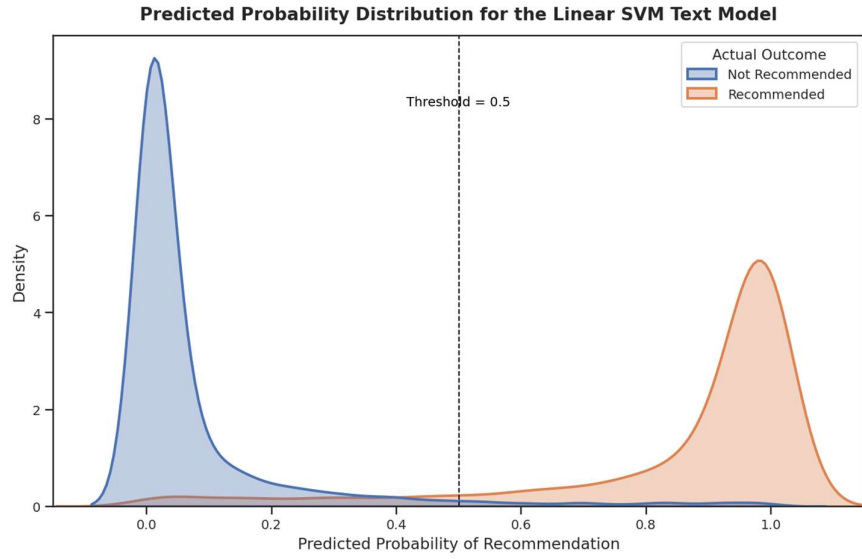


Figure 16: Predicted Probability Distribution (Linear SVM)

Confusion matrices (Table 16) show lower recall for recommended cases, indicating difficulty capturing nuanced positive experiences. The probability distribution (Figure 16) confirms class separation but highlights ambiguity near the decision threshold.

Table 17: Numerical vs Text Model Comparison

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Numerical Logistic Regression	0.9449	0.9290	0.9159	0.9224	0.9795
Numerical Random Forest	0.9631	0.9532	0.9428	0.9480	0.9884
Text Logistic Regression	0.9371	0.9332	0.8876	0.9098	0.9771
Text Random Forest	0.9158	0.9053	0.8536	0.8787	0.9677
Text Linear SVM	0.9406	0.9315	0.8998	0.9154	0.9781

Comparison with numerical models (Table 17) shows slightly lower predictive accuracy. However, text models provide greater explanatory value by identifying drivers such as delays, refunds, and service quality (Figure 17).

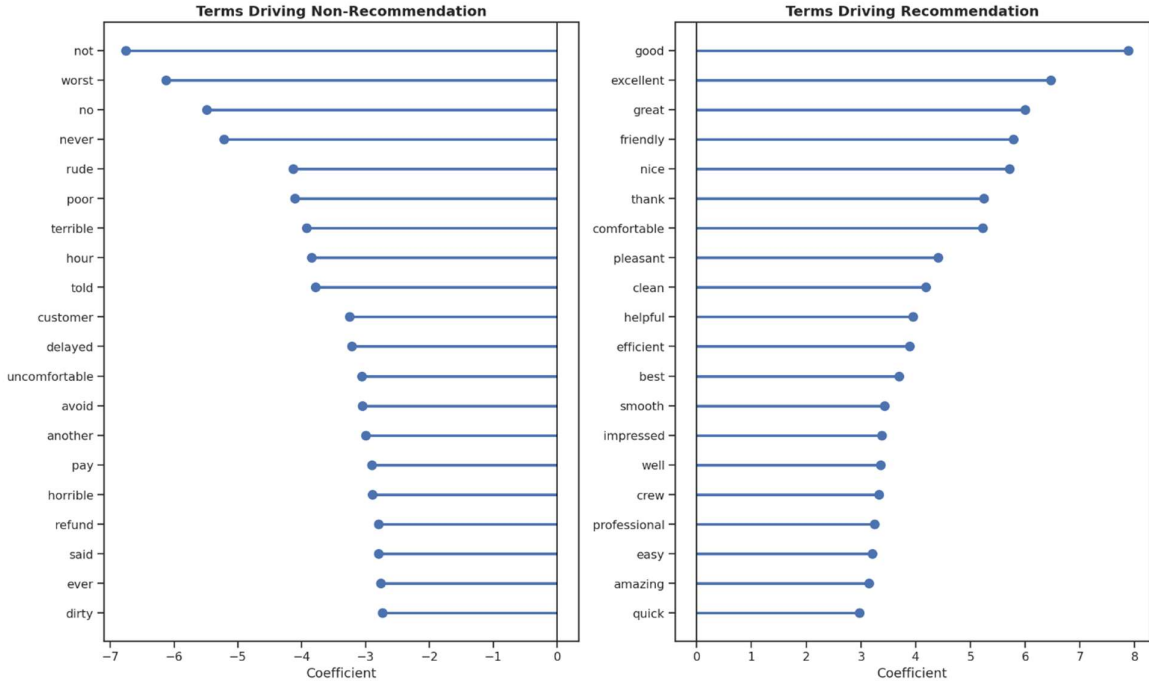


Figure 17: Key Terms Driving Recommendation and Non-Recommendation

From a managerial perspective, text models should not be used in isolation. While they diagnose service failures, decision-making should prioritise operational performance. Integrating text insights with structured data enables targeted improvements in disruption management and service recovery, directly influencing recommendation and customer loyalty.

Predictive Analytics Combining Numerical and Text Features

Two feature integration strategies were considered for combining numerical ratings with textual data. Early fusion integrates scaled numerical variables with TF-IDF features into a unified model, enabling interaction between structured evaluations and sentiment, though it may introduce redundancy where ratings already capture overall judgement (Sagi and Rokach, 2018). Late fusion preserves modality-specific strengths but limits cross-feature learning and may overlook interactions between sentiment and ratings (Zhang et al., 2023). Early fusion was selected to assess whether textual features provide incremental predictive value beyond established numerical drivers in airline contexts.

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Numerical Random Forest (Best)	0.9631	0.9532	0.9428	0.9480	0.9884
Text Logistic Regression (Best)	0.9371	0.9332	0.8876	0.9098	0.9771
Text Linear SVM	0.9406	0.9315	0.8998	0.9154	0.9781
Combined Logistic Regression	0.9578	0.9438	0.9377	0.9407	0.9863
Combined Linear SVM	0.9615	0.9536	0.9377	0.9456	0.9874
Combined Random Forest	0.9612	0.9530	0.9377	0.9453	0.9883

Table 18 Performance Comparison of Numerical, Text, and Combined Models

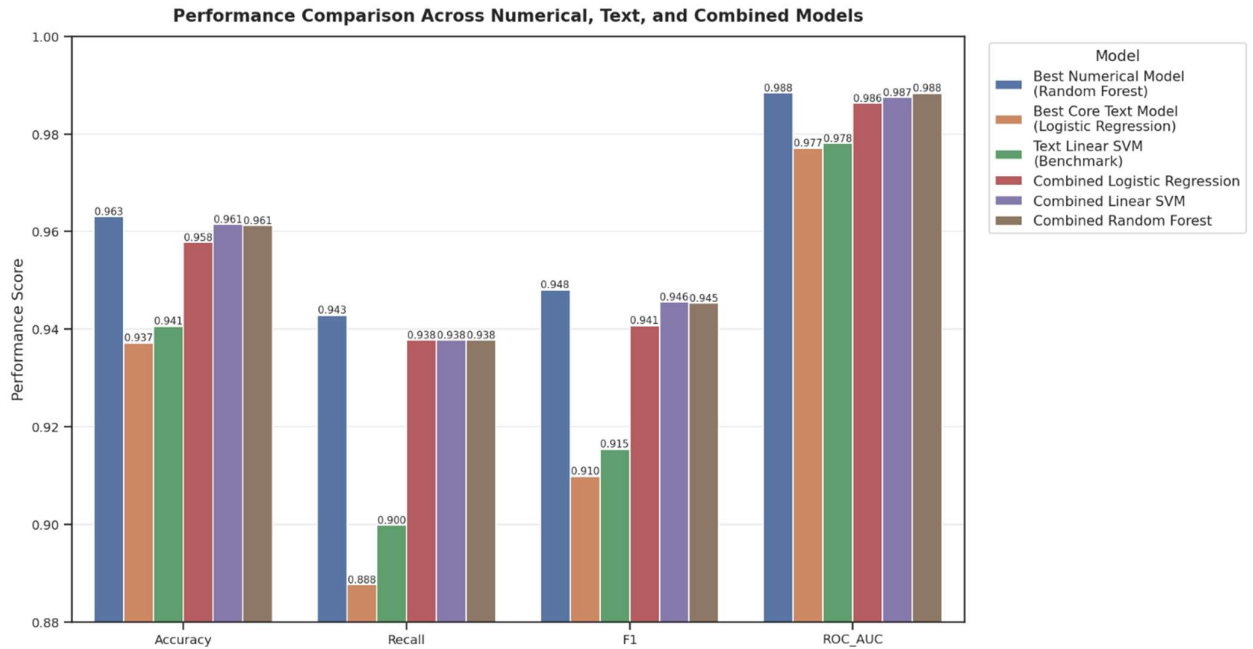


Figure 18 Predictive Performance Comparison Across Model Types

Performance results (Table 18; Figure 18) show that combined models achieve strong outcomes but do not outperform the best numerical model. The Combined Linear SVM achieves an accuracy of 0.9615 and ROC-AUC of 0.9874, remaining marginally below the numerical Random Forest baseline. However, combined models consistently outperform text-only models, particularly in recall and F1 score. This indicates that numerical ratings capture the dominant behavioural signal driving recommendation decisions, while textual features provide limited independent predictive value.

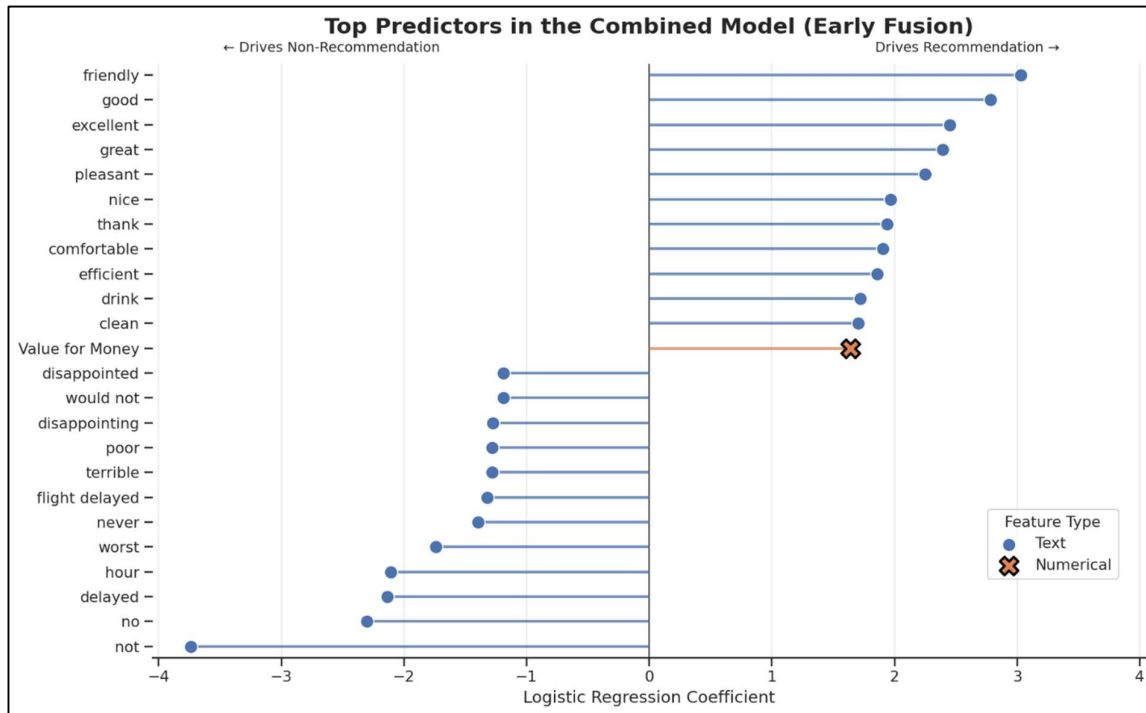


Figure 19 Top Predictors in Combined Model

Feature contribution analysis (Figure 19) reinforces this finding. Although textual variables dominate the coefficient space, Value for Money remains the only numerical feature with substantial influence. This suggests that textual features largely reflect evaluative information already embedded in structured ratings, rather than introducing new predictive signal. In airline contexts, where recommendation is driven by overall outcome rather than narrative detail, sentiment features amplify rather than redefine model decisions.

<i>Review ID</i>	<i>Actual</i>	<i>Predicted</i>	<i>Probability</i>	<i>Review Excerpt</i>	<i>Key Issue</i>
6389	Recommended	Not Recommended	0.0005	Positive assessment of staff service and overall experience despite disrupted journey	Text–rating conflict
16900	Not Recommended	Recommended	0.9992	Strongly positive evaluation of comfort, service quality, and overall flight experience	Text dominates label
3531	Recommended	Not Recommended	0.0011	Severe disruption due to flight cancellation and failure of compensation process	Behavioural inconsistency
13453	Recommended	Not Recommended	0.0029	Highly favourable description of service quality, punctuality, and onboard experience	Missing rating signal
17513	Not Recommended	Recommended	0.9951	Generally positive service experience with minor delay-related disruption	Mixed sentiment

Table 19 Selected Misclassified Cases from the Combined Model

Failure analysis (Table 19) highlights systematic misclassification patterns. Reviews with positive language but low ratings are predicted as non-recommended, while negatively framed reviews with high ratings are classified as recommended. For example, Review 6389 is misclassified despite positive service descriptions due to a disrupted journey, while Review 16900 is predicted as recommended based on strong sentiment despite a negative label. These cases demonstrate that recommendation behaviour is outcome-driven and anchored in overall evaluation rather than descriptive language. Early fusion models are therefore unable to fully resolve contradictions between sentiment and rating signals.

From a managerial perspective, these findings indicate that numerical models provide the most reliable basis for prediction, while textual features offer diagnostic value. For airlines, integrating text analytics supports identification of specific service failures, such as delays, cancellations, and customer support issues, but decision-making should prioritise improvements in core service delivery, where the greatest impact on customer recommendation is achieved.

Results and Discussion

Model evaluation across numerical, text-based, and combined approaches reveals a clear performance hierarchy. Numerical models achieve the strongest results, with Random Forest delivering the highest accuracy and ROC-AUC (Table 7), indicating that structured service ratings provide a stable signal of recommendation behaviour. Text-based models also perform well (Table 15) but with slightly lower accuracy, while combined models (Table 17) do not outperform the numerical baseline. This suggests that textual features add limited predictive value, as they largely reflect evaluative signals already captured in numerical ratings.

However, these results require critical interpretation. The dominance of variables such as Value for Money and Overall Rating raises concerns regarding construct redundancy and potential target leakage, as these features are closely aligned with the outcome variable. Strong predictive performance may therefore reflect the structure of the data rather than deeper behavioural insight. While text-based models provide explanatory value by identifying service issues such as delays and customer support failures, their contribution to prediction remains limited.

Several analytical limitations must be acknowledged. The dominance of Value for Money introduces feature bias and may obscure more complex decision processes. The analysis assumes that numerical ratings accurately reflect customer preferences, despite inconsistencies with textual sentiment. This contrasts with prior research emphasising the diagnostic value of online reviews in shaping customer decision-making (Filieri et al., 2018). While textual data provides insight into how customers interpret service experiences, this does not translate into improved predictive performance, as key signals are already captured in structured ratings. The dataset is also subject to self-selection bias, as dissatisfied customers are more likely to submit reviews, potentially overstating the importance of negative experiences. In addition, sentiment models struggle with mixed and context-dependent language, limiting their ability to capture final evaluative judgement.

The temporal relevance of these findings depends on external conditions. Core insights, including the importance of perceived value and the asymmetric impact of service failures, are likely to remain valid if the data were collected today. However, during disruption, passengers may prioritise reliability and recovery over traditional service attributes, reducing model generalisability (O’Connell and Williams, 2005). This is particularly relevant given persistent operational inefficiencies in aviation, where 72.5% of flights operate on time and impose significant costs (EUROCONTROL, 2025).

From a strategic perspective, airlines should prioritise perceived value and operational reliability, while using text analytics to diagnose service failures and guide targeted improvements.

Conclusion

This study finds that customer recommendation is explained most strongly by structured service ratings, particularly Value for Money and Overall Rating, while textual reviews add greater diagnostic than predictive value. Across the analysis, recommendation behaviour appears threshold-based and sensitive to operational failures such as delays, cancellations, refunds, and customer support problems.

The findings provide clear, data-driven priorities for improving customer recommendation. First, perceived value emerges as the dominant predictor of recommendation behaviour, with both correlation analysis and model coefficients showing a strong association between Value for Money and recommendation outcomes. The threshold effects observed in Figures 3 and 4 suggest that incremental improvements below a critical level have limited impact, whereas moving customers beyond this point is associated with a substantial increase in recommendation likelihood. This indicates that pricing, service consistency, and expectation management must be closely aligned so that customers perceive sufficient value relative to cost.

Second, the analysis highlights the disproportionate impact of operational failures. Text-based theme analysis and misclassification patterns consistently show that delays, cancellations, refunds, and customer support failures are strongly associated with non-recommendation. These findings suggest that preventing service breakdowns is more impactful than enhancing already satisfactory service elements. In practical terms, airlines should prioritise disruption management through faster recovery processes, proactive communication, and more consistent handling of compensation and support.

Finally, the persistence of rating–sentiment misalignment suggests that recommendation behaviour is outcome-driven rather than purely sentiment-driven. Airlines should therefore manage the customer journey as an integrated process, ensuring reliable delivery across all touchpoints. Those prioritising value perception and operational reliability are more likely to achieve sustained improvements in customer advocacy and long-term loyalty.

References

- Bellizzi, M.G., Eboli, L. and Mazzulla, G. (2020) “Air Transport Service Quality Factors: a Systematic Literature Review,” *Transportation research procedia (Online)*, 45, pp. 218–225. Available at: <https://doi.org/10.1016/j.trpro.2020.03.010>.
- Breiman, L. (2001) “Random forests,” *Machine learning*, 45(1), pp. 5–32. Available at: <https://doi.org/10.1023/A:1010933404324>.
- Dixon, F. (2010) “Stop Trying to Delight Your Customers,” *Harvard business review*, 1 October, pp. 23–23.
- EUROCONTROL (2025) *Performance Review Report 2024: An assessment of air traffic management in Europe*. Brussels: European Organisation for the Safety of Air Navigation. Available at: <https://www.eurocontrol.int/publication/performance-review-report-prr-2024> (Accessed: 23 April 2026).
- Filieri, R. *et al.* (2018) “Consumer perceptions of information helpfulness and determinants of purchase intention in online consumer reviews of services,” *Information & management*, 55(8), pp. 956–970. Available at: <https://doi.org/10.1016/j.im.2018.04.010>.
- Filieri, R., Hofacker, C.F. and Alguezaui, S. (2018) “What makes information in online consumer reviews diagnostic over time? The role of review relevancy, factuality, currency, source credibility and ranking score,” *Computers in human behavior*, 80, pp. 122–131. Available at: <https://doi.org/10.1016/j.chb.2017.10.039>.
- Guo, Y., Barnes, S.J. and Jia, Q. (2017) “Mining meaning from online ratings and reviews: Tourist satisfaction analysis using latent dirichlet allocation,” *Tourism management*, 59, pp. 467–483. Available at: <https://doi.org/10.1016/j.tourman.2016.09.009>.
- Han, H., Yu, J. and Kim, W. (2019) “Environmental corporate social responsibility and the strategy to boost the airline’s image and customer loyalty intentions,” *Journal of travel & tourism marketing*, 36(3), pp. 371–383. Available at: <https://doi.org/10.1080/10548408.2018.1557580>.
- Hollebeek, L.D., Srivastava, R.K. and Chen, T. (2019) ‘S-D logic–informed customer engagement: integrative framework, revised fundamental propositions, and application to CRM’, *Journal of the Academy of Marketing Science*, 47(1), pp. 161–185. <https://doi.org/10.1007/s11747-016-0494-5>
- Hutto, C.J. and Gilbert, E. (2014) ‘VADER: A parsimonious rule-based model for sentiment analysis of social media text’, *Proceedings of the International AAAI Conference on Web and Social Media (ICWSM)*, 8(1), pp. 216–225. <https://doi.org/10.1609/icwsml.v8i1.14550>
- James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021) “An Introduction to Statistical Learning: with Applications in R (2nd ed.),” New York: Springer. Available at: <https://doi.org/10.1007/978-1-0716-1418-1>.
- Joachims, T. (1998) ‘Text categorization with Support Vector Machines: Learning with many relevant features’, in Nédellec, C. and Rouveirol, C. (eds.) *Machine Learning: ECML-98*. Lecture Notes in Computer Science, vol. 1398. Berlin: Springer, pp. 137–142. Available at: <https://doi.org/10.1007/BFb0026683> (Accessed: 22 April 2026).
- Keiningham, T. *et al.* (2020) “Customer experience driven business model innovation,” *Journal of business research*, 116, pp. 431–440. Available at: <https://doi.org/10.1016/j.jbusres.2019.08.003>.

- Khan, M.H., Naseem, U., Keya, A.J., Maitra, S., Mithu, K. and Alam, M.G.R. (2023) "A systematic review on airline industries based on sentiment analysis and topic modeling," *Preprint*. Available at: <https://doi.org/10.21203/rs.3.rs-3475984/v1>.
- Kunić, T. (2019) "A Practical Guide to Sentiment Analysis," *Medijska istraživanja*, 25(1), p. 125.
- Lemon, K.N. and Verhoef, P.C. (2016) "Understanding Customer Experience Throughout the Customer Journey," *Journal of marketing*, 80(6), pp. 69–96. Available at: <https://doi.org/10.1509/jm.15.0420>.
- Liu, Y. *et al.* (2019) "RoBERTa: A Robustly Optimized BERT Pretraining Approach," *arXiv.org* [Preprint].
- Liu, Y. *et al.* (2025) "Injecting new insights: How do review sentiment and rating inconsistency shape the helpfulness of airline reviews?," *Information processing & management*, 62(4). Available at: <https://doi.org/10.1016/j.ipm.2025.104088>.
- Lucini, F.R. *et al.* (2020) "Text mining approach to explore dimensions of airline customer satisfaction using online customer reviews," *Journal of air transport management*, 83. Available at: <https://doi.org/10.1016/j.jairtraman.2019.101760>.
- Mariani, M. *et al.* (2018) "Business intelligence and big data in hospitality and tourism: a systematic literature review," *International journal of contemporary hospitality management*, 30(12), pp. 3514–3554. Available at: <https://doi.org/10.1108/IJCHM-07-2017-0461>.
- McKinsey & Company (2024) *The state of aviation*. Available at: <https://www.mckinsey.com/industries/travel/our-insights/the-state-of-aviation> (Accessed: 22 April 2026).
- Molnar, C. (2022) *Interpretable machine learning*. 2nd edn. Available at: <https://christophm.github.io/interpretable-ml-book/> (Accessed: 23 April 2026).
- O'Connell, J.F. and Williams, G. (2005) "Passengers' perceptions of low cost airlines and full service carriers: A case study involving Ryanair, Aer Lingus, Air Asia and Malaysia Airlines," *Journal of air transport management*, 11(4), pp. 259–272. Available at: <https://doi.org/10.1016/j.jairtraman.2005.01.007>.
- Parasuraman, A. (1988) "SERVQUAL: a multiple-item scale for measuring consumer perceptions of service quality," *Journal of retailing*, 64(1), pp. 12–37.
- PwC (2018) *Experience is everything: Here's how to get it right*. Available at: <https://www.pwc.com/us/en/services/consulting/library/consumer-intelligence-series/future-of-customer-experience.html> (Accessed: 23 April 2026).
- Sagi, O. and Rokach, L. (2018) "Ensemble learning: A survey," *Wiley interdisciplinary reviews. Data mining and knowledge discovery*, 8(4), pp. e1249-n/a. Available at: <https://doi.org/10.1002/widm.1249>.
- Shearer, C. (2000) The CRISP-DM Model: The New Blueprint for Data Mining. *Journal of Data Warehousing*, 5, 13-22.
- Yang, Z. *et al.* (2023) "Online Product Decision Support Using Sentiment Analysis and Fuzzy Cloud-Based Multi-Criteria Model Through Multiple E-Commerce Platforms," *IEEE transactions on fuzzy systems*, pp. 1–15. Available at: <https://doi.org/10.1109/TFUZZ.2023.3269741>.

Zhang, Y. and Yang, Q. (2022) “A Survey on Multi-Task Learning,” *IEEE transactions on knowledge and data engineering*, 34(12), pp. 5586–5609. Available at: <https://doi.org/10.1109/TKDE.2021.3070203>.

Zhang, Y., Lee, S.-Y. and Gu, Y. (2023) “A review of air transport service quality studies: current status and future research agenda,” *Journal of the Air Transport Research Society*, 1(1), pp. 9–21. Available at: <https://doi.org/10.59521/EF52BB6324BD7035>.

Appendix

Appendix A: Full Python Code

```
# =====
# ITAO7105 Data Mining
# Assignment: Descriptive, Predictive, and Text Analytics for Airline
# Customer Reviews
# Student: Aqeel Shahzad
# =====

# =====
# 0. COLAB SETUP
# =====
!pip -q install vaderSentiment transformers torch accelerate sentencepiece

# =====
# 1. IMPORTS AND GLOBAL SETTINGS
# =====
import warnings
import re
import random
from collections import Counter

import matplotlib.pyplot as plt
import nltk
import numpy as np
import pandas as pd
import seaborn as sns

from google.colab import files
from IPython.display import display

from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import CountVectorizer

warnings.filterwarnings("ignore")

pd.set_option("display.max_columns", 100)
pd.set_option("display.max_rows", 100)
pd.set_option("display.max_colwidth", None)

RANDOM_STATE = 40495557
random.seed(RANDOM_STATE)
np.random.seed(RANDOM_STATE)

sns.set_theme(style="whitegrid", context="talk")
plt.rcParams["figure.facecolor"] = "white"
plt.rcParams["axes.facecolor"] = "white"
plt.rcParams["figure.dpi"] = 140

print("Random state:", RANDOM_STATE)

# =====
# 1A. DOWNLOAD NLTK RESOURCES
# =====
nltk.download("stopwords")
nltk.download("wordnet")
nltk.download("omw-1.4")
```

```

# =====
# 3. DATA UPLOAD
# =====
uploaded = files.upload()

if len(uploaded) == 0:
    raise FileNotFoundError("No file uploaded. Please upload the airline
review CSV.")

uploaded_filenames = list(uploaded.keys())
csv_candidates = [f for f in uploaded_filenames if
f.lower().endswith(".csv")]

if len(csv_candidates) == 0:
    raise FileNotFoundError("No CSV file detected among uploaded files.")

DATA_PATH = csv_candidates[0]
print("Using CSV file:", DATA_PATH)

# =====
# 4. HELPER FUNCTIONS
# =====
def print_section(title: str) -> None:
    """Print a consistent section header for readable notebook/script
output."""
    print("\n" + "=" * 100)
    print(title)
    print("=" * 100)

def show_table(title: str, table: pd.DataFrame, index: bool = True) ->
None:
    """Display a titled table using a consistent print format."""
    print_section(title)
    if index:
        display(table)
    else:
        display(table.reset_index(drop=True))

def finalise_plot(title: str) -> None:
    """
    Apply consistent final formatting to plots so visual style stays
aligned
    across the full assignment.
    """
    ax = plt.gca()
    ax.set_title(title, fontsize=13.5, pad=12, weight="bold")
    ax.tick_params(axis="both", labelsize=10)
    plt.tight_layout()
    plt.show()

def add_bar_labels(
    bars,
    labels,
    y_offset: float,
    fontsize: int = 10,
) -> None:
    """Add value labels above bars to improve readability."""
    for bar, label in zip(bars, labels):

```

```

        plt.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + y_offset,
            label,
            ha="center",
            va="bottom",
            fontsize=fontsize,
        )

def clean_review_date(value: str) -> pd.Timestamp:
    """
    Standardise review dates by removing ordinal suffixes before parsing.
    This is needed for later temporal analysis and robustness testing.
    """
    if pd.isna(value):
        return pd.NaT

    value = str(value).strip()
    value = re.sub(r"(\d{1,2})(st|nd|rd|th)", r"\1", value)

    return pd.to_datetime(value, format="%d %B %Y", errors="coerce")

def clean_text_field(series: pd.Series) -> pd.Series:
    """
    Standardise object/string columns so text comparisons and grouping
    behave consistently across the script.
    """
    return series.astype("string").str.strip()

# =====
# 5. DATA LOADING AND VARIABLE SETUP
# =====
df = pd.read_csv(DATA_PATH)
df_raw = df.copy()

print_section("DATASET LOADED")
print(f"Shape: {df.shape}")
display(df.head())

numeric_cols = [
    "Overall_Rating",
    "Seat Comfort",
    "Cabin Staff Service",
    "Food & Beverages",
    "Ground Service",
    "Inflight Entertainment",
    "Wifi & Connectivity",
    "Value For Money",
]

categorical_cols = [
    "Airline Name",
    "Review Title",
    "Review",
    "Aircraft",
    "Type Of Traveller",
    "Seat Type",
    "Route",
]

```

```

optional_services = [
    "Wifi & Connectivity",
    "Inflight Entertainment",
    "Food & Beverages",
]

core_services = [
    "Seat Comfort",
    "Cabin Staff Service",
    "Ground Service",
]

date_cols = [
    "Review Date",
    "Date Flown",
]

# %% =====
# SECTION 3. DATA PRE-PROCESSING (DATA CLEANING AND PREPARATION)
# =====

# -----
# 3.1 For Numerical Data:
# Initial inspection and type standardisation
# -----
print_section("3.1 INITIAL DATA INSPECTION")
print(df.dtypes)
print("\nRecommended values:", df["Recommended"].unique())
print("Verified values:", df["Verified"].unique())
print("Seat Type values:", df["Seat Type"].unique()[:10])

# Standardise the binary target early
df["Recommended"] = (
    df["Recommended"]
    .astype("string")
    .str.strip()
    .str.lower()
    .map({"yes": 1, "no": 0})
    .astype("Int64")
)

# Convert numerical ratings
for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")

# Standardise key text/categorical variables
for col in categorical_cols:
    df[col] = clean_text_field(df[col])

# Parse dates
df["Review Date"] = df["Review Date"].apply(clean_review_date)
df["Date Flown"] = pd.to_datetime(
    df["Date Flown"].astype("string"),
    format="%b-%y",
    errors="coerce",
)

df["Review_Year"] = df["Review Date"].dt.year
df["Review_Month"] = df["Review Date"].dt.month
df["Flown_Year"] = df["Date Flown"].dt.year

```

```

df["Flown_Month"] = df["Date Flown"].dt.month

print_section("3.1 POST-STANDARDISATION DATA TYPES")
print(df.dtypes)

# -----
# 3.1A For Numerical Data:
# Flag reviews recorded before the reported flight month
# -----
df["Review_Before_Flight_Flag"] = (
    (df["Review Date"].notna()) &
    (df["Date Flown"].notna()) &
    (df["Review Date"] < df["Date Flown"])
).astype("Int64")

# -----
# 3.1B For Numerical Data:
# Print full reviews for cases where Review Date < Date Flown
# -----
past_review_full = df.loc[
    df["Review_Before_Flight_Flag"] == 1,
    [
        "Airline Name",
        "Review Date",
        "Date Flown",
        "Review Title",
        "Review",
        "Type Of Traveller",
        "Seat Type",
        "Route",
        "Recommended",
    ]
].copy()

past_review_full["Days_Difference"] = (
    past_review_full["Date Flown"] - past_review_full["Review Date"]
).dt.days

past_review_full = past_review_full.sort_values(
    by=["Days_Difference", "Review Date"],
    ascending=[False, True]
).reset_index(drop=True)

print_section("3.1B FULL REVIEWS: REVIEW DATE BEFORE FLIGHT DATE")

if past_review_full.empty:
    print("No cases found where the review date is earlier than the
    recorded flight date.")
else:
    for i, row in past_review_full.iterrows():
        print("\n" + "-" * 100)
        print(f"Case {i + 1}")
        print("-" * 100)
        print(f"Airline Name      : {row['Airline Name']}")
        print(f"Review Date       : {row['Review Date']}")
        print(f"Date Flown        : {row['Date Flown']}")
        print(f"Days Difference    : {row['Days_Difference']}")
        print(f"Review Title      : {row['Review Title']}")
        print(f"Traveller Type    : {row['Type Of Traveller']}")
        print(f"Seat Type         : {row['Seat Type']}")
        print(f"Route            : {row['Route']}")

```

```

        print(f"Recommended          : {row['Recommended']}")
        print("\nFull Review:")
        print(row["Review"])

# -----
# 3.1C Manual validation note
# -----
print_section("3.1C MANUAL VALIDATION NOTE")
print(
    "All cases where the review date appeared earlier than the recorded  

    flight date "  

    "were manually read and assessed. These entries were retained because  

    the review "  

    "text indicated valid complaints or pre-flight service issues, rather  

    than clear "  

    "data-entry errors."  

)

# -----
# 3.2 For Numerical Data:
# Identify missingness before cleaning
# -----
missing_before = pd.DataFrame(
    {
        "Missing_Before": df[numeric_cols].isna().sum(),
        "Percent_Before": (df[numeric_cols].isna().mean() * 100).round(2),
    }
).sort_values("Percent_Before", ascending=False)

show_table("3.2 MISSING VALUES BEFORE CLEANING", missing_before)

# -----
# 3.3 For Numerical Data:
# Examine systematic missingness in optional services
# -----
optional_summary_rows = []

for col in optional_services:
    available_mask = df[col].notna()
    optional_summary_rows.append(
        {
            "Feature": col,
            "Mean_Rating_Available_Only": round(df.loc[available_mask,
col].mean(), 2),
            "Availability_Percent": round(available_mask.mean() * 100, 2),
            "Not_Available_Percent": round((1 - available_mask.mean()) *
100, 2),
        }
    )

optional_summary = pd.DataFrame(optional_summary_rows)
show_table(
    "3.3 OPTIONAL SERVICES: QUALITY VERSUS AVAILABILITY",
    optional_summary,
    index=False,
)

# -----
# 3.4 For Numerical Data:
# Logical conflicts between Overall Rating and Recommendation
# -----

```

```

df["Inconsistent_Rating_Flag"] = (
    ((pd.to_numeric(df["Overall_Rating"], errors="coerce") <= 5) &
    (df["Recommended"] == 1)) |
    ((pd.to_numeric(df["Overall_Rating"], errors="coerce") > 5) &
    (df["Recommended"] == 0))
).astype("Int64")

inconsistency_summary = pd.DataFrame({
    "Count":
df["Inconsistent_Rating_Flag"].value_counts(dropna=False).sort_index()
})
inconsistency_summary.index = ["Consistent / Not Flagged", "Potentially
Inconsistent"]

show_table("3.4 LOGICAL CONFLICT FLAG: OVERALL RATING VS RECOMMENDATION",
inconsistency_summary)

# -----
# 3.5 For Numerical Data:
# Alternative cleaning / imputation strategies
# -----

# Strategy comparison 1:
# Dropping all rows with missing optional-service ratings
df_drop_optional = df.dropna(subset=optional_services).copy()

rec_original = df["Recommended"].value_counts(normalize=True)
rec_dropped = df_drop_optional["Recommended"].value_counts(normalize=True)

optional_strategy_summary = pd.DataFrame(
    {
        "Metric": [
            "Total Rows",
            "Rows Removed",
            "Percent Rows Removed",
            "Not Recommended Percent",
            "Recommended Percent",
        ],
        "Original Dataset": [
            f"{len(df):,}",
            "0",
            "0.00",
            f"{rec_original.get(0, 0) * 100:.2f}",
            f"{rec_original.get(1, 0) * 100:.2f}",
        ],
        "After Dropping Optional-Service Missingness": [
            f"{len(df_drop_optional):,}",
            f"{len(df) - len(df_drop_optional):,}",
            f"{((len(df) - len(df_drop_optional)) / len(df)) * 100:.2f}",
            f"{rec_dropped.get(0, 0) * 100:.2f}",
            f"{rec_dropped.get(1, 0) * 100:.2f}",
        ],
    },
)

show_table(
    "3.5 ALTERNATIVE STRATEGY 1: DROPPING OPTIONAL-SERVICE MISSINGNESS",
    optional_strategy_summary,
    index=False,
)

```

```

# Strategy comparison 2:
# Zero-imputation versus median-imputation for core services
df_zero_core = df.copy()
df_median_core = df.copy()

for col in core_services:
    df_zero_core[col] = df_zero_core[col].fillna(0)
    df_median_core[col] =
df_median_core[col].fillna(df_median_core[col].median())

core_strategy_comparison = pd.DataFrame(
    {
        "Original_Mean": df[core_services].mean().round(2),
        "Zero_Imputed_Mean": df_zero_core[core_services].mean().round(2),
        "Median_Imputed_Mean":
df_median_core[core_services].mean().round(2),
        "Original_Median": df[core_services].median().round(2),
        "Zero_Imputed_Median":
df_zero_core[core_services].median().round(2),
        "Median_Imputed_Median":
df_median_core[core_services].median().round(2),
        "Missing_Count": df[core_services].isna().sum(),
    }
)

show_table(
    "3.5 ALTERNATIVE STRATEGY 2: ZERO VERSUS MEDIAN IMPUTATION FOR CORE
SERVICES",
    core_strategy_comparison,
)

# -----
# 3.6 For Numerical Data:
# -----
# Final strategy retained:
# - Drop rows missing Overall_Rating or Value For Money
# - Treat optional-service missingness as service non-availability
# - Median-impute core service ratings

df_clean = df.copy()

df_clean = df_clean.dropna(subset=["Overall_Rating", "Value For
Money"]).copy()

for col in optional_services:
    df_clean[f"{col}_available"] = df_clean[col].notna().astype(int)
    df_clean[col] = df_clean[col].fillna(0)

for col in core_services:
    df_clean[col] = df_clean[col].fillna(df_clean[col].median())

# -----
# 3.7 For Numerical Data:
# Pre- and post-cleaning comparison
# -----
missing_after = pd.DataFrame(
    {
        "Missing_After": df_clean[numeric_cols].isna().sum(),
        "Percent_After": (df_clean[numeric_cols].isna().mean() *
100).round(2),
    }
)

```



```

).sort_values("Percent_After", ascending=False)

show_table("3.7 MISSING VALUES AFTER CLEANING", missing_after)

missing_compare = missing_before.join(missing_after)
missing_compare["Reduction"] = (
    missing_compare["Missing_Before"] - missing_compare["Missing_After"]
)
missing_compare["Percent_Reduction"] = (
    missing_compare["Percent_Before"] - missing_compare["Percent_After"]
).round(2)

show_table("3.7 BEFORE AND AFTER MISSING-VALUE COMPARISON",
missing_compare)

comparison_plot_df = pd.DataFrame(
    {
        "Original": df[core_services].mean(),
        "Zero Imputation": df_zero_core[core_services].mean(),
        "Median Imputation": df_median_core[core_services].mean(),
    }
)

comparison_plot_df.plot(kind="bar", figsize=(10, 5), edgecolor="black")
plt.ylabel("Average Rating")
plt.xlabel("Core Service Variable")
plt.xticks(rotation=0)
plt.grid(axis="y", color="#e6e6e6", linewidth=0.8)
plt.gca().set_axisbelow(True)
finalise_plot("3.7 Impact of Alternative Imputation Choices on Core Service
Ratings")

print_section("3.7 FINAL CLEANED DATASET")
print(f"Final cleaned shape: {df_clean.shape}")
print(df_clean[numeric_cols].isna().sum())

# -----
# 3.8 Cleaning audit table
# -----
audit_rows = []

def add_audit_row(
    variable: str,
    rows_before: int,
    method: str,
    rows_removed: int,
    rows_after: int,
) -> None:
    """Store a transparent record of how each cleaning decision affected
the data."""
    audit_rows.append(
        {
            "Variable_Name": variable,
            "Rows_Before": rows_before,
            "Cleaning_Method_Used": method,
            "Rows_Removed": rows_removed,
            "Rows_After": rows_after,
        }
    )

df_audit = df.copy()

```

```

rows_before = len(df_audit)
rows_removed = df_audit["Overall_Rating"].isna().sum()
df_audit = df_audit.dropna(subset=["Overall_Rating"]).copy()
add_audit_row("Overall_Rating", rows_before, "Drop missing rows",
rows_removed, len(df_audit))

rows_before = len(df_audit)
rows_removed = df_audit["Value For Money"].isna().sum()
df_audit = df_audit.dropna(subset=["Value For Money"]).copy()
add_audit_row("Value For Money", rows_before, "Drop missing rows",
rows_removed, len(df_audit))

for col in optional_services:
    rows_before = len(df_audit)
    df_audit[f"{col}_available"] = df_audit[col].notna().astype(int)
    df_audit[col] = df_audit[col].fillna(0)
    add_audit_row(
        col,
        rows_before,
        "Add availability indicator and fill missing with 0",
        0,
        len(df_audit),
    )

for col in core_services:
    rows_before = len(df_audit)
    median_value = df_audit[col].median()
    df_audit[col] = df_audit[col].fillna(median_value)
    add_audit_row(
        col,
        rows_before,
        f"Fill missing with median ({median_value:.1f})",
        0,
        len(df_audit),
    )

cleaning_audit = pd.DataFrame(audit_rows)
show_table("3.8 VARIABLE-LEVEL CLEANING AUDIT", cleaning_audit,
index=False)

# -----
# 3.9 For Textual Data:
# Create text analysis dataset
# -----
text_df = df_clean[
    [
        "Review",
        "Recommended",
        "Overall_Rating",
        "Airline Name",
        "Seat Type",
        "Type Of Traveller",
        "Route",
    ]
].dropna(subset=["Review"]).copy()

print_section("3.9 TEXT DATASET CREATED")
print(f"Text dataset shape: {text_df.shape}")

# -----

```

```

# 3.10 For Textual Data:
# Context-specific preprocessing resources
# -----
lemmatizer = WordNetLemmatizer()

base_stopwords = set(stopwords.words("english"))
negation_words = {"no", "not", "nor", "never", "cannot"}
custom_stopwords = base_stopwords - negation_words

domain_keywords = {
    "delay", "delayed", "late",
    "cancel", "cancelled", "canceled",
    "refund", "voucher",
    "staff", "crew", "service",
    "seat", "comfort",
    "food", "meal",
    "wifi", "internet",
    "baggage", "luggage", "bag",
    "boarding", "checkin", "check-in",
    "customer", "support",
}

# -----
# 3.11 For Textual Data:
# Text cleaning functions
# -----
def basic_clean_text(text: str) -> str:
    """
    Clean review text while preserving sentiment-carrying structure such as
    negation.
    """
    text = str(text).lower()
    text = re.sub(r"http\S+|www\S+", " ", text)
    text = re.sub(r"<.*?>", " ", text)

    text = text.replace("can't", "can not")
    text = text.replace("won't", "will not")
    text = text.replace("n't", " not")
    text = text.replace("'re", " are")
    text = text.replace("'ve", " have")
    text = text.replace("'ll", " will")
    text = text.replace("'d", " would")
    text = text.replace("'m", " am")

    text = re.sub(r"^[^a-z\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()

    return text

def tokenize_and_lemmatize(text: str) -> list:
    """
    Tokenise and lemmatise while preserving negation and airline-specific
    vocabulary.
    """
    tokens = text.split()
    cleaned_tokens = []

    for token in tokens:
        if token in negation_words:
            cleaned_tokens.append(token)

```

```

        elif token in domain_keywords:
            cleaned_tokens.append(lemmatizer.lemmatize(token))
        elif token not in custom_stopwords and len(token) > 2:
            cleaned_tokens.append(lemmatizer.lemmatize(token))

    return cleaned_tokens

# -----
# 3.12 For Textual Data:
# Apply preprocessing pipeline
# -----
text_df["review_clean_basic"] = text_df["Review"].apply(basic_clean_text)
text_df["review_tokens"] =
text_df["review_clean_basic"].apply(tokenize_and_lemmatize)
text_df["review_clean_final"] = text_df["review_tokens"].apply(lambda
tokens: " ".join(tokens))

print_section("3.12 SAMPLE CLEANED REVIEWS")
print(text_df[["Review", "review_clean_final"]].head(3))

# -----
# 3.13 For Textual Data:
# Dataset-specific issue themes
# -----
theme_dictionary = {
    "delay_theme": ["delay", "delayed", "late"],
    "cancellation_theme": ["cancel", "cancelled", "canceled"],
    "refund_theme": ["refund", "voucher"],
    "staff_theme": ["staff", "crew", "service"],
    "seat_theme": ["seat", "comfort"],
    "food_theme": ["food", "meal"],
    "wifi_theme": ["wifi", "internet"],
    "baggage_theme": ["baggage", "luggage", "bag"],
    "boarding_theme": ["boarding", "checkin", "check", "gate"],
    "customer_support_theme": ["customer", "support", "call", "email"],
}

for theme_name, keywords in theme_dictionary.items():
    text_df[theme_name] = text_df["review_clean_final"].apply(
        lambda text: int(any(word in text.split() for word in keywords))
    )

theme_summary = pd.DataFrame(
    {"Theme_Count": text_df[list(theme_dictionary.keys())].sum()}
).sort_values(by="Theme_Count", ascending=False)

show_table("3.13 SERVICE-THEME SUMMARY", theme_summary)

theme_by_recommendation_rows = []

for theme_name in theme_dictionary.keys():
    grouped = text_df.groupby("Recommended")[theme_name].mean() * 100
    theme_by_recommendation_rows.append(
        {
            "Theme": theme_name,
            "Not_Recommended_Percent": round(grouped.get(0, np.nan), 2),
            "Recommended_Percent": round(grouped.get(1, np.nan), 2),
            "Difference_pp": round(grouped.get(0, 0) - grouped.get(1, 0),
2),
        }
    )
)

```

```

theme_by_recommendation_df =
pd.DataFrame(theme_by_recommendation_rows).sort_values(
    "Difference_pp",
    ascending=False,
)

show_table(
    "3.13 THEME PREVALENCE BY RECOMMENDATION",
    theme_by_recommendation_df,
    index=False,
)

# -----
# 3.14 For Textual Data:
# Distinctive linguistic patterns via bigrams
# -----
vectorizer_bigrams = CountVectorizer(
    ngram_range=(2, 2),
    min_df=20,
    max_df=0.80,
)

X_bigrams = vectorizer_bigrams.fit_transform(text_df["review_clean_final"])
bigram_counts = np.asarray(X_bigrams.sum(axis=0)).ravel()

bigrams_df = pd.DataFrame(
    {
        "Bigram": vectorizer_bigrams.get_feature_names_out(),
        "Count": bigram_counts,
    }
).sort_values(by="Count", ascending=False)

show_table("3.14 TOP BIGRAMS OVERALL", bigrams_df.head(20), index=False)

not_rec_text = text_df.loc[text_df["Recommended"] == 0,
    "review_clean_final"]
rec_text = text_df.loc[text_df["Recommended"] == 1, "review_clean_final"]

vectorizer_not = CountVectorizer(ngram_range=(2, 2), min_df=10,
    max_df=0.85)
vectorizer_yes = CountVectorizer(ngram_range=(2, 2), min_df=10,
    max_df=0.85)

X_not = vectorizer_not.fit_transform(not_rec_text)
X_yes = vectorizer_yes.fit_transform(rec_text)

not_bigrams_df = pd.DataFrame(
    {
        "Bigram": vectorizer_not.get_feature_names_out(),
        "Count": np.asarray(X_not.sum(axis=0)).ravel(),
    }
).sort_values(by="Count", ascending=False)

yes_bigrams_df = pd.DataFrame(
    {
        "Bigram": vectorizer_yes.get_feature_names_out(),
        "Count": np.asarray(X_yes.sum(axis=0)).ravel(),
    }
).sort_values(by="Count", ascending=False)

```

```

show_table(
    "3.14 TOP BIGRAMS IN NOT-RECOMMENDED REVIEWS",
    not_bigrams_df.head(15),
    index=False,
)
show_table(
    "3.14 TOP BIGRAMS IN RECOMMENDED REVIEWS",
    yes_bigrams_df.head(15),
    index=False,
)

# -----
# 3.15 For Textual Data:
# Mixed and contradictory wording flags
# -----
positive_cues = {
    "good", "great", "excellent", "comfortable", "friendly",
    "helpful", "pleasant", "smooth", "nice", "amazing",
}

negative_cues = {
    "bad", "poor", "terrible", "worst", "delay",
    "cancel", "dirty", "rude", "unprofessional",
}

def count_positive_cues(tokens: list) -> int:
    return sum(1 for word in tokens if word in positive_cues)

def count_negative_cues(tokens: list) -> int:
    return sum(1 for word in tokens if word in negative_cues)

def has_negated_positive(tokens: list) -> int:
    for i in range(len(tokens) - 1):
        if tokens[i] == "not" and tokens[i + 1] in positive_cues:
            return 1
    return 0

text_df["positive_cue_count"] =
text_df["review_tokens"].apply(count_positive_cues)
text_df["negative_cue_count"] =
text_df["review_tokens"].apply(count_negative_cues)
text_df["negated_positive"] =
text_df["review_tokens"].apply(has_negated_positive)

mixed_sentiment_reviews = text_df[
    (text_df["positive_cue_count"] >= 2) &
    (text_df["negative_cue_count"] >= 2)
][["Overall_Rating", "Recommended", "Review"]].head(10)

contradiction_reviews = text_df[
    (text_df["Recommended"] == 0) &
    (text_df["positive_cue_count"] >= 2) &
    (text_df["negated_positive"] == 0)
][["Overall_Rating", "Recommended", "Review"]].head(10)

show_table("3.15 MIXED-SENTIMENT REVIEW EXAMPLES", mixed_sentiment_reviews,
index=False)
show_table(
    "3.15 POSITIVE-WORDING BUT NOT-RECOMMENDED EXAMPLES",
    contradiction_reviews,
    index=False
)

```

```

)

all_tokens = [token for tokens in text_df["review_tokens"] for token in
tokens]
token_summary = pd.DataFrame(
    Counter(all_tokens).most_common(30),
    columns=["Token", "Count"],
)
show_table("3.15 TOP MEANINGFUL TOKENS AFTER PREPROCESSING", token_summary,
index=False)

# %% =====
# SECTION 4.a. DESCRIPTIVE AND PREDICTIVE ANALYTICS (NUMERICAL FEATURES)
# =====

# -----
# 4.a.1 Descriptive statistics for all numerical features
# -----
desc_stats = (
    df_clean[numeric_cols]
    .describe()
    .T[["count", "mean", "std", "min", "25%", "50%", "75%", "max"]]
    .round(2)
)

desc_stats["missing_count"] = df_clean[numeric_cols].isna().sum()
desc_stats["missing_percent"] = (df_clean[numeric_cols].isna().mean() *
100).round(2)
desc_stats = desc_stats.reset_index().rename(columns={"index": "Feature"})

show_table("4.a.1 DESCRIPTIVE STATISTICS FOR NUMERICAL VARIABLES",
desc_stats, index=False)

# -----
# 4.a.2 Visualisation 1:
# Distribution of customer recommendation
# -----
recommend_counts = df_clean["Recommended"].value_counts().sort_index()
recommend_labels = ["No", "Yes"]
recommend_perc = (recommend_counts / recommend_counts.sum() * 100).round(1)

plt.figure(figsize=(7.6, 5.4))

bars = plt.bar(
    recommend_labels,
    recommend_counts.values,
    color=["#F08D39", "#2F5D8A"],
    edgecolor="black",
    linewidth=1.0,
    width=0.45,
)

plt.xlabel("Recommended", fontsize=11)
plt.ylabel("Number of Reviews", fontsize=11)
plt.ylim(0, recommend_counts.max() * 1.12)

plt.grid(axis="y", color="#e6e6e6", linewidth=0.8)
plt.gca().grid(False, axis="x")
plt.gca().set_axisbelow(True)

```

```

for bar, count, pct in zip(bars, recommend_counts.values,
    recommend_perc.values):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        count + recommend_counts.max() * 0.015,
        f"{count:},\n({pct:.1f}%)",
        ha="center",
        va="bottom",
        fontsize=10,
    )

ax = plt.gca()
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.spines["left"].set_linewidth(0.8)
ax.spines["bottom"].set_linewidth(0.8)
ax.spines["left"].set_color("#bdbdbd")
ax.spines["bottom"].set_color("#bdbdbd")

finalise_plot("Distribution of Customer Recommendation")

# -----
# 4.a.3 Visualisation 2:
# Consistent versus inconsistent rating-recommendation cases
# -----
df_clean["Inconsistent_Rating"] = (
    ((df_clean["Overall_Rating"] <= 5) & (df_clean["Recommended"] == 1)) |
    ((df_clean["Overall_Rating"] > 5) & (df_clean["Recommended"] == 0))
)

consistency_counts = (
    df_clean["Inconsistent_Rating"]
    .value_counts()
    .reindex([False, True], fill_value=0)
)

consistency_labels = ["Consistent", "Inconsistent"]
consistency_perc = (consistency_counts / consistency_counts.sum() *
    100).round(1)

plt.figure(figsize=(7.8, 5.4))

bars = plt.bar(
    consistency_labels,
    consistency_counts.values,
    color=["#2F5D8A", "#F08D39"],
    edgecolor="black",
    linewidth=1.0,
    width=0.45,
)

plt.xlabel("")
plt.ylabel("Number of Reviews", fontsize=11)
plt.ylim(0, consistency_counts.max() * 1.12)

plt.grid(axis="y", color="#e6e6e6", linewidth=0.8)
plt.gca().grid(False, axis="x")
plt.gca().set_axisbelow(True)

for bar, count, pct in zip(bars, consistency_counts.values,
    consistency_perc.values):

```



```

plt.text(
    bar.get_x() + bar.get_width() / 2,
    count + consistency_counts.max() * 0.015,
    f"{count:,}\n({pct:.1f}%)",
    ha="center",
    va="bottom",
    fontsize=10,
)

ax = plt.gca()
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.spines["left"].set_linewidth(0.8)
ax.spines["bottom"].set_linewidth(0.8)
ax.spines["left"].set_color("#bdbdbd")
ax.spines["bottom"].set_color("#bdbdbd")

finalise_plot("Consistent vs Inconsistent Rating-Recommendation Cases")

# -----
# 4.a.4 Visualisation 3:
# Recommendation rate by value for money
# -----
value_rec = df_clean.groupby("Value For Money")["Recommended"].mean() * 100
value_counts = df_clean["Value For Money"].value_counts().sort_index()

plt.figure(figsize=(8.8, 5.6))

plt.plot(
    value_rec.index,
    value_rec.values,
    marker="o",
    linewidth=2.2,
    markersize=7.5,
    color="#2F5D8A",
    markeredgecolor="white",
    markeredgewidth=0.8,
    zorder=3,
)

for x, y in zip(value_rec.index, value_rec.values):
    if y < 5:
        offset = 3.0
    elif y > 90:
        offset = 1.5
    else:
        offset = 2.0

    plt.text(
        x,
        y + offset,
        f"n={value_counts.get(x, 0)}",
        ha="center",
        va="bottom",
        fontsize=9,
    )

plt.ylim(0, 102)
plt.xlim(-0.25, 5.25)
plt.xlabel("Value for Money Rating", fontsize=11)
plt.ylabel("Recommendation Rate (%)", fontsize=11)

```

```

plt.xticks(sorted(df_clean["Value For Money"].dropna().unique()))

plt.grid(axis="y", color="#e9e9e9", linewidth=0.7)
plt.gca().grid(False, axis="x")
plt.gca().set_axisbelow(True)

ax = plt.gca()
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.spines["left"].set_linewidth(0.8)
ax.spines["bottom"].set_linewidth(0.8)
ax.spines["left"].set_color("#bdbdbd")
ax.spines["bottom"].set_color("#bdbdbd")

finalise_plot("Recommendation Rate by Value for Money")

# -----
# 4.a.5 Visualisation 4:
# Correlation between service rating features
# -----
corr_matrix = df_clean[numeric_cols].corr()

display_labels = [
    "Overall Rating",
    "Seat Comfort",
    "Cabin Staff Service",
    "Food & Beverages",
    "Ground Service",
    "Inflight Entertainment",
    "Wifi & Connectivity",
    "Value For Money",
]

corr_display = corr_matrix.copy()
corr_display.index = display_labels
corr_display.columns = display_labels

mask = np.triu(np.ones_like(corr_display, dtype=bool), k=1)

plt.figure(figsize=(8.8, 7.2))
ax = sns.heatmap(
    corr_display,
    mask=mask,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    vmin=-0.05,
    vmax=1.00,
    center=0.50,
    square=True,
    linewidths=0.6,
    linecolor="white",
    cbar_kws={
        "shrink": 0.82,
        "pad": 0.03,
    },
    annot_kws={"size": 8.5},
)

plt.xticks(rotation=90, fontsize=10)
plt.yticks(rotation=0, fontsize=10)

```

```

ax.grid(False)
for spine in ax.spines.values():
    spine.set_visible(False)

ax.set_facecolor("white")
cbar = ax.collections[0].colorbar
cbar.ax.tick_params(labelsize=9)

finalise_plot("Correlation Heatmap of Service Features")

# -----
# 4.a.6 Visualisation 5:
# Recommendation rate by seat type
# -----
seat_df = df_clean.dropna(subset=["Seat Type", "Recommended"]).copy()
seat_df["Seat Type"] = seat_df["Seat Type"].astype(str).str.strip()

seat_order = ["First Class", "Business Class", "Premium Economy", "Economy
Class"]

seat_summary = (
    seat_df.groupby("Seat Type", as_index=False)
    .agg(
        Recommendation_Rate=("Recommended", "mean"),
        Review_Count=("Recommended", "size"),
    )
)

seat_summary = (
    seat_summary[seat_summary["Seat Type"].isin(seat_order)]
    .assign(
        Seat_Type_Order=lambda x: pd.Categorical(
            x["Seat Type"], categories=seat_order, ordered=True
        )
    )
    .sort_values("Seat_Type_Order")
)

seat_summary["Recommendation_Rate"] = (seat_summary["Recommendation_Rate"]
* 100).round(1)

seat_colors = {
    "First Class": "#1F3A5F",
    "Business Class": "#2F5D8A",
    "Premium Economy": "#6F8FAF",
    "Economy Class": "#B7C7D9",
}

bar_colors = [seat_colors[s] for s in seat_summary["Seat Type"]]

plt.figure(figsize=(8.6, 5.6))

bars = plt.barh(
    seat_summary["Seat Type"],
    seat_summary["Recommendation_Rate"],
    color=bar_colors,
    edgecolor="black",
    linewidth=0.8,
    height=0.75,
)

```

```

plt.gca().invert_yaxis()

for bar, rate, count in zip(
    bars, seat_summary["Recommendation_Rate"], seat_summary["Review_Count"]
):
    plt.text(
        rate + 0.8,
        bar.get_y() + bar.get_height() / 2,
        f"{rate:.1f}% (n={count:,})",
        va="center",
        ha="left",
        fontsize=9,
    )

plt.xlim(0, 70)
plt.xlabel("Recommendation Rate (%)", fontsize=11)
plt.ylabel("Seat Type", fontsize=11)

plt.grid(axis="x", color="#eaeaea", linewidth=0.6)
plt.gca().grid(False, axis="y")
plt.gca().set_axisbelow(True)

ax = plt.gca()
for spine in ["top", "right"]:
    ax.spines[spine].set_visible(False)

ax.spines["left"].set_color("#c0c0c0")
ax.spines["bottom"].set_color("#c0c0c0")

finalise_plot("Recommendation Rate by Seat Type")

# -----
# 4.a.7 Visualisation 6:
# Correlation of numerical features with recommendation
# -----
corr_target = (
    df_clean[numeric_cols]
    .corrwith(df_clean["Recommended"])
    .sort_values(ascending=True)
)

display_name_map = {
    "Overall_Rating": "Overall Rating",
    "Seat Comfort": "Seat Comfort",
    "Cabin Staff Service": "Cabin Staff Service",
    "Food & Beverages": "Food & Beverages",
    "Ground Service": "Ground Service",
    "Inflight Entertainment": "Inflight Entertainment",
    "Wifi & Connectivity": "Wifi & Connectivity",
    "Value For Money": "Value For Money",
}

corr_plot = corr_target.rename(index=display_name_map)

cmap = plt.cm.Blues
norm = plt.Normalize(corr_plot.min(), corr_plot.max())
colors = cmap(0.20 + 0.78 * norm(corr_plot.values))

plt.figure(figsize=(8.8, 5.8))

```

```

bars = plt.barh(
    corr_plot.index,
    corr_plot.values,
    color=colors,
    edgecolor="black",
    linewidth=0.8,
    height=0.76,
)

for bar, value in zip(bars, corr_plot.values):
    plt.text(
        value + 0.012,
        bar.get_y() + bar.get_height() / 2,
        f"{value:.2f}",
        va="center",
        ha="left",
        fontsize=9,
    )

plt.xlim(0, 0.89)
plt.xlabel("Correlation with Recommendation", fontsize=11)
plt.ylabel("")

plt.grid(axis="x", color="#eaeaea", linewidth=0.6)
plt.gca().grid(False, axis="y")
plt.gca().set_axisbelow(True)

ax = plt.gca()
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.spines["left"].set_color("#c0c0c0")
ax.spines["bottom"].set_color("#c0c0c0")
ax.spines["left"].set_linewidth(0.8)
ax.spines["bottom"].set_linewidth(0.8)

finalise_plot("Correlation of Numerical Features with Recommendation")

# -----
# 4.a.8 Visualisation 7:
# Recommendation rate by overall rating
# -----
rating_rec = df_clean.groupby("Overall_Rating")["Recommended"].mean() * 100
rating_counts = df_clean["Overall_Rating"].value_counts().sort_index()

plt.figure(figsize=(8.8, 5.6))

plt.plot(
    rating_rec.index,
    rating_rec.values,
    marker="o",
    linewidth=2.2,
    markersize=7,
    color="#2F5D8A",
    markeredgecolor="white",
    markeredgewidth=0.8,
    zorder=3,
)

for x, y in zip(rating_rec.index, rating_rec.values):
    if y < 10:
        offset = 3.2

```

```

elif y > 90:
    offset = 1.5
else:
    offset = 2.2

plt.text(
    x,
    y + offset,
    f"n={rating_counts.get(x, 0):,}",
    ha="center",
    va="bottom",
    fontsize=8.5,
    color="#444444",
)

plt.ylim(0, 102)
plt.xlim(0.6, 9.4)
plt.xlabel("Overall Rating", fontsize=11)
plt.ylabel("Recommendation Rate (%)", fontsize=11)
plt.xticks(range(1, 10))

plt.grid(axis="y", color="#e9e9e9", linewidth=0.7)
plt.gca().grid(False, axis="x")
plt.gca().set_axisbelow(True)

ax = plt.gca()
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.spines["left"].set_color("#bdbdbd")
ax.spines["bottom"].set_color("#bdbdbd")

finalise_plot("Recommendation Rate by Overall Rating")

# -----
# 4.a.9 Imports for predictive analytics
# -----
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
    roc_curve,
)
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# -----
# 4.a.10 Modelling helper functions
# -----
def evaluate_model(y_true, y_pred, y_prob, model_name: str) -> dict:
    """
    Evaluate a binary classifier using standard metrics.
    """
    results = {
        "Model": model_name,

```

```

        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred, zero_division=0),
        "Recall": recall_score(y_true, y_pred, zero_division=0),
        "F1": f1_score(y_true, y_pred, zero_division=0),
        "ROC_AUC": roc_auc_score(y_true, y_prob),
    }

    print_section(f"MODEL PERFORMANCE: {model_name}")
    for metric, value in results.items():
        if metric != "Model":
            print(f"{metric}: {value:.4f}")

    print("\nClassification Report:")
    print(classification_report(y_true, y_pred, zero_division=0))
    return results

def evaluate_at_threshold(y_true, y_prob, threshold: float, model_name:
str) -> dict:
    """
    Evaluate performance at a specified threshold.
    """
    y_pred = (y_prob >= threshold).astype(int)

    results = {
        "Model": model_name,
        "Threshold": threshold,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred, zero_division=0),
        "Recall": recall_score(y_true, y_pred, zero_division=0),
        "F1": f1_score(y_true, y_pred, zero_division=0),
    }

    print_section(f"THRESHOLD EVALUATION: {model_name} |
threshold={threshold}")
    for metric, value in results.items():
        if metric not in ["Model", "Threshold"]:
            print(f"{metric}: {value:.4f}")

    return results

def print_conf_matrix(y_true, y_pred, model_name: str) -> pd.DataFrame:
    """Print a labelled confusion matrix."""
    cm = confusion_matrix(y_true, y_pred)
    cm_df = pd.DataFrame(
        cm,
        index=["Actual 0", "Actual 1"],
        columns=["Predicted 0", "Predicted 1"],
    )
    show_table(f"CONFUSION MATRIX: {model_name}", cm_df)
    return cm_df

def format_results_table(results_list: list, labels: list, title: str) ->
pd.DataFrame:
    """
    Convert a list of results dictionaries into one clean comparison table.
    """
    results_df = pd.DataFrame(results_list).copy()
    results_df["Model"] = labels

```

```

        results_df = (
            results_df
            .drop(columns=["Threshold"], errors="ignore")
            .set_index("Model")
            .round(4)
        )
        if "ROC_AUC" in results_df.columns:
            results_df["ROC_AUC"] = results_df["ROC_AUC"].fillna("-")
        show_table(title, results_df)
        return results_df

# -----
# 4.a.11 Modelling dataset and random split
# -----
model_features = [
    "Overall_Rating",
    "Seat Comfort",
    "Cabin Staff Service",
    "Food & Beverages",
    "Ground Service",
    "Inflight Entertainment",
    "Wifi & Connectivity",
    "Value For Money",
    "Wifi & Connectivity_available",
    "Inflight Entertainment_available",
    "Food & Beverages_available",
]

target = "Recommended"

model_df = df_clean[model_features + [target, "Review Date", "Airline
Name"]].copy()

print_section("4.a.11 MODELLING DATASET")
print("Shape:", model_df.shape)
print(model_df[target].value_counts(normalize=True).round(4))

X = model_df[model_features]
y = model_df[target]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y,
)

print_section("4.a.11 RANDOM TRAIN-TEST SPLIT")
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print(y_train.value_counts(normalize=True).round(4))
print(y_test.value_counts(normalize=True).round(4))

# -----
# 4.a.12 Logistic Regression model
# -----
log_pipeline = Pipeline(
    [
        ("scaler", StandardScaler()),
        (

```



```

        "model",
        LogisticRegression(
            max_iter=1000,
            random_state=RANDOM_STATE,
        ),
    ],
)

log_pipeline.fit(X_train, y_train)

log_pred = log_pipeline.predict(X_test)
log_prob = log_pipeline.predict_proba(X_test)[:, 1]

log_results = evaluate_model(
    y_test,
    log_pred,
    log_prob,
    "Logistic Regression",
)

# -----
# 4.a.13 Random Forest model
# -----
rf_model = RandomForestClassifier(
    n_estimators=200,
    random_state=RANDOM_STATE,
)

rf_model.fit(X_train, y_train)

rf_pred = rf_model.predict(X_test)
rf_prob = rf_model.predict_proba(X_test)[:, 1]

rf_results = evaluate_model(
    y_test,
    rf_pred,
    rf_prob,
    "Random Forest",
)

# -----
# 4.a.14 Threshold comparison
# -----
log_threshold_03 = evaluate_at_threshold(
    y_test,
    log_prob,
    0.30,
    "Logistic Regression",
)

rf_threshold_03 = evaluate_at_threshold(
    y_test,
    rf_prob,
    0.30,
    "Random Forest",
)

log_pred_03 = (log_prob >= 0.30).astype(int)
rf_pred_03 = (rf_prob >= 0.30).astype(int)

```

```

baseline_comparison_full = format_results_table(
    results_list=[log_results, rf_results, log_threshold_03,
rf_threshold_03],
    labels=[
        "Logistic Regression (0.5)",
        "Random Forest (0.5)",
        "Logistic Regression (0.3)",
        "Random Forest (0.3)",
    ],
    title="4.a.14 BASELINE MODEL COMPARISON",
)

# -----
# 4.a.15 LASSO Logistic Regression for feature selection
# -----
lasso_pipeline = Pipeline(
    [
        ("scaler", StandardScaler()),
        (
            "model",
            LogisticRegression(
                penalty="l1",
                solver="liblinear",
                max_iter=1000,
                random_state=RANDOM_STATE,
            ),
        ),
    ]
)

lasso_param_grid = {
    "model__C": [0.01, 0.1, 1, 10],
}

lasso_grid = GridSearchCV(
    estimator=lasso_pipeline,
    param_grid=lasso_param_grid,
    scoring="roc_auc",
    cv=5,
    n_jobs=-1,
)

lasso_grid.fit(X_train, y_train)

best_lasso = lasso_grid.best_estimator_

print_section("4.a.15 LASSO TUNING RESULTS")
print("Best parameters:", lasso_grid.best_params_)
print("Best CV ROC-AUC:", round(lasso_grid.best_score_, 4))

lasso_coef = best_lasso.named_steps["model"].coef_[0]

lasso_features = pd.DataFrame(
    {
        "Feature": model_features,
        "Coefficient": lasso_coef,
    }
)

selected_features = (
    lasso_features[lasso_features["Coefficient"] != 0]

```

```

        .copy()
        .sort_values(by="Coefficient", key=np.abs, ascending=False)
    )

show_table("4.a.15 FEATURES SELECTED BY LASSO", selected_features,
index=False)

lasso_pred = best_lasso.predict(X_test)
lasso_prob = best_lasso.predict_proba(X_test)[:, 1]

lasso_results = evaluate_model(
    y_test,
    lasso_pred,
    lasso_prob,
    "LASSO Logistic Regression",
)

lasso_threshold_03 = evaluate_at_threshold(
    y_test,
    lasso_prob,
    0.30,
    "LASSO Logistic Regression",
)

lasso_pred_03 = (lasso_prob >= 0.30).astype(int)

all_model_results = format_results_table(
    results_list=[
        log_results,
        rf_results,
        lasso_results,
        log_threshold_03,
        rf_threshold_03,
        lasso_threshold_03,
    ],
    labels=[
        "Logistic Regression (0.5)",
        "Random Forest (0.5)",
        "LASSO Logistic Regression (0.5)",
        "Logistic Regression (0.3)",
        "Random Forest (0.3)",
        "LASSO Logistic Regression (0.3)",
    ],
    title="4.a.15 MODEL COMPARISON INCLUDING LASSO",
)

# -----
# 4.a.16 Confusion matrices
# -----
print_conf_matrix(y_test, log_pred, "Logistic Regression (0.5)")
print_conf_matrix(y_test, rf_pred, "Random Forest (0.5)")
print_conf_matrix(y_test, lasso_pred, "LASSO Logistic Regression (0.5)")
print_conf_matrix(y_test, log_pred_03, "Logistic Regression (0.3)")
print_conf_matrix(y_test, rf_pred_03, "Random Forest (0.3)")
print_conf_matrix(y_test, lasso_pred_03, "LASSO Logistic Regression (0.3)")

# -----
# 4.a.17 Hyperparameter tuning
# -----
log_tuned_pipeline = Pipeline(
    [

```

```

        ("scaler", StandardScaler()),
        (
            "model",
            LogisticRegression(
                max_iter=2000,
                random_state=RANDOM_STATE,
            ),
        ),
    ]
)

log_param_grid = {
    "model__C": [0.01, 0.1, 1, 10],
    "model__class_weight": [None, "balanced"],
}

log_grid = GridSearchCV(
    estimator=log_tuned_pipeline,
    param_grid=log_param_grid,
    scoring="roc_auc",
    cv=5,
    n_jobs=-1,
)

log_grid.fit(X_train, y_train)

best_log_model = log_grid.best_estimator_

print_section("4.a.17 TUNED LOGISTIC REGRESSION RESULTS")
print("Best parameters:", log_grid.best_params_)
print("Best CV ROC-AUC:", round(log_grid.best_score_, 4))

log_tuned_pred = best_log_model.predict(X_test)
log_tuned_prob = best_log_model.predict_proba(X_test)[:, 1]

log_tuned_results = evaluate_model(
    y_test,
    log_tuned_pred,
    log_tuned_prob,
    "Tuned Logistic Regression",
)

log_tuned_threshold_03 = evaluate_at_threshold(
    y_test,
    log_tuned_prob,
    0.30,
    "Tuned Logistic Regression",
)

log_tuned_pred_03 = (log_tuned_prob >= 0.30).astype(int)

rf_param_grid = {
    "n_estimators": [200, 300],
    "max_depth": [None, 10, 20],
    "min_samples_leaf": [1, 3, 5],
    "class_weight": [None, "balanced"],
}

rf_grid = GridSearchCV(
    estimator=RandomForestClassifier(random_state=RANDOM_STATE),
    param_grid=rf_param_grid,

```

```

        scoring="roc_auc",
        cv=5,
        n_jobs=-1,
    )

    rf_grid.fit(X_train, y_train)

    best_rf_model = rf_grid.best_estimator_

    print_section("4.a.17 TUNED RANDOM FOREST RESULTS")
    print("Best parameters:", rf_grid.best_params_)
    print("Best CV ROC-AUC:", round(rf_grid.best_score_, 4))

    rf_tuned_pred = best_rf_model.predict(X_test)
    rf_tuned_prob = best_rf_model.predict_proba(X_test)[: , 1]

    rf_tuned_results = evaluate_model(
        y_test,
        rf_tuned_pred,
        rf_tuned_prob,
        "Tuned Random Forest",
    )

    rf_tuned_threshold_03 = evaluate_at_threshold(
        y_test,
        rf_tuned_prob,
        0.30,
        "Tuned Random Forest",
    )

    rf_tuned_pred_03 = (rf_tuned_prob >= 0.30).astype(int)

    tuned_model_results = format_results_table(
        results_list=[
            log_tuned_results,
            rf_tuned_results,
            log_tuned_threshold_03,
            rf_tuned_threshold_03,
        ],
        labels=[
            "Tuned Logistic Regression (0.5)",
            "Tuned Random Forest (0.5)",
            "Tuned Logistic Regression (0.3)",
            "Tuned Random Forest (0.3)",
        ],
        title="4.a.17 TUNED MODEL COMPARISON",
    )

    baseline_vs_tuned = format_results_table(
        results_list=[
            log_results,
            rf_results,
            log_tuned_results,
            rf_tuned_results,
        ],
        labels=[
            "Baseline Logistic Regression",
            "Baseline Random Forest",
            "Tuned Logistic Regression",
            "Tuned Random Forest",
        ],
    ),

```

```

        title="4.a.17 BASELINE VERSUS TUNED MODELS",
    )

    print_conf_matrix(y_test, log_tuned_pred, "Tuned Logistic Regression
(0.5)")
    print_conf_matrix(y_test, rf_tuned_pred, "Tuned Random Forest (0.5)")
    print_conf_matrix(y_test, log_tuned_pred_03, "Tuned Logistic Regression
(0.3)")
    print_conf_matrix(y_test, rf_tuned_pred_03, "Tuned Random Forest (0.3)")

    # -----
    # 4.a.18 Model robustness / stress test:
    # Time-based split
    # -----
    time_df = model_df.dropna(subset=["Review Date"]).copy()
    time_df = time_df.sort_values("Review Date").reset_index(drop=True)

    split_index = int(len(time_df) * 0.80)

    train_time = time_df.iloc[:split_index].copy()
    test_time = time_df.iloc[split_index:].copy()

    X_train_time = train_time[model_features]
    y_train_time = train_time[target]

    X_test_time = test_time[model_features]
    y_test_time = test_time[target]

    print_section("4.a.18 TIME-BASED TRAIN-TEST SPLIT")
    print("Training rows:", len(train_time))
    print("Testing rows:", len(test_time))
    print("Training period:", train_time["Review Date"].min(), "to",
train_time["Review Date"].max())
    print("Testing period :", test_time["Review Date"].min(), "to",
test_time["Review Date"].max())

    log_time_model = Pipeline(
        [
            ("scaler", StandardScaler()),
            (
                "model",
                LogisticRegression(
                    max_iter=1000,
                    random_state=RANDOM_STATE,
                ),
            ),
        ]
    )

    log_time_model.fit(X_train_time, y_train_time)

    log_time_pred = log_time_model.predict(X_test_time)
    log_time_prob = log_time_model.predict_proba(X_test_time)[: , 1]

    log_time_results = evaluate_model(
        y_test_time,
        log_time_pred,
        log_time_prob,
        "Logistic Regression (Time-Based Split)",
    )

```

```

log_time_threshold_03 = evaluate_at_threshold(
    y_test_time,
    log_time_prob,
    0.30,
    "Logistic Regression (Time-Based Split)",
)

log_time_pred_03 = (log_time_prob >= 0.30).astype(int)

rf_time_model = RandomForestClassifier(
    n_estimators=200,
    random_state=RANDOM_STATE,
)

rf_time_model.fit(X_train_time, y_train_time)

rf_time_pred = rf_time_model.predict(X_test_time)
rf_time_prob = rf_time_model.predict_proba(X_test_time)[: , 1]

rf_time_results = evaluate_model(
    y_test_time,
    rf_time_pred,
    rf_time_prob,
    "Random Forest (Time-Based Split)",
)

rf_time_threshold_03 = evaluate_at_threshold(
    y_test_time,
    rf_time_prob,
    0.30,
    "Random Forest (Time-Based Split)",
)

rf_time_pred_03 = (rf_time_prob >= 0.30).astype(int)

lasso_time_model = Pipeline(
    [
        ("scaler", StandardScaler()),
        (
            "model",
            LogisticRegression(
                penalty="l1",
                solver="liblinear",
                max_iter=1000,
                random_state=RANDOM_STATE,
                C=lasso_grid.best_params_["model__C"],
            ),
        ),
    ],
)

lasso_time_model.fit(X_train_time, y_train_time)

lasso_time_pred = lasso_time_model.predict(X_test_time)
lasso_time_prob = lasso_time_model.predict_proba(X_test_time)[: , 1]

lasso_time_results = evaluate_model(
    y_test_time,
    lasso_time_pred,
    lasso_time_prob,
    "LASSO Logistic Regression (Time-Based Split)",
)

```

```

)

lasso_time_threshold_03 = evaluate_at_threshold(
    y_test_time,
    lasso_time_prob,
    0.30,
    "LASSO Logistic Regression (Time-Based Split)",
)

lasso_time_pred_03 = (lasso_time_prob >= 0.30).astype(int)

time_based_results = format_results_table(
    results_list=[
        log_time_results,
        rf_time_results,
        lasso_time_results,
        log_time_threshold_03,
        rf_time_threshold_03,
        lasso_time_threshold_03,
    ],
    labels=[
        "Logistic Regression Time (0.5)",
        "Random Forest Time (0.5)",
        "LASSO Logistic Regression Time (0.5)",
        "Logistic Regression Time (0.3)",
        "Random Forest Time (0.3)",
        "LASSO Logistic Regression Time (0.3)",
    ],
    title="4.a.18 TIME-BASED MODEL COMPARISON",
)

robustness_comparison = format_results_table(
    results_list=[
        log_results,
        rf_results,
        lasso_results,
        log_time_results,
        rf_time_results,
        lasso_time_results,
    ],
    labels=[
        "Logistic Regression (Random Split)",
        "Random Forest (Random Split)",
        "LASSO Logistic Regression (Random Split)",
        "Logistic Regression (Time-Based Split)",
        "Random Forest (Time-Based Split)",
        "LASSO Logistic Regression (Time-Based Split)",
    ],
    title="4.a.18 RANDOM SPLIT VERSUS TIME-BASED SPLIT",
)

print_conf_matrix(y_test_time, log_time_pred, "Logistic Regression Time (0.5)")
print_conf_matrix(y_test_time, rf_time_pred, "Random Forest Time (0.5)")
print_conf_matrix(y_test_time, lasso_time_pred, "LASSO Logistic Regression Time (0.5)")
print_conf_matrix(y_test_time, log_time_pred_03, "Logistic Regression Time (0.3)")
print_conf_matrix(y_test_time, rf_time_pred_03, "Random Forest Time (0.3)")
print_conf_matrix(y_test_time, lasso_time_pred_03, "LASSO Logistic Regression Time (0.3)")

```



```

# ROC coordinates
fpr_log, tpr_log, _ = roc_curve(y_test, log_prob)
fpr_rf, tpr_rf, _ = roc_curve(y_test, rf_prob)

# AUC values
auc_log = roc_auc_score(y_test, log_prob)
auc_rf = roc_auc_score(y_test, rf_prob)

plt.figure(figsize=(8.8, 5.6))

plt.plot(
    fpr_log,
    tpr_log,
    linewidth=2.2,
    color="#2F5D8A",
    label=f"Logistic Regression (ROC-AUC = {auc_log:.3f})",
    zorder=3,
)

plt.plot(
    fpr_rf,
    tpr_rf,
    linewidth=2.2,
    color="#F08D39",
    label=f"Random Forest (ROC-AUC = {auc_rf:.3f})",
    zorder=3,
)

plt.xlim(0, 1)
plt.ylim(0, 1.02)
plt.xlabel("False Positive Rate", fontsize=11)
plt.ylabel("True Positive Rate", fontsize=11)

plt.grid(axis="y", color="#e9e9e9", linewidth=0.7)
plt.gca().grid(False, axis="x")
plt.gca().set_axisbelow(True)

ax = plt.gca()
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.spines["left"].set_color("#bdbdbd")
ax.spines["bottom"].set_color("#bdbdbd")

plt.legend(
    loc="lower right",
    frameon=True,
    facecolor="white",
    edgecolor="#cfcfcf",
    framealpha=0.96,
    fontsize=10,
    title="Models",
    title_fontsize=10,
    borderpad=0.8,
    labelspacing=0.6,
)

finalise_plot("ROC Curve Comparison for Predicting Customer
Recommendation")

# -----

```

```

# 4.b.1 Imports and text-modelling helpers
# -----
from matplotlib.lines import Line2D
from scipy.sparse import csr_matrix, hstack
from sklearn.calibration import CalibratedClassifierCV
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import roc_curve
from sklearn.svm import LinearSVC
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

import torch
from transformers import pipeline

VADER_POS_THRESHOLD = 0.05
VADER_NEG_THRESHOLD = -0.05
ROBERTA_POS_THRESHOLD = 0.60
ROBERTA_NEG_THRESHOLD = 0.60

# -----
# 4.b.2 VADER sentiment extraction
# -----
analyzer = SentimentIntensityAnalyzer()

def vader_sentiment_score(text):
    """
    Compute the VADER compound score.
    """
    if pd.isna(text):
        return np.nan
    return analyzer.polarity_scores(str(text))["compound"]

def vader_sentiment_label(score):
    """
    Convert continuous VADER score into categorical label.
    """
    if pd.isna(score):
        return np.nan
    if score >= VADER_POS_THRESHOLD:
        return "Positive"
    if score <= VADER_NEG_THRESHOLD:
        return "Negative"
    return "Neutral"

text_df = text_df.copy()
text_df["vader_score"] = text_df["Review"].apply(vader_sentiment_score)
text_df["vader_label"] =
text_df["vader_score"].apply(vader_sentiment_label)
text_df["Review_ID"] = text_df.index

print_section("4.b.2 VADER SENTIMENT DISTRIBUTION")
print(text_df["vader_label"].value_counts(dropna=False))

# -----
# 4.b.3 RoBERTa sentiment extraction
# -----
import os
import math

device = -1

```

```

print("RoBERTa device: CPU")

roberta_pipe = pipeline(
    "sentiment-analysis",
    model="cardiffnlp/twitter-roberta-base-sentiment-latest",
    tokenizer="cardiffnlp/twitter-roberta-base-sentiment-latest",
    device=device
)

def map_roberta_output(label: str, score: float) -> str:
    if pd.isna(label):
        return np.nan

    label_clean = str(label).strip().lower()

    if label_clean in ["negative", "label_0"]:
        return "Negative"
    if label_clean in ["neutral", "label_1"]:
        return "Neutral"
    if label_clean in ["positive", "label_2"]:
        return "Positive"

    if "neg" in label_clean:
        return "Negative"
    if "neu" in label_clean or "mixed" in label_clean:
        return "Neutral"
    if "pos" in label_clean:
        return "Positive"

    return "Neutral"

def run_roberta_in_batches_with_checkpoint(
    df_input,
    text_col="Review",
    id_col="Review_ID",
    batch_size=8,
    max_length=96,
    progress_every=20,
    checkpoint_every=50,
    checkpoint_path="roberta_full_checkpoint.csv",
    final_output_path="text_df_roberta_full_scored.csv"
):
    """
    Run RoBERTa on the full dataset with checkpointing and resume support.
    """

    df_work = df_input[[id_col, text_col]].copy()
    df_work[text_col] = df_work[text_col].fillna("").astype(str)

    # Resume if checkpoint exists
    if os.path.exists(checkpoint_path):
        scored_df = pd.read_csv(checkpoint_path)
        done_ids = set(scored_df[id_col].tolist())
        df_remaining = df_work[~df_work[id_col].isin(done_ids)].copy()

        print(f"Checkpoint found: {checkpoint_path}")
        print(f"Already scored: {len(scored_df):,}")
        print(f"Remaining: {len(df_remaining):,}")
    else:
        scored_df = pd.DataFrame(columns=[

```

```

        id_col,
        "roberta_raw_label",
        "roberta_confidence",
        "roberta_label"
    ])
    df_remaining = df_work.copy()
    print("No checkpoint found. Starting full run from scratch.")

    texts = df_remaining[text_col].tolist()
    ids = df_remaining[id_col].tolist()

    total_batches = math.ceil(len(texts) / batch_size)
    batch_counter = 0

    new_rows = []

    for start in range(0, len(texts), batch_size):
        batch_texts = texts[start:start + batch_size]
        batch_ids = ids[start:start + batch_size]

        outputs = roberta_pipe(
            batch_texts,
            truncation=True,
            padding=True,
            max_length=max_length
        )

        for rid, out in zip(batch_ids, outputs):
            raw_label = out["label"]
            conf = out["score"]
            mapped_label = map_roberta_output(raw_label, conf)

            new_rows.append({
                id_col: rid,
                "roberta_raw_label": raw_label,
                "roberta_confidence": conf,
                "roberta_label": mapped_label
            })

        batch_counter += 1

        if batch_counter % progress_every == 0 or batch_counter ==
total_batches:
            print(f"Processed batch {batch_counter}/{total_batches}")

            if batch_counter % checkpoint_every == 0:
                temp_df = pd.concat([scored_df, pd.DataFrame(new_rows)],
ignore_index=True)
                temp_df.to_csv(checkpoint_path, index=False)
                print(f"Checkpoint saved at batch {batch_counter}:
{checkpoint_path}")

            # Final save
            final_scored = pd.concat([scored_df, pd.DataFrame(new_rows)],
ignore_index=True)
            final_scored.to_csv(checkpoint_path, index=False)
            final_scored.to_csv(final_output_path, index=False)

            print(f"Final checkpoint saved: {checkpoint_path}")
            print(f"Final output saved: {final_output_path}")

```

```

    return final_scored

# Ensure Review_ID exists
if "Review_ID" not in text_df.columns:
    text_df = text_df.copy()
    text_df["Review_ID"] = text_df.index

# Run on FULL DATASET
roberta_full_results = run_roberta_in_batches_with_checkpoint(
    df_input=text_df,
    text_col="Review",
    id_col="Review_ID",
    batch_size=8,
    max_length=96,
    progress_every=20,
    checkpoint_every=50,
    checkpoint_path="roberta_full_checkpoint.csv",
    final_output_path="text_df_roberta_full_scored.csv"
)

# Merge results back into text_df
text_df = text_df.drop(
    columns=["roberta_raw_label", "roberta_confidence", "roberta_label",
"roberta_score_signed"],
    errors="ignore"
).merge(
    roberta_full_results,
    on="Review_ID",
    how="left"
)

# Signed score for later visualisation alignment with ratings
roberta_score_map = {"Negative": -1, "Neutral": 0, "Positive": 1}
text_df["roberta_score_signed"] = (
    text_df["roberta_label"].map(roberta_score_map) *
text_df["roberta_confidence"]
)

print_section("4.b.3 ROBERTA SENTIMENT DISTRIBUTION (FULL DATASET)")
print(text_df["roberta_label"].value_counts(dropna=False))

# To Upload text_df_roberta_full_scored.csv (takes 2 hours to run - Can
Skip Re-run with file Upload)
from google.colab import files
uploaded = files.upload()

roberta_full = pd.read_csv("text_df_roberta_full_scored.csv")
print(roberta_full.shape)
roberta_full.head()

roberta_full = pd.read_csv("text_df_roberta_full_scored.csv")

roberta_cols = [
    "Review_ID",
    "roberta_raw_label",
    "roberta_confidence",
    "roberta_label"
]

text_df = text_df.drop(

```

```

        columns=[
            "roberta_raw_label",
            "roberta_confidence",
            "roberta_label",
            "roberta_score_signed"
        ],
        errors="ignore"
    ).merge(
        roberta_full[roberta_cols],
        on="Review_ID",
        how="left"
    )

    roberta_score_map = {"Negative": -1, "Neutral": 0, "Positive": 1}

    text_df["roberta_score_signed"] = (
        text_df["roberta_label"].map(roberta_score_map)
        * text_df["roberta_confidence"]
    )

    text_df_compare = text_df.dropna(subset=["roberta_label"]).copy()

    print(text_df_compare.shape)
    print(text_df_compare["roberta_label"].value_counts(dropna=False))

    # -----
    # 4.b.4 Sentiment label alignment with recommendation
    # -----
    def sentiment_recommendation_summary(df_input, label_col, title):
        summary = (
            df_input.groupby(label_col, as_index=False)
            .agg(
                Total_Reviews=("Recommended", "size"),
                Recommended_Count=("Recommended", "sum")
            )
        )

        summary["Not_Recommended_Count"] = (
            summary["Total_Reviews"] - summary["Recommended_Count"]
        )
        summary["Recommendation_Rate"] = (
            summary["Recommended_Count"] / summary["Total_Reviews"] * 100
        ).round(1)

        summary = summary.rename(columns={label_col: "Sentiment_Label"})
        show_table(title, summary, index=False)
        return summary

    vader_rec_summary = sentiment_recommendation_summary(
        text_df_compare,
        "vader_label",
        "4.b.4 VADER SENTIMENT LABELS VERSUS RECOMMENDATION"
    )

    roberta_rec_summary = sentiment_recommendation_summary(
        text_df_compare,
        "roberta_label",
        "4.b.4 ROBERTA SENTIMENT LABELS VERSUS RECOMMENDATION"
    )

```

```

# -----
# 4.b.5 Candidate ambiguous and contradictory cases
# -----
candidate_cases = text_df_compare[
    (
        ((text_df_compare["vader_label"] == "Positive") &
(text_df_compare["Recommended"] == 0)) |
        ((text_df_compare["vader_label"] == "Negative") &
(text_df_compare["Recommended"] == 1)) |
        ((text_df_compare["roberta_label"] == "Positive") &
(text_df_compare["Recommended"] == 0)) |
        ((text_df_compare["roberta_label"] == "Negative") &
(text_df_compare["Recommended"] == 1)) |
        (text_df_compare["vader_label"] !=
text_df_compare["roberta_label"])) |
        ((text_df_compare["positive_cue_count"] >= 2) &
(text_df_compare["negative_cue_count"] >= 2))
    )
].copy()

show_table(
    "4.b.5 CANDIDATE AMBIGUOUS OR MISCLASSIFIED REVIEWS",
    candidate_cases.head(25),
    index=False
)

# -----
# 4.b.6 Automated review groups for manual inspection
# -----
base_cols = [
    "Review_ID",
    "Overall_Rating",
    "Recommended",
    "vader_score",
    "vader_label",
    "roberta_confidence",
    "roberta_label",
    "positive_cue_count",
    "negative_cue_count",
    "Review"
]

df_groups = text_df_compare.copy()

df_groups["vader_wrong"] = (
    ((df_groups["vader_label"] == "Positive") & (df_groups["Recommended"]
== 0)) |
    ((df_groups["vader_label"] == "Negative") & (df_groups["Recommended"]
== 1))
)

df_groups["roberta_wrong"] = (
    ((df_groups["roberta_label"] == "Positive") & (df_groups["Recommended"]
== 0)) |
    ((df_groups["roberta_label"] == "Negative") & (df_groups["Recommended"]
== 1))
)

df_groups["method_disagree"] = df_groups["vader_label"] !=
df_groups["roberta_label"]

```

```

df_groups["mixed_cues"] = (
    (df_groups["positive_cue_count"] >= 2) &
    (df_groups["negative_cue_count"] >= 2)
)

group1_reviews = df_groups[
    (df_groups["vader_wrong"] == True) &
    (df_groups["roberta_wrong"] == False)
][base_cols].copy()
group1_reviews["Review_Group"] = "VADER misclassification"

group2_reviews = df_groups[
    (df_groups["roberta_wrong"] == True) &
    (df_groups["vader_wrong"] == False)
][base_cols].copy()
group2_reviews["Review_Group"] = "RoBERTa misclassification"

group3_reviews = df_groups[
    (
        ((df_groups["vader_wrong"] == True) & (df_groups["roberta_wrong"]
        == True)) |
        (df_groups["method_disagree"] == True) |
        (df_groups["mixed_cues"] == True)
    )
][base_cols].copy()
group3_reviews["Review_Group"] = "Ambiguity / mixed-language cases"

group3_reviews = group3_reviews[
    ~group3_reviews["Review_ID"].isin(group1_reviews["Review_ID"]) &
    ~group3_reviews["Review_ID"].isin(group2_reviews["Review_ID"])
]

cols_order = [
    "Review_Group",
    "Review_ID",
    "Overall_Rating",
    "Recommended",
    "vader_score",
    "vader_label",
    "roberta_confidence",
    "roberta_label",
    "positive_cue_count",
    "negative_cue_count",
    "Review"
]

group1_reviews = group1_reviews[cols_order].copy()
group2_reviews = group2_reviews[cols_order].copy()
group3_reviews = group3_reviews[cols_order].copy()

print_section("4.b.6 MUTUALLY EXCLUSIVE GROUP COUNTS")
print("VADER misclassification:", len(group1_reviews))
print("RoBERTa misclassification:", len(group2_reviews))
print("Ambiguity / mixed-language cases:", len(group3_reviews))

# -----
# 4.b.7 Final manually reviewed cases
# -----
group1_reviews["selection_strength"] = (
    group1_reviews["vader_score"].abs() +
    group1_reviews["negative_cue_count"] +

```



```

    group1_reviews["positive_cue_count"]
)

group2_reviews["selection_strength"] = (
    group2_reviews["roberta_confidence"] +
    group2_reviews["negative_cue_count"] +
    group2_reviews["positive_cue_count"]
)

group3_reviews["selection_strength"] = (
    group3_reviews["vader_score"].abs() +
    group3_reviews["roberta_confidence"] +
    group3_reviews["negative_cue_count"] +
    group3_reviews["positive_cue_count"]
)

group1_final = (
    group1_reviews
    .sort_values(["selection_strength", "Review_ID"], ascending=[False,
True])
    .head(5)
    .drop(columns="selection_strength")
)

group2_final = (
    group2_reviews
    .sort_values(["selection_strength", "Review_ID"], ascending=[False,
True])
    .head(5)
    .drop(columns="selection_strength")
)

group3_final = (
    group3_reviews
    .sort_values(["selection_strength", "Review_ID"], ascending=[False,
True])
    .head(5)
    .drop(columns="selection_strength")
)

manual_review_df = pd.concat(
    [group1_final, group2_final, group3_final],
    axis=0
).reset_index(drop=True)

show_table(
    "4.b.7 FINAL REVIEWS SELECTED FOR MANUAL ANALYSIS",
    manual_review_df,
    index=False
)

comparison_table = manual_review_df.copy()

# -----
# 4.b.8 Manual review notes
# -----
def manual_review_reason(row):
    reasons = []

    if row["Review_Group"] == "VADER misclassification":

```

```

        reasons.append("VADER misclassifies sentiment relative to
recommendation")

        if row["Review_Group"] == "RoBERTa misclassification":
            reasons.append("RoBERTa misclassifies sentiment despite stronger
contextual modelling")

        if row["Review_Group"] == "Ambiguity / mixed-language cases":
            reasons.append("Review contains ambiguity, mixed sentiment, or
conflicting evaluative cues")

        if row["positive_cue_count"] >= 2 and row["negative_cue_count"] >= 2:
            reasons.append("Contains both positive and negative cues")

        return "; ".join(reasons)

manual_review_df["manual_review_reason"] = manual_review_df.apply(
    manual_review_reason,
    axis=1
)

show_table(
    "4.b.8 MANUAL REVIEW NOTES",
    manual_review_df[[
        "Review_Group",
        "Review_ID",
        "Recommended",
        "vader_label",
        "roberta_label",
        "manual_review_reason"
    ]],
    index=False
)

# -----
# 4.b.9 Prepare text visualisation data
# -----
text_vis = text_df_compare.copy()

text_vis["Recommended_label"] = text_vis["Recommended"].map({
    1: "Recommended",
    0: "Not Recommended"
})

text_vis["vader_label"] = pd.Categorical(
    text_vis["vader_label"],
    categories=["Negative", "Neutral", "Positive"],
    ordered=True
)

text_vis["roberta_label"] = pd.Categorical(
    text_vis["roberta_label"],
    categories=["Negative", "Neutral", "Positive"],
    ordered=True
)

text_vis["contradiction_flag"] = np.where(
    (
        ((text_vis["vader_label"] == "Positive") & (text_vis["Recommended"]
== 0)) |

```

```

        ((text_vis["vader_label"] == "Negative") & (text_vis["Recommended"]
== 1)) |
        ((text_vis["roberta_label"] == "Positive") &
(text_vis["Recommended"] == 0)) |
        ((text_vis["roberta_label"] == "Negative") &
(text_vis["Recommended"] == 1))
    ),
    "Contradictory",
    "Aligned"
)

text_vis["is_mixed_sentiment"] = np.where(
    (text_vis["positive_cue_count"] > 0) & (text_vis["negative_cue_count"]
> 0),
    "Mixed",
    "Not Mixed"
)

comparison_plot = comparison_table.copy()
label_to_num = {"Negative": -1, "Neutral": 0, "Positive": 1}

comparison_plot["vader_num"] =
comparison_plot["vader_label"].map(label_to_num)
comparison_plot["roberta_num"] =
comparison_plot["roberta_label"].map(label_to_num)
comparison_plot["disagreement_magnitude"] = (
    comparison_plot["vader_num"] - comparison_plot["roberta_num"]
).abs()

# -----
# 4.b.10 Plot 1: Overall rating versus VADER score
# -----
plot_df = text_vis.dropna(subset=["Overall_Rating", "vader_score",
"Recommended"]).copy()
contradiction_df = plot_df[plot_df["contradiction_flag"] ==
"Contradictory"].copy()

fig, ax = plt.subplots(figsize=(11, 8))

ax.scatter(
    plot_df["Overall_Rating"],
    plot_df["vader_score"],
    s=10,
    alpha=0.08,
    color="grey",
    edgecolor="none"
)

ax.scatter(
    contradiction_df["Overall_Rating"],
    contradiction_df["vader_score"],
    s=35,
    alpha=0.70,
    c=np.where(contradiction_df["Recommended"] == 0, "crimson",
"darkorange"),
    edgecolor="black",
    linewidth=0.3
)

ax.axvline(5, linestyle="--", color="black", linewidth=1)
ax.axhline(0, linestyle="--", color="black", linewidth=1)

```

```

ax.set_xlabel("Overall Rating")
ax.set_ylabel("VADER Compound Sentiment Score")
ax.set_xlim(0.5, 10.5)
ax.set_ylim(-1.05, 1.05)

legend_elements = [
    Line2D([0], [0], marker="o", color="w",
           label="Positive sentiment / Not recommended",
           markerfacecolor="crimson", markeredgecolor="black",
           markersize=8),
    Line2D([0], [0], marker="o", color="w",
           label="Negative sentiment / Recommended",
           markerfacecolor="darkorange", markeredgecolor="black",
           markersize=8)
]

ax.legend(handles=legend_elements, frameon=True, loc="lower right")
finalise_plot("Alignment Between Overall Rating and VADER Sentiment")

# -----
# 4.b.11 Plot 2: VADER distribution by recommendation
# -----
plot_df = text_vis.dropna(subset=["vader_score",
                                   "Recommended_label"]).copy()

plt.figure(figsize=(10, 7))

sns.violinplot(
    data=plot_df,
    x="Recommended_label",
    y="vader_score",
    palette=["#5E7AC4", "#F08D39"],
    inner=None,
    cut=0,
    linewidth=1.2
)

sns.boxplot(
    data=plot_df,
    x="Recommended_label",
    y="vader_score",
    width=0.22,
    showcaps=True,
    showfliers=False,
    boxprops={"facecolor": "white", "zorder": 3},
    whiskerprops={"linewidth": 1.2},
    medianprops={"color": "black", "linewidth": 1.4}
)

sns.stripplot(
    data=plot_df.sample(min(2000, len(plot_df))),
    random_state=RANDOM_STATE,
    x="Recommended_label",
    y="vader_score",
    color="black",
    alpha=0.10,
    size=2.3,
    jitter=0.22
)

```

```

plt.axhline(0, linestyle="--", color="black", linewidth=1)
plt.xlabel("")
plt.ylabel("VADER Compound Sentiment Score")
plt.ylim(-1.05, 1.05)
finalise_plot("Distribution of VADER Sentiment Scores by Recommendation Outcome")

# -----
# 4.b.14 Plot 5: RoBERTa distribution by recommendation
# -----
plot_df_r = text_vis.dropna(subset=["roberta_label", "Recommended"]).copy()

plot_df_r["Recommended_label"] = plot_df_r["Recommended"].map({0: "Not Recommended", 1: "Recommended"})
plot_df_r["roberta_label"] =
plot_df_r["roberta_label"].astype(str).str.strip().str.title()

print(plot_df_r[["Recommended", "Recommended_label",
"roberta_label"]].head())
print(plot_df_r["Recommended_label"].value_counts(dropna=False))
print(plot_df_r["roberta_label"].value_counts(dropna=False))

# Build percentage table safely
counts = pd.crosstab(plot_df_r["Recommended_label"],
plot_df_r["roberta_label"])
dist_df = counts.div(counts.sum(axis=1), axis=0).mul(100)

# force column order if present
for col in ["Negative", "Neutral", "Positive"]:
    if col not in dist_df.columns:
        dist_df[col] = 0

dist_df = dist_df[["Negative", "Neutral", "Positive"]]
dist_df = dist_df.reindex(["Not Recommended", "Recommended"])

print(dist_df)

sns.set_theme(style="ticks")

ax = dist_df.plot(
    kind="bar",
    figsize=(8.5, 5.5),
    edgecolor="black",
    width=0.72
)

ax.grid(False)
sns.despine()

plt.xlabel("")
plt.ylabel("Percentage of Reviews")
plt.xticks(rotation=0)
plt.ylim(0, 100)

plt.legend(
    title="RoBERTa Sentiment",
    fontsize=10,
    title_fontsize=11,
    loc="upper center",
    bbox_to_anchor=(0.5, 0.98),
    ncol=3,

```

```

        frameon=True
    )

    finalise_plot("RoBERTa Sentiment Distribution by Recommendation Outcome")

    plot_df_v = text_vis.dropna(subset=["vader_label", "Recommended"]).copy()

    plot_df_v["Recommended_label"] = plot_df_v["Recommended"].map({0: "Not
Recommended", 1: "Recommended"})
    plot_df_v["vader_label"] =
    plot_df_v["vader_label"].astype(str).str.strip().str.title()

    print(plot_df_v[["Recommended", "Recommended_label",
    "vader_label"]].head())
    print(plot_df_v["Recommended_label"].value_counts(dropna=False))
    print(plot_df_v["vader_label"].value_counts(dropna=False))

    # Build percentage table safely
    counts = pd.crosstab(plot_df_v["Recommended_label"],
    plot_df_v["vader_label"])
    dist_df_v = counts.div(counts.sum(axis=1), axis=0).mul(100)

    # Force column order if present
    for col in ["Negative", "Neutral", "Positive"]:
        if col not in dist_df_v.columns:
            dist_df_v[col] = 0

    dist_df_v = dist_df_v[["Negative", "Neutral", "Positive"]]
    dist_df_v = dist_df_v.reindex(["Not Recommended", "Recommended"])

    print(dist_df_v)

    sns.set_theme(style="ticks")

    ax = dist_df_v.plot(
        kind="bar",
        figsize=(8.5, 5.5),
        edgecolor="black",
        width=0.72,
        color=["#2F5D8A", "#F39C12", "#3A9D5D"]
    )

    ax.grid(False)
    sns.despine()

    plt.xlabel("")
    plt.ylabel("Percentage of Reviews")
    plt.xticks(rotation=0)
    plt.ylim(0, 100)

    plt.legend(
        title="VADER Sentiment",
        fontsize=10,
        title_fontsize=11,
        loc="upper center",
        bbox_to_anchor=(0.5, 0.98),
        ncol=3,
        frameon=True
    )

    finalise_plot("VADER Sentiment Distribution by Recommendation Outcome")

```

```

plt.show()

# -----
# 4.b.12 Plot 3: Recommendation heatmap by rating and VADER sentiment
# -----
heat_df = text_vis.dropna(subset=["Overall_Rating", "vader_score",
"Recommended"]).copy()

heat_df["Rating_Band"] = pd.cut(
    heat_df["Overall_Rating"],
    bins=[0, 2, 4, 6, 8, 10],
    labels=["1-2", "3-4", "5-6", "7-8", "9-10"],
    include_lowest=True
)

heat_df["Sentiment_Band"] = pd.cut(
    heat_df["vader_score"],
    bins=[-1.01, -0.3, 0.3, 1.01],
    labels=["Negative", "Neutral/Mixed", "Positive"],
    include_lowest=True
)

heatmap_df = (
    heat_df.groupby(["Sentiment_Band", "Rating_Band"],
observed=False)["Recommended"]
        .mean()
        .mul(100)
        .unstack()
)

count_df = (
    heat_df.groupby(["Sentiment_Band", "Rating_Band"],
observed=False)["Recommended"]
        .size()
        .unstack()
)

heatmap_df = heatmap_df.astype(float)
count_df = count_df.fillna(0).astype(int)

annot = heatmap_df.round(1).astype(str) + "%\n(n=" + count_df.astype(str) +
")"

plt.figure(figsize=(11, 6))
sns.heatmap(
    heatmap_df,
    annot=annot,
    fmt="",
    cmap="RdYlGn",
    linewidths=0.8,
    linecolor="white",
    cbar_kws={"label": "Recommendation rate (%)"}
)

plt.xlabel("Overall Rating Band")
plt.ylabel("VADER Sentiment Band")
finalise_plot("Recommendation Rate Across Rating and Sentiment Bands")

# -----
# 4.b.13 Plot 4: Recommendation heatmap by rating and RoBERTa sentiment
# -----

```

```

heat_df_r = text_df.dropna(subset=["Overall_Rating", "roberta_label",
"Recommended"]).copy()

heat_df_r["Rating_Band"] = pd.cut(
    heat_df_r["Overall_Rating"],
    bins=[0, 2, 4, 6, 8, 10],
    labels=["1-2", "3-4", "5-6", "7-8", "9-10"],
    include_lowest=True
)

heat_df_r["Sentiment_Band"] = pd.Categorical(
    heat_df_r["roberta_label"],
    categories=["Negative", "Neutral", "Positive"],
    ordered=True
)

heatmap_df_r = (
    heat_df_r.groupby(["Sentiment_Band", "Rating_Band"],
observed=False)["Recommended"]
        .mean()
        .mul(100)
        .unstack()
)

count_df_r = (
    heat_df_r.groupby(["Sentiment_Band", "Rating_Band"],
observed=False)["Recommended"]
        .size()
        .unstack()
)

heatmap_df_r = heatmap_df_r.astype(float)
count_df_r = count_df_r.fillna(0).astype(int)

annot_r = heatmap_df_r.round(1).astype(str) + "%\n(n=" +
count_df_r.astype(str) + ")"

plt.figure(figsize=(11, 6))
sns.heatmap(
    heatmap_df_r,
    annot=annot_r,
    fmt="",
    cmap="RdYlGn",
    linewidths=0.8,
    linecolor="white",
    cbar_kws={"label": "Recommendation rate (%)"}
)

plt.xlabel("Overall Rating Band")
plt.ylabel("RoBERTa Sentiment Band")
finalise_plot("Recommendation Rate Across Rating and RoBERTa Sentiment
Bands")

# -----
# 4.b.13 Plot 4: VADER versus RoBERTa agreement heatmap
# -----
agreement_df = text_vis[["vader_label", "roberta_label"]].dropna().copy()
order = ["Negative", "Neutral", "Positive"]

counts = pd.crosstab(
    agreement_df["vader_label"],

```



```

    agreement_df["roberta_label"]
).reindex(index=order, columns=order, fill_value=0)

row_pct = counts.div(counts.sum(axis=1), axis=0) * 100
annot = counts.astype(str) + "\n(" + row_pct.round(1).astype(str) + "%)"

plt.figure(figsize=(8.5, 5.8))
sns.heatmap(
    counts,
    annot=annot,
    fmt="",
    cmap="Purples",
    linewidths=0.8,
    linecolor="white",
    cbar_kws={"label": "Number of Reviews"},
    annot_kws={"size": 10}
)

plt.xlabel("RoBERTa Sentiment Label")
plt.ylabel("VADER Sentiment Label")
finalise_plot("Agreement Between VADER and RoBERTa Sentiment Labels")

# -----
# 4.b.14 Plot 5: Dumbbell plot for manually reviewed cases
# -----
case_df = comparison_plot.dropna(subset=["vader_num",
"roberta_num"]).copy()

case_df["gap"] = (case_df["vader_num"] - case_df["roberta_num"]).abs()
case_df = case_df.sort_values(
    by=["gap", "Review_ID"],
    ascending=[False, True]
).reset_index(drop=True)

fig, ax = plt.subplots(figsize=(13.2, 7.0))
y_positions = np.arange(len(case_df))

# connection lines
for i, row in case_df.iterrows():
    if row["gap"] > 0:
        strong = row["gap"] >= 2
        ax.plot(
            [row["vader_num"], row["roberta_num"]],
            [i, i],
            color="#2f2f2f" if strong else "#c7c7c7",
            linewidth=1.5 if strong else 0.9,
            alpha=0.60 if strong else 0.45,
            zorder=1
        )

# agreement vs disagreement
agree_df = case_df[case_df["gap"] == 0].copy()
disagree_df = case_df[case_df["gap"] > 0].copy()

agree_y = agree_df.index.to_numpy()
disagree_y = disagree_df.index.to_numpy()

# disagreement points
vader_scatter = ax.scatter(
    disagree_df["vader_num"],
    disagree_y,

```

```

        s=88,
        color="#4C72B0",
        label="VADER",
        zorder=3
    )

    roberta_scatter = ax.scatter(
        disagree_df["roberta_num"],
        disagree_y,
        s=88,
        marker="s",
        color="#DD8452",
        label="RoBERTa",
        zorder=3
    )

    # agreement points
    agreement_scatter = ax.scatter(
        agree_df["vader_num"],
        agree_y,
        s=98,
        marker="D",
        color="#3a3a3a",
        label="Agreement",
        zorder=4
    )

    # y-axis labels
    labels = [f"ID {rid}" for rid in case_df["Review_ID"]]
    ax.set_yticks(y_positions)
    ax.set_yticklabels(labels, fontsize=10)

    # x-axis labels
    ax.set_xticks([-1, 0, 1])
    ax.set_xticklabels(["Negative", "Neutral", "Positive"], fontsize=11)
    ax.set_xlabel("Sentiment Classification", fontsize=13)
    ax.set_ylabel("")

    # grid styling
    ax.grid(axis="x", color="#e6e6e6", linewidth=0.8)
    ax.grid(axis="y", visible=False)
    ax.set_axisbelow(True)

    # spine cleanup
    for spine in ["top", "right"]:
        ax.spines[spine].set_visible(False)

    # legend outside
    handles = [agreement_scatter, vader_scatter, roberta_scatter]
    legend_labels = ["Agreement", "VADER", "RoBERTa"]

    ax.legend(
        handles,
        legend_labels,
        loc="center left",
        bbox_to_anchor=(1.02, 0.5),
        frameon=True,
        fontsize=11,
        title_fontsize=12,
        borderaxespad=0.0,
        labelspacing=0.9,

```

```

        handletextpad=0.8,
        borderpad=0.8,
        handlelength=1.6,
        markerscale=1.0
    )

plt.tight_layout(rect=[0, 0, 0.82, 1])
finalise_plot("Agreement and Disagreement Across Manually Reviewed Cases")

# -----
# 4.b.14 Plot 5: Contradiction rate by sentiment method
# Contradictions defined as:
# - Positive sentiment + Not Recommended
# - Negative sentiment + Recommended
# -----

contradiction_df = text_df.dropna(
    subset=["Recommended", "vader_label", "roberta_label"]
).copy()

contradiction_df["Recommended"] = pd.to_numeric(
    contradiction_df["Recommended"], errors="coerce"
)

contradiction_df["vader_label"] =
contradiction_df["vader_label"].astype(str).str.strip().str.title()
contradiction_df["roberta_label"] =
contradiction_df["roberta_label"].astype(str).str.strip().str.title()

plot_rows = []

# VADER Positive contradiction:
# among VADER Positive reviews, % that are Not Recommended
temp = contradiction_df[contradiction_df["vader_label"] == "Positive"]
plot_rows.append({
    "Category": "VADER Positive\n→ Not Recommended",
    "Contradiction_Rate": (temp["Recommended"] == 0).mean() * 100,
    "Count": len(temp)
})

# RoBERTa Positive contradiction:
# among RoBERTa Positive reviews, % that are Not Recommended
temp = contradiction_df[contradiction_df["roberta_label"] == "Positive"]
plot_rows.append({
    "Category": "RoBERTa Positive\n→ Not Recommended",
    "Contradiction_Rate": (temp["Recommended"] == 0).mean() * 100,
    "Count": len(temp)
})

# VADER Negative contradiction:
# among VADER Negative reviews, % that are Recommended
temp = contradiction_df[contradiction_df["vader_label"] == "Negative"]
plot_rows.append({
    "Category": "VADER Negative\n→ Recommended",
    "Contradiction_Rate": (temp["Recommended"] == 1).mean() * 100,
    "Count": len(temp)
})

# RoBERTa Negative contradiction:
# among RoBERTa Negative reviews, % that are Recommended
temp = contradiction_df[contradiction_df["roberta_label"] == "Negative"]

```

```

plot_rows.append({
    "Category": "RoBERTa Negative\n→ Recommended",
    "Contradiction_Rate": (temp["Recommended"] == 1).mean() * 100,
    "Count": len(temp)
})

contradiction_plot_df = pd.DataFrame(plot_rows)

plt.figure(figsize=(10.5, 6.2))
ax = sns.barplot(
    data=contradiction_plot_df,
    x="Category",
    y="Contradiction_Rate",
    palette=["#3A9D5D", "#2F5D8A", "#3A9D5D", "#2F5D8A"],
    edgecolor="black",
    linewidth=1.0
)

# Annotate bars
for i, row in contradiction_plot_df.iterrows():
    ax.text(
        i,
        row["Contradiction_Rate"] + 0.8,
        f'{row["Contradiction_Rate"]:.1f}%\n(n={row["Count"]:,}) ',
        ha="center",
        va="bottom",
        fontsize=10
    )

plt.xlabel("")
plt.ylabel("Contradiction rate (%)")
plt.ylim(0, contradiction_plot_df["Contradiction_Rate"].max() * 1.18)
plt.grid(axis="y", color="#e6e6e6", linewidth=0.8)
finalise_plot("Contradictory Sentiment Classifications by Method")
plt.show()

# -----
# 4.b.15 Plot 6: Recommendation rate by service theme
# Colourful version
# -----

theme_df = text_df.dropna(subset=["Recommended"]).copy()
theme_df["Recommended"] = pd.to_numeric(theme_df["Recommended"],
errors="coerce")

theme_cols = [
    "delay_theme",
    "cancellation_theme",
    "refund_theme",
    "staff_theme",
    "seat_theme",
    "food_theme"
]

theme_labels = {
    "delay_theme": "Delay",
    "cancellation_theme": "Cancellation",
    "refund_theme": "Refund",
    "staff_theme": "Staff",
    "seat_theme": "Seat",
    "food_theme": "Food"
}

```

```

}

plot_rows = []

for col in theme_cols:
    temp = theme_df[theme_df[col] == 1].copy()
    plot_rows.append({
        "Theme": theme_labels[col],
        "Recommendation_Rate": temp["Recommended"].mean() * 100 if
len(temp) > 0 else 0,
        "Count": len(temp)
    })

theme_plot_df = pd.DataFrame(plot_rows)
theme_plot_df = theme_plot_df.sort_values(
    by="Recommendation_Rate",
    ascending=True
).reset_index(drop=True)

# Colour progression from low-recommendation themes to high-recommendation
themes
bar_colors = [
    "#C0392B", # Refund
    "#E74C3C", # Cancellation
    "#F39C12", # Delay
    "#5DA5DA", # Staff
    "#3E7CB1", # Seat
    "#2E8B57"  # Food
]

plt.figure(figsize=(11, 6.4))
ax = sns.barplot(
    data=theme_plot_df,
    x="Theme",
    y="Recommendation_Rate",
    palette=bar_colors,
    edgecolor="black",
    linewidth=1.1
)

for i, row in theme_plot_df.iterrows():
    ax.text(
        i,
        row["Recommendation_Rate"] + 1.3,
        f'{row["Recommendation_Rate"]:.1f}%\n(n={row["Count"]:,}) ',
        ha="center",
        va="bottom",
        fontsize=10
    )

plt.xlabel("")
plt.ylabel("Recommendation rate (%)")
plt.ylim(0, 100)
plt.grid(axis="y", color="#e6e6e6", linewidth=0.8)
finalise_plot("Recommendation Rate by Service Theme")
plt.show()

# %% =====
# 4.b. PREDICTIVE ANALYTICS (TEXT FEATURES)
# =====

```

```

# %% -----
# 4.b.15 Text modelling dataset and train-test split
# -----
text_model_df = text_df[[
    "Review_ID",
    "review_clean_final",
    "Recommended",
    "Overall_Rating",
    "vader_score",
    "positive_cue_count",
    "negative_cue_count"
]].dropna(subset=["review_clean_final", "Recommended"]).copy()

text_model_df["text_length"] =
text_model_df["review_clean_final"].str.split().apply(len)

print_section("4.b.15 TEXT MODELLING DATASET")
print("Shape:", text_model_df.shape)
print(text_model_df["Recommended"].value_counts(normalize=True).round(4))

X_text = text_model_df["review_clean_final"]
y_text = text_model_df["Recommended"]

X_train_text, X_test_text, y_train_text, y_test_text, id_train_text,
id_test_text = train_test_split(
    X_text,
    y_text,
    text_model_df["Review_ID"],
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y_text
)

print_section("4.b.15 TEXT TRAIN-TEST SPLIT")
print("X_train_text shape:", X_train_text.shape)
print("X_test_text shape:", X_test_text.shape)
print(y_train_text.value_counts(normalize=True).round(4))
print(y_test_text.value_counts(normalize=True).round(4))

# %% -----
# 4.b.16 TF-IDF feature engineering
# -----
tfidf_vectorizer = TfidfVectorizer(
    ngram_range=(1, 2),
    min_df=5,
    max_df=0.90,
    sublinear_tf=True
)

X_train_tfidf = tfidf_vectorizer.fit_transform(X_train_text)
X_test_tfidf = tfidf_vectorizer.transform(X_test_text)

feature_names = np.array(tfidf_vectorizer.get_feature_names_out())

print_section("4.b.16 TF-IDF FEATURE SPACE")
print("Training matrix shape:", X_train_tfidf.shape)
print("Testing matrix shape:", X_test_tfidf.shape)
print("Number of text features:", len(feature_names))

```

```

# %% -----
# 4.b.17 Logistic regression using TF-IDF
# -----
text_log_model = LogisticRegression(
    max_iter=2000,
    random_state=RANDOM_STATE
)

text_log_model.fit(X_train_tfidf, y_train_text)

text_log_pred = text_log_model.predict(X_test_tfidf)
text_log_prob = text_log_model.predict_proba(X_test_tfidf)[: , 1]

text_log_results = evaluate_model(
    y_test_text,
    text_log_pred,
    text_log_prob,
    "Text Logistic Regression"
)

text_log_cm = print_conf_matrix(
    y_test_text,
    text_log_pred,
    "Text Logistic Regression"
)

# %% -----
# 4.b.18 Random forest using TF-IDF
# -----
text_rf_model = RandomForestClassifier(
    n_estimators=300,
    random_state=RANDOM_STATE,
    n_jobs=-1
)

text_rf_model.fit(X_train_tfidf, y_train_text)

text_rf_pred = text_rf_model.predict(X_test_tfidf)
text_rf_prob = text_rf_model.predict_proba(X_test_tfidf)[: , 1]

text_rf_results = evaluate_model(
    y_test_text,
    text_rf_pred,
    text_rf_prob,
    "Text Random Forest"
)

text_rf_cm = print_conf_matrix(
    y_test_text,
    text_rf_pred,
    "Text Random Forest"
)

# %% -----
# 4.b.19 Additional benchmark: Linear SVM using TF-IDF
# -----
text_svm_base = LinearSVC(
    random_state=RANDOM_STATE
)

```

```

text_svm_model = CalibratedClassifierCV(
    estimator=text_svm_base,
    method="sigmoid",
    cv=3
)

text_svm_model.fit(X_train_tfidf, y_train_text)

text_svm_pred = text_svm_model.predict(X_test_tfidf)
text_svm_prob = text_svm_model.predict_proba(X_test_tfidf)[: , 1]

text_svm_results = evaluate_model(
    y_test_text,
    text_svm_pred,
    text_svm_prob,
    "Text Linear SVM"
)

text_svm_cm = print_conf_matrix(
    y_test_text,
    text_svm_pred,
    "Text Linear SVM"
)

# %% -----
# 4.b.20 Text model comparison tables
# -----
text_model_results = pd.DataFrame([
    text_log_results,
    text_rf_results,
    text_svm_results
]).set_index("Model").round(4)

show_table("4.b.20 TEXT MODEL COMPARISON", text_model_results)

part4a_vs_text = pd.DataFrame([
    {
        "Model": "Numerical Logistic Regression",
        "Accuracy": log_results["Accuracy"],
        "Precision": log_results["Precision"],
        "Recall": log_results["Recall"],
        "F1": log_results["F1"],
        "ROC_AUC": log_results["ROC_AUC"]
    },
    {
        "Model": "Numerical Random Forest",
        "Accuracy": rf_results["Accuracy"],
        "Precision": rf_results["Precision"],
        "Recall": rf_results["Recall"],
        "F1": rf_results["F1"],
        "ROC_AUC": rf_results["ROC_AUC"]
    },
    {
        "Model": "Text Logistic Regression",
        "Accuracy": text_log_results["Accuracy"],
        "Precision": text_log_results["Precision"],
        "Recall": text_log_results["Recall"],
        "F1": text_log_results["F1"],
        "ROC_AUC": text_log_results["ROC_AUC"]
    },
],

```



```

{
    "Model": "Text Random Forest",
    "Accuracy": text_rf_results["Accuracy"],
    "Precision": text_rf_results["Precision"],
    "Recall": text_rf_results["Recall"],
    "F1": text_rf_results["F1"],
    "ROC_AUC": text_rf_results["ROC_AUC"]
},
{
    "Model": "Text Linear SVM",
    "Accuracy": text_svm_results["Accuracy"],
    "Precision": text_svm_results["Precision"],
    "Recall": text_svm_results["Recall"],
    "F1": text_svm_results["F1"],
    "ROC_AUC": text_svm_results["ROC_AUC"]
}
}).set_index("Model").round(4)

show_table("4.b.20 NUMERICAL VERSUS TEXT MODEL COMPARISON", part4a_vs_text)

# %% -----
# 4.b.21 Plot 6: ROC curve comparison for text models
# -----
fpr_log, tpr_log, _ = roc_curve(y_test_text, text_log_prob)
fpr_rf, tpr_rf, _ = roc_curve(y_test_text, text_rf_prob)
fpr_svm, tpr_svm, _ = roc_curve(y_test_text, text_svm_prob)

plt.figure(figsize=(9.6, 6.6))

plt.plot(
    fpr_log, tpr_log,
    linewidth=2.6,
    label=f"Logistic Regression (AUC = {text_log_results['ROC_AUC']:.3f})"
)

plt.plot(
    fpr_rf, tpr_rf,
    linewidth=2.6,
    label=f"Random Forest (AUC = {text_rf_results['ROC_AUC']:.3f})"
)

plt.plot(
    fpr_svm, tpr_svm,
    linewidth=2.6,
    label=f"Linear SVM (AUC = {text_svm_results['ROC_AUC']:.3f})"
)

plt.plot([0, 1], [0, 1],
         linestyle="--",
         color="black",
         linewidth=1,
         alpha=0.6)

plt.xlim(0, 1)
plt.ylim(0, 1.02)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")

plt.legend(
    loc="lower right",
    frameon=True,

```

```

        fontsize=11
    )

    finalise_plot("ROC Curve Comparison for Text Models")

# %% -----
# 4.b.22 Plot 7: Predicted probability distributions
# -----
probability_plot_df = pd.DataFrame({
    "Actual": y_test_text.values,
    "Logistic_Probability": text_log_prob,
    "Random_Forest_Probability": text_rf_prob,
    "Linear_SVM_Probability": text_svm_prob
})

probability_plot_df["Actual_Label"] = probability_plot_df["Actual"].map({
    0: "Not Recommended",
    1: "Recommended"
})

plt.figure(figsize=(9.5, 6.2))

sns.kdeplot(
    data=probability_plot_df,
    x="Linear_SVM_Probability",
    hue="Actual_Label",
    fill=True,
    common_norm=False,
    alpha=0.35,
    linewidth=2,
    bw_adjust=0.9
)

plt.axvline(0.5, linestyle="--", color="black", linewidth=1)
plt.xlim(-0.15, 1.15)

plt.xlabel("Predicted Probability of Recommendation")
plt.ylabel("Density")

ymax = plt.gca().get_ylim()[1]
plt.text(
    0.5, ymax * 0.85,
    "Threshold = 0.5",
    ha="center",
    fontsize=10,
    color="black"
)

legend = plt.gca().get_legend()
if legend is not None:
    legend.set_title("Actual Outcome")
    legend.get_title().set_fontsize(11)
    for text in legend.get_texts():
        text.set_fontsize(10)

    finalise_plot("Predicted Probability Distribution for the Linear SVM Text
Model")

# %% -----
# 4.b.23 Top predictive words from logistic regression

```

```

# -----
logistic_coef_df = pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": text_log_model.coef_[0]
})

top_positive_terms = (
    logistic_coef_df
    .sort_values("Coefficient", ascending=False)
    .head(20)
    .sort_values("Coefficient", ascending=True)
)

top_negative_terms = (
    logistic_coef_df
    .sort_values("Coefficient", ascending=True)
    .head(20)
    .sort_values("Coefficient", ascending=False)
)

show_table("TOP POSITIVE TEXT FEATURES FROM LOGISTIC REGRESSION",
top_positive_terms, index=False)
show_table("TOP NEGATIVE TEXT FEATURES FROM LOGISTIC REGRESSION",
top_negative_terms, index=False)

fig, axes = plt.subplots(1, 2, figsize=(15, 9))

axes[0].hlines(
    y=top_negative_terms["Feature"],
    xmin=0,
    xmax=top_negative_terms["Coefficient"],
    linewidth=2.5
)
axes[0].plot(
    top_negative_terms["Coefficient"],
    top_negative_terms["Feature"],
    "o",
    markersize=8
)
axes[0].axvline(0, color="black", linewidth=1)
axes[0].set_xlabel("Coefficient")
axes[0].set_title("Terms Driving Non-Recommendation", fontsize=13,
weight="bold")

axes[1].hlines(
    y=top_positive_terms["Feature"],
    xmin=0,
    xmax=top_positive_terms["Coefficient"],
    linewidth=2.5
)
axes[1].plot(
    top_positive_terms["Coefficient"],
    top_positive_terms["Feature"],
    "o",
    markersize=8
)
axes[1].axvline(0, color="black", linewidth=1)
axes[1].set_xlabel("Coefficient")
axes[1].set_title("Terms Driving Recommendation", fontsize=13,
weight="bold")

```

```

plt.tight_layout()
plt.show()

# %% -----
# 4.b.24 Distinctive language by class using average TF-IDF
# -----
full_tfidf =
tfidf_vectorizer.transform(text_model_df["review_clean_final"])

recommended_mask = (text_model_df["Recommended"].astype(int).to_numpy() ==
1)
not_recommended_mask = (text_model_df["Recommended"].astype(int).to_numpy()
== 0)

avg_tfidf_rec = np.asarray(full_tfidf[recommended_mask,
:].mean(axis=0)).ravel()
avg_tfidf_not = np.asarray(full_tfidf[not_recommended_mask,
:].mean(axis=0)).ravel()

class_language_df = pd.DataFrame({
    "Feature": feature_names,
    "Avg_TFIDF_Recommended": avg_tfidf_rec,
    "Avg_TFIDF_Not_Recommended": avg_tfidf_not
})

class_language_df["Lift_Recommended"] = (
    class_language_df["Avg_TFIDF_Recommended"] -
class_language_df["Avg_TFIDF_Not_Recommended"]
)

class_language_df["Lift_Not_Recommended"] = (
    class_language_df["Avg_TFIDF_Not_Recommended"] -
class_language_df["Avg_TFIDF_Recommended"]
)

recommended_language = class_language_df.sort_values(
    "Lift_Recommended", ascending=False
).head(15).copy()

not_recommended_language = class_language_df.sort_values(
    "Lift_Not_Recommended", ascending=False
).head(15).copy()

show_table(
    "LANGUAGE MOST CHARACTERISTIC OF RECOMMENDED REVIEWS",
    recommended_language,
    index=False
)

show_table(
    "LANGUAGE MOST CHARACTERISTIC OF NOT-RECOMMENDED REVIEWS",
    not_recommended_language,
    index=False
)

# %% -----
# 4.b.25 Plot 8: Language contrast between classes
# -----
recommended_language_plot = recommended_language.head(12).copy()

```

```

not_recommended_language_plot = not_recommended_language.head(12).copy()

rec_plot = (
    recommended_language_plot[["Feature", "Lift_Recommended"]]
    .rename(columns={"Lift_Recommended": "Lift"})
    .sort_values("Lift", ascending=True)
    .copy()
)

not_plot = (
    not_recommended_language_plot[["Feature", "Lift_Not_Recommended"]]
    .rename(columns={"Lift_Not_Recommended": "Lift"})
    .sort_values("Lift", ascending=True)
    .copy()
)

max_val = max(rec_plot["Lift"].max(), not_plot["Lift"].max())

fig, axes = plt.subplots(1, 2, figsize=(15.5, 8.2), sharex=True)

axes[0].hlines(
    y=rec_plot["Feature"],
    xmin=0,
    xmax=rec_plot["Lift"],
    linewidth=2.4,
    color="#DD8452",
    alpha=0.85
)
axes[0].scatter(
    rec_plot["Lift"],
    rec_plot["Feature"],
    s=85,
    color="#DD8452",
    zorder=3
)
axes[0].set_xlim(0, max_val * 1.08)
axes[0].set_xlabel("Average TF-IDF Lift")
axes[0].set_ylabel("")
axes[0].set_title("Key Positive Language Driving Recommendations",
    fontsize=13, weight="bold")
axes[0].grid(axis="x", color="#e6e6e6", linewidth=0.8)
axes[0].set_axisbelow(True)

axes[1].hlines(
    y=not_plot["Feature"],
    xmin=0,
    xmax=not_plot["Lift"],
    linewidth=2.4,
    color="#4C72B0",
    alpha=0.85
)
axes[1].scatter(
    not_plot["Lift"],
    not_plot["Feature"],
    s=85,
    color="#4C72B0",
    zorder=3
)
axes[1].set_xlim(0, max_val * 1.08)
axes[1].set_xlabel("Average TF-IDF Lift")
axes[1].set_ylabel("")

```

```

axes[1].set_title("Key Negative Language Driving Non-Recommendations",
    fontsize=13, weight="bold")
axes[1].grid(axis="x", color="#e6e6e6", linewidth=0.8)
axes[1].set_axisbelow(True)

plt.suptitle(
    "Distinctive Language Patterns in Recommended and Not-Recommended
Reviews",
    fontsize=15,
    weight="bold",
    y=0.98
)

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()

# %% -----
# 4.b.26 Misclassified reviews for qualitative follow-up
# -----
if text_log_results["ROC_AUC"] >= text_rf_results["ROC_AUC"]:
    best_text_model_name = "Text Logistic Regression"
    best_pred = text_log_pred
    best_prob = text_log_prob
else:
    best_text_model_name = "Text Random Forest"
    best_pred = text_rf_pred
    best_prob = text_rf_prob

misclassified_text_df = pd.DataFrame({
    "Review_ID": id_test_text.values,
    "Actual": y_test_text.values,
    "Predicted": best_pred,
    "Predicted_Probability": best_prob
})

misclassified_text_df["Correct"] = np.where(
    misclassified_text_df["Actual"] == misclassified_text_df["Predicted"],
    "Yes",
    "No"
)

misclassified_text_df["Prediction_Confidence"] = np.where(
    misclassified_text_df["Predicted"] == 1,
    misclassified_text_df["Predicted_Probability"],
    1 - misclassified_text_df["Predicted_Probability"]
)

misclassified_text_df = misclassified_text_df.merge(
    text_df[[
        "Review_ID",
        "Review",
        "Overall_Rating",
        "vader_score",
        "positive_cue_count",
        "negative_cue_count"
    ]],
    on="Review_ID",
    how="left"
)

top_misclassified_reviews = misclassified_text_df[

```

```

        misclassified_text_df["Correct"] == "No"
    ].sort_values("Prediction_Confidence", ascending=False).head(10)

show_table(
    f"TOP 10 MISCLASSIFIED REVIEWS FROM {best_text_model_name}",
    top_misclassified_reviews,
    index=False
)

# %% =====
# 4.c. PREDICTIVE ANALYTICS COMBINING NUMERICAL AND TEXT FEATURES
# =====

# %% -----
# 4.c.1 Proposed feature integration strategies
# -----
integration_strategies = pd.DataFrame([
    {
        "Strategy": "Early Fusion",
        "Description": "Concatenate scaled numerical features and TF-IDF
text features into one feature matrix, then train a single classifier.",
        "Implemented": "Yes"
    },
    {
        "Strategy": "Late Fusion",
        "Description": "Train numerical and text models separately, then
combine predicted probabilities using a simple average or weighted
average.",
        "Implemented": "No"
    }
])

show_table("4.c.1 PROPOSED FEATURE INTEGRATION STRATEGIES",
integration_strategies, index=False)

# %% -----
# 4.c.2 Combined modelling dataset
# -----
combined_df = df_clean[[
    "Review",
    "Review Date",
    "Recommended"
] + model_features].copy()

combined_df["review_clean_basic"] =
combined_df["Review"].apply(basic_clean_text)
combined_df["review_tokens"] =
combined_df["review_clean_basic"].apply(tokenize_and_lemmatize)
combined_df["review_clean_final"] =
combined_df["review_tokens"].apply(lambda x: " ".join(x))

combined_df = combined_df.dropna(subset=["review_clean_final",
"Recommended"]).copy()
combined_df["Review_ID"] = combined_df.index

print_section("4.c.2 COMBINED MODELLING DATASET")
print("Shape:", combined_df.shape)
print(combined_df["Recommended"].value_counts(normalize=True).round(4))

```

```

# %% -----
# 4.c.3 Train-test split for combined features
# -----
X_num_combined = combined_df[model_features].copy()
X_text_combined = combined_df["review_clean_final"].copy()
y_combined = combined_df["Recommended"].copy()

(
    X_num_train,
    X_num_test,
    X_text_train,
    X_text_test,
    y_train_combined,
    y_test_combined,
    id_train_combined,
    id_test_combined
) = train_test_split(
    X_num_combined,
    X_text_combined,
    y_combined,
    combined_df["Review_ID"],
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y_combined
)

print_section("4.c.3 COMBINED TRAIN-TEST SPLIT")
print("Numerical train shape:", X_num_train.shape)
print("Numerical test shape :", X_num_test.shape)
print("Text train shape      :", X_text_train.shape)
print("Text test shape       :", X_text_test.shape)
print(y_train_combined.value_counts(normalize=True).round(4))
print(y_test_combined.value_counts(normalize=True).round(4))

# %% -----
# 4.c.4 Early fusion feature matrix
# -----
combined_scaler = StandardScaler()
X_num_train_scaled = combined_scaler.fit_transform(X_num_train)
X_num_test_scaled = combined_scaler.transform(X_num_test)

combined_tfidf = TfidfVectorizer(
    ngram_range=(1, 2),
    min_df=5,
    max_df=0.90,
    sublinear_tf=True
)

X_text_train_tfidf = combined_tfidf.fit_transform(X_text_train)
X_text_test_tfidf = combined_tfidf.transform(X_text_test)

X_num_train_sparse = csr_matrix(X_num_train_scaled)
X_num_test_sparse = csr_matrix(X_num_test_scaled)

X_train_fused = hstack([X_num_train_sparse, X_text_train_tfidf],
    format="csr")
X_test_fused = hstack([X_num_test_sparse, X_text_test_tfidf], format="csr")

combined_feature_names = np.concatenate([
    np.array([f"NUM_{col}" for col in model_features]),

```



```

    np.array([f"TEXT_{feat}" for feat in
combined_tfidf.get_feature_names_out()])
])

print_section("4.c.4 EARLY FUSION FEATURE SPACE")
print("Fused training matrix shape:", X_train_fused.shape)
print("Fused testing matrix shape :", X_test_fused.shape)
print("Numerical feature count      :", len(model_features))
print("Text feature count           :",
len(combined_tfidf.get_feature_names_out()))
print("Total fused feature count    :", len(combined_feature_names))

# %% -----
# 4.c.5 Combined logistic regression
# -----
combined_log_model = LogisticRegression(
    max_iter=2000,
    random_state=RANDOM_STATE
)

combined_log_model.fit(X_train_fused, y_train_combined)

combined_log_pred = combined_log_model.predict(X_test_fused)
combined_log_prob = combined_log_model.predict_proba(X_test_fused)[:, 1]

combined_log_results = evaluate_model(
    y_test_combined,
    combined_log_pred,
    combined_log_prob,
    "Combined Logistic Regression"
)

combined_log_cm = print_conf_matrix(
    y_test_combined,
    combined_log_pred,
    "Combined Logistic Regression"
)

# %% -----
# 4.c.6 Optional benchmark: combined linear SVM
# -----
combined_svm_base = LinearSVC(
    random_state=RANDOM_STATE
)

combined_svm_model = CalibratedClassifierCV(
    estimator=combined_svm_base,
    method="sigmoid",
    cv=3
)

combined_svm_model.fit(X_train_fused, y_train_combined)

combined_svm_pred = combined_svm_model.predict(X_test_fused)
combined_svm_prob = combined_svm_model.predict_proba(X_test_fused)[:, 1]

combined_svm_results = evaluate_model(
    y_test_combined,
    combined_svm_pred,
    combined_svm_prob,

```

```

    "Combined Linear SVM"
)

combined_svm_cm = print_conf_matrix(
    y_test_combined,
    combined_svm_pred,
    "Combined Linear SVM"
)

# %% -----
# 4.c.7 Performance impact of feature integration
# -----
if text_log_results["ROC_AUC"] >= text_rf_results["ROC_AUC"]:
    best_core_text_name = "Text Logistic Regression"
    best_core_text_results = text_log_results
else:
    best_core_text_name = "Text Random Forest"
    best_core_text_results = text_rf_results

combined_comparison = pd.DataFrame([
    {
        "Model": "Best Numerical Model (Random Forest)",
        "Accuracy": rf_results["Accuracy"],
        "Precision": rf_results["Precision"],
        "Recall": rf_results["Recall"],
        "F1": rf_results["F1"],
        "ROC_AUC": rf_results["ROC_AUC"]
    },
    {
        "Model": f"Best Core Text Model ({best_core_text_name.replace('Text', ' ')})",
        "Accuracy": best_core_text_results["Accuracy"],
        "Precision": best_core_text_results["Precision"],
        "Recall": best_core_text_results["Recall"],
        "F1": best_core_text_results["F1"],
        "ROC_AUC": best_core_text_results["ROC_AUC"]
    },
    {
        "Model": "Text Linear SVM (Benchmark)",
        "Accuracy": text_svm_results["Accuracy"],
        "Precision": text_svm_results["Precision"],
        "Recall": text_svm_results["Recall"],
        "F1": text_svm_results["F1"],
        "ROC_AUC": text_svm_results["ROC_AUC"]
    },
    {
        "Model": "Combined Logistic Regression",
        "Accuracy": combined_log_results["Accuracy"],
        "Precision": combined_log_results["Precision"],
        "Recall": combined_log_results["Recall"],
        "F1": combined_log_results["F1"],
        "ROC_AUC": combined_log_results["ROC_AUC"]
    },
    {
        "Model": "Combined Linear SVM",
        "Accuracy": combined_svm_results["Accuracy"],
        "Precision": combined_svm_results["Precision"],
        "Recall": combined_svm_results["Recall"],
        "F1": combined_svm_results["F1"],
        "ROC_AUC": combined_svm_results["ROC_AUC"]
    }
])

```

```

    }
).set_index("Model").round(4)

show_table("4.c.7 PERFORMANCE IMPACT OF FEATURE INTEGRATION",
combined_comparison)

combined_delta = pd.DataFrame([
    {
        "Comparison": "Combined Logistic vs Best Numerical",
        "Accuracy_Delta": combined_log_results["Accuracy"] -
rf_results["Accuracy"],
        "Recall_Delta": combined_log_results["Recall"] -
rf_results["Recall"],
        "F1_Delta": combined_log_results["F1"] - rf_results["F1"],
        "ROC_AUC_Delta": combined_log_results["ROC_AUC"] -
rf_results["ROC_AUC"]
    },
    {
        "Comparison": "Combined Logistic vs Best Core Text Model",
        "Accuracy_Delta": combined_log_results["Accuracy"] -
best_core_text_results["Accuracy"],
        "Recall_Delta": combined_log_results["Recall"] -
best_core_text_results["Recall"],
        "F1_Delta": combined_log_results["F1"] -
best_core_text_results["F1"],
        "ROC_AUC_Delta": combined_log_results["ROC_AUC"] -
best_core_text_results["ROC_AUC"]
    },
    {
        "Comparison": "Combined Logistic vs Text SVM Benchmark",
        "Accuracy_Delta": combined_log_results["Accuracy"] -
text_svm_results["Accuracy"],
        "Recall_Delta": combined_log_results["Recall"] -
text_svm_results["Recall"],
        "F1_Delta": combined_log_results["F1"] - text_svm_results["F1"],
        "ROC_AUC_Delta": combined_log_results["ROC_AUC"] -
text_svm_results["ROC_AUC"]
    },
    {
        "Comparison": "Combined SVM vs Best Numerical",
        "Accuracy_Delta": combined_svm_results["Accuracy"] -
rf_results["Accuracy"],
        "Recall_Delta": combined_svm_results["Recall"] -
rf_results["Recall"],
        "F1_Delta": combined_svm_results["F1"] - rf_results["F1"],
        "ROC_AUC_Delta": combined_svm_results["ROC_AUC"] -
rf_results["ROC_AUC"]
    },
    {
        "Comparison": "Combined SVM vs Best Core Text Model",
        "Accuracy_Delta": combined_svm_results["Accuracy"] -
best_core_text_results["Accuracy"],
        "Recall_Delta": combined_svm_results["Recall"] -
best_core_text_results["Recall"],
        "F1_Delta": combined_svm_results["F1"] -
best_core_text_results["F1"],
        "ROC_AUC_Delta": combined_svm_results["ROC_AUC"] -
best_core_text_results["ROC_AUC"]
    },
    {
        "Comparison": "Combined SVM vs Text SVM Benchmark",

```

```

        "Accuracy_Delta": combined_svm_results["Accuracy"] -
text_svm_results["Accuracy"],
        "Recall_Delta": combined_svm_results["Recall"] -
text_svm_results["Recall"],
        "F1_Delta": combined_svm_results["F1"] - text_svm_results["F1"],
        "ROC_AUC_Delta": combined_svm_results["ROC_AUC"] -
text_svm_results["ROC_AUC"]
    }
}).set_index("Comparison").round(4)

show_table("4.c.7 CHANGE IN PERFORMANCE AFTER INTEGRATION", combined_delta)

# %% -----
# 4.c.6B Optional benchmark: combined Random Forest
# -----
combined_rf_model = RandomForestClassifier(
    n_estimators=300,
    random_state=RANDOM_STATE,
    n_jobs=-1
)

combined_rf_model.fit(X_train_fused, y_train_combined)

combined_rf_pred = combined_rf_model.predict(X_test_fused)
combined_rf_prob = combined_rf_model.predict_proba(X_test_fused)[:, 1]

combined_rf_results = evaluate_model(
    y_test_combined,
    combined_rf_pred,
    combined_rf_prob,
    "Combined Random Forest"
)

combined_rf_cm = print_conf_matrix(
    y_test_combined,
    combined_rf_pred,
    "Combined Random Forest"
)

# %% -----
# 4.c.8 Dominant predictors in the combined logistic model
# -----
combined_coef_df = pd.DataFrame({
    "Feature": combined_feature_names,
    "Coefficient": combined_log_model.coef_[0]
})

combined_coef_df["Feature_Type"] = np.where(
    combined_coef_df["Feature"].str.startswith("NUM__"),
    "Numerical",
    "Text"
)

combined_coef_df["Abs_Coefficient"] = combined_coef_df["Coefficient"].abs()

top_combined_positive = (
    combined_coef_df
    .sort_values("Coefficient", ascending=False)
    .head(20)
    .copy()
)

```

```

top_combined_negative = (
    combined_coef_df
    .sort_values("Coefficient", ascending=True)
    .head(20)
    .copy()
)

show_table(
    "4.c.8 TOP POSITIVE PREDICTORS IN THE COMBINED LOGISTIC MODEL",
    top_combined_positive,
    index=False
)
show_table(
    "4.c.8 TOP NEGATIVE PREDICTORS IN THE COMBINED LOGISTIC MODEL",
    top_combined_negative,
    index=False
)

dominance_summary = (
    combined_coef_df
    .sort_values("Abs_Coefficient", ascending=False)
    .head(30)
    .groupby("Feature_Type")
    .size()
    .reset_index(name="Count_in_Top_30")
)

show_table(
    "4.c.8 FEATURE DOMINANCE IN TOP 30 COMBINED LOGISTIC PREDICTORS",
    dominance_summary,
    index=False
)

# %% -----
# 4.c.9 Plot 1: Performance comparison across modelling approaches
# -----
best_core_text_label = best_core_text_name.replace("Text ", "")

combined_candidates = [
    ("Combined Logistic Regression", combined_log_results),
    ("Combined Linear SVM", combined_svm_results),
    ("Combined Random Forest", combined_rf_results),
]

best_combined_name, best_combined_results = max(
    combined_candidates,
    key=lambda x: x[1]["ROC_AUC"]
)

combined_comparison_plot = pd.DataFrame([
    {
        "Model": "Best Numerical Model\n(Random Forest)",
        "Accuracy": rf_results["Accuracy"],
        "Recall": rf_results["Recall"],
        "F1": rf_results["F1"],
        "ROC_AUC": rf_results["ROC_AUC"]
    },
    {
        "Model": f"Best Core Text Model\n({best_core_text_label})",

```

```

        "Accuracy": best_core_text_results["Accuracy"],
        "Recall": best_core_text_results["Recall"],
        "F1": best_core_text_results["F1"],
        "ROC_AUC": best_core_text_results["ROC_AUC"]
    },
    {
        "Model": "Text Linear SVM\n(Benchmark)",
        "Accuracy": text_svm_results["Accuracy"],
        "Recall": text_svm_results["Recall"],
        "F1": text_svm_results["F1"],
        "ROC_AUC": text_svm_results["ROC_AUC"]
    },
    {
        "Model": "Combined Logistic Regression",
        "Accuracy": combined_log_results["Accuracy"],
        "Recall": combined_log_results["Recall"],
        "F1": combined_log_results["F1"],
        "ROC_AUC": combined_log_results["ROC_AUC"]
    },
    {
        "Model": "Combined Linear SVM",
        "Accuracy": combined_svm_results["Accuracy"],
        "Recall": combined_svm_results["Recall"],
        "F1": combined_svm_results["F1"],
        "ROC_AUC": combined_svm_results["ROC_AUC"]
    },
    {
        "Model": "Combined Random Forest",
        "Accuracy": combined_rf_results["Accuracy"],
        "Recall": combined_rf_results["Recall"],
        "F1": combined_rf_results["F1"],
        "ROC_AUC": combined_rf_results["ROC_AUC"]
    }
]

model_order = [
    "Best Numerical Model\n(Random Forest)",
    f"Best Core Text Model\n({best_core_text_label})",
    "Text Linear SVM\n(Benchmark)",
    "Combined Logistic Regression",
    "Combined Linear SVM",
    "Combined Random Forest"
]

plot_metrics_df = combined_comparison_plot.melt(
    id_vars="Model",
    value_vars=["Accuracy", "Recall", "F1", "ROC_AUC"],
    var_name="Metric",
    value_name="Score"
)

plot_metrics_df["Model"] = pd.Categorical(
    plot_metrics_df["Model"],
    categories=model_order,
    ordered=True
)

plt.figure(figsize=(14, 7.4))
ax = sns.barplot(
    data=plot_metrics_df,
    x="Metric",

```

```

        y="Score",
        hue="Model",
        hue_order=model_order
    )

    plt.ylim(0.88, 1.00)
    plt.ylabel("Performance Score")
    plt.xlabel("")
    plt.grid(axis="y", color="#e6e6e6", linewidth=0.8)
    plt.gca().set_axisbelow(True)

    for container in ax.containers:
        ax.bar_label(container, fmt="%.3f", fontsize=8)

    plt.legend(
        title="Model",
        frameon=True,
        bbox_to_anchor=(1.02, 1),
        loc="upper left"
    )

    finalise_plot("Performance Comparison Across Numerical, Text, and Combined
Models")

# %% -----
# 4.c.10 Plot 2: Top predictors in the combined logistic model
# -----
top_n_each_side = 16

top_combined_positive = (
    combined_coef_df
    .sort_values("Coefficient", ascending=False)
    .head(top_n_each_side)
    .copy()
)

top_combined_negative = (
    combined_coef_df
    .sort_values("Coefficient", ascending=True)
    .head(top_n_each_side)
    .copy()
)

top_combined_plot = pd.concat(
    [top_combined_negative, top_combined_positive],
    axis=0
).copy()

top_combined_plot["Feature_Type"] = np.where(
    top_combined_plot["Feature"].str.startswith("NUM__"),
    "Numerical",
    "Text"
)

drop_features = {
    "TEXT__another",
    "TEXT__told",
    "TEXT__okay",
    "TEXT__sof",
    "TEXT__overall",

```

```

        "TEXT__flew",
        "TEXT__front",
        "TEXT__new",
    }

    top_combined_plot = top_combined_plot[
        ~top_combined_plot["Feature"].isin(drop_features)
    ].copy()

    neg_plot = (
        top_combined_plot[top_combined_plot["Coefficient"] < 0]
        .sort_values("Coefficient", ascending=True)
        .head(12)
        .copy()
    )

    pos_plot = (
        top_combined_plot[top_combined_plot["Coefficient"] > 0]
        .sort_values("Coefficient", ascending=False)
        .head(12)
        .copy()
    )

    top_combined_plot = pd.concat([neg_plot, pos_plot], axis=0).copy()

    display_name_map = {
        "NUM__Value For Money": "Value for Money",
        "TEXT__flight delayed": "flight delayed",
        "TEXT__friendly": "friendly",
        "TEXT__good": "good",
        "TEXT__excellent": "excellent",
        "TEXT__great": "great",
        "TEXT__pleasant": "pleasant",
        "TEXT__nice": "nice",
        "TEXT__thank": "thank",
        "TEXT__comfortable": "comfortable",
        "TEXT__efficient": "efficient",
        "TEXT__drink": "drink",
        "TEXT__clean": "clean",
        "TEXT__best": "best",
        "TEXT__helpful": "helpful",
        "TEXT__impressed": "impressed",
        "TEXT__flight time": "flight time",
        "TEXT__not": "not",
        "TEXT__no": "no",
        "TEXT__delayed": "delayed",
        "TEXT__hour": "hour",
        "TEXT__worst": "worst",
        "TEXT__never": "never",
        "TEXT__terrible": "terrible",
        "TEXT__poor": "poor",
        "TEXT__disappointing": "disappointing",
        "TEXT__would not": "would not",
        "TEXT__disappointed": "disappointed",
        "TEXT__cannot": "cannot",
        "TEXT__customer": "customer",
        "TEXT__changed": "changed",
    }

    top_combined_plot["Display_Feature"] =
    top_combined_plot["Feature"].map(display_name_map)

```



```

top_combined_plot["Display_Feature"] =
top_combined_plot["Display_Feature"].fillna(
    top_combined_plot["Feature"]
    .str.replace("NUM__", "", regex=False)
    .str.replace("TEXT__", "", regex=False)
)

top_combined_plot = (
    top_combined_plot
    .sort_values("Coefficient", ascending=True)
    .reset_index(drop=True)
)

fig, ax = plt.subplots(figsize=(11.8, 8.0))

text_color = "#4C72B0"
num_color = "#DD8452"

ax.axvline(0, color="black", linewidth=1.0, alpha=0.75, zorder=1)

for _, row in top_combined_plot.iterrows():
    line_color = text_color if row["Feature_Type"] == "Text" else num_color
    ax.hlines(
        y=row["Display_Feature"],
        xmin=0,
        xmax=row["Coefficient"],
        color=line_color,
        linewidth=2.0,
        alpha=0.78,
        zorder=2
    )

text_features_plot = top_combined_plot[top_combined_plot["Feature_Type"] ==
"Text"]
text_scatter = ax.scatter(
    text_features_plot["Coefficient"],
    text_features_plot["Display_Feature"],
    s=95,
    color=text_color,
    edgecolor="white",
    linewidth=0.8,
    label="Text",
    zorder=3
)

num_features_plot = top_combined_plot[top_combined_plot["Feature_Type"] ==
"Numerical"]
num_scatter = ax.scatter(
    num_features_plot["Coefficient"],
    num_features_plot["Display_Feature"],
    s=180,
    marker="x",
    color=num_color,
    edgecolor="black",
    linewidth=1.8,
    label="Numerical",
    zorder=4
)

limit = np.ceil(np.max(np.abs(top_combined_plot["Coefficient"]))) * 10) / 10
ax.set_xlim(-limit - 0.25, limit + 0.25)

```

```

ax.set_xlabel("Logistic Regression Coefficient", fontsize=12)
ax.set_ylabel("")
ax.grid(axis="x", color="#e6e6e6", linewidth=0.8)
ax.set_axisbelow(True)
ax.margins(y=0.02)

for spine in ["top", "right", "left"]:
    ax.spines[spine].set_visible(False)

ax.tick_params(axis="y", labelsize=11)
ax.tick_params(axis="x", labelsize=11)

ax.set_title(
    "Top Predictors in the Combined Logistic Model (Early Fusion)",
    fontsize=16,
    fontweight="bold",
    pad=22
)

ax.text(
    0.18, 1.015,
    "← Drives Non-Recommendation",
    transform=ax.transAxes,
    fontsize=10,
    ha="center",
    va="bottom"
)

ax.text(
    0.82, 1.015,
    "Drives Recommendation →",
    transform=ax.transAxes,
    fontsize=10,
    ha="center",
    va="bottom"
)

legend = ax.legend(
    handles=[text_scatter, num_scatter],
    title="Feature Type",
    frameon=True,
    loc="lower right",
    bbox_to_anchor=(0.98, 0.12)
)
legend.get_title().set_fontsize(11)

plt.tight_layout(rect=[0, 0, 1, 0.92])
plt.show()

# %% -----
# 4.c.11 Brief failure analysis for the combined model
# -----
best_combined_name = "Combined Logistic Regression"
best_combined_pred = combined_log_pred
best_combined_prob = combined_log_prob

combined_failure_df = pd.DataFrame({
    "Review_ID": id_test_combined.values,
    "Actual": y_test_combined.astype(int).to_numpy(),
    "Predicted": pd.Series(best_combined_pred).astype(int).to_numpy(),

```

```

        "Predicted_Probability": best_combined_prob
    })

combined_failure_df["Correct"] = np.where(
    combined_failure_df["Actual"] == combined_failure_df["Predicted"],
    "Yes",
    "No"
)

combined_failure_df["Prediction_Confidence"] = np.where(
    combined_failure_df["Predicted"] == 1,
    combined_failure_df["Predicted_Probability"],
    1 - combined_failure_df["Predicted_Probability"]
)

failure_context = combined_df[[
    "Review_ID",
    "Review",
    "review_clean_final",
    "Overall_Rating",
    "Value For Money",
    "Seat Comfort",
    "Cabin Staff Service",
    "Ground Service"
]].copy()

combined_failure_df = combined_failure_df.merge(
    failure_context,
    on="Review_ID",
    how="left"
)

top_combined_failures = combined_failure_df[
    combined_failure_df["Correct"] == "No"
].sort_values("Prediction_Confidence", ascending=False).head(10)

show_table(
    f"4.c.11 TOP 10 MISCLASSIFIED CASES FROM {best_combined_name}",
    top_combined_failures,
    index=False
)

# %% =====
# APPENDIX A. ADDITIONAL EXPLORATORY ANALYSIS
# =====

# %% -----
# A.1 Helper copies and safe standardisation
# -----
appendix_df = df_clean.copy()

for col in ["Verified", "Aircraft", "Type Of Traveller", "Seat Type",
"Route"]:
    if col in appendix_df.columns:
        appendix_df[col] = appendix_df[col].astype("string").str.strip()

appendix_df["Recommended_Label"] = appendix_df["Recommended"].map({1:
"Yes", 0: "No"})

# %% -----

```

```

# A.2 VERIFIED: counts, recommendation rate, and rating profile
# -----
if "Verified" in appendix_df.columns:
    verified_df = appendix_df.dropna(subset=["Verified",
"Recommended"]).copy()

    verified_df["Verified_Clean"] = (
        verified_df["Verified"]
        .str.lower()
        .replace({
            "true": "Verified",
            "false": "Not Verified",
            "yes": "Verified",
            "no": "Not Verified"
        })
    )

    verified_df = verified_df[
        verified_df["Verified_Clean"].isin(["Verified", "Not Verified"])
    ].copy()

    verified_summary = (
        verified_df.groupby("Verified_Clean", as_index=False)
        .agg(
            Review_Count=("Recommended", "size"),
            Recommendation_Rate=("Recommended", "mean"),
            Mean_Overall_Rating=("Overall_Rating", "mean"),
            Mean_Value_For_Money=("Value For Money", "mean")
        )
    )

    verified_summary["Recommendation_Rate"] =
    (verified_summary["Recommendation_Rate"] * 100).round(1)
    verified_summary["Mean_Overall_Rating"] =
    verified_summary["Mean_Overall_Rating"].round(2)
    verified_summary["Mean_Value_For_Money"] =
    verified_summary["Mean_Value_For_Money"].round(2)

    show_table("APPENDIX A.2 VERIFIED REVIEW SUMMARY", verified_summary,
    index=False)

    verified_crosstab = pd.crosstab(
        verified_df["Verified_Clean"],
        verified_df["Recommended_Label"],
        margins=True
    )
    show_table("APPENDIX A.2 VERIFIED BY RECOMMENDATION CROSSTAB",
    verified_crosstab)

    plt.figure(figsize=(8.4, 5.4))

    order = ["Verified", "Not Verified"]
    plot_df =
    verified_summary.set_index("Verified_Clean").reindex(order).reset_index()

    bars = plt.bar(
        plot_df["Verified_Clean"],
        plot_df["Recommendation_Rate"],
        color=["#2F5D8A", "#B7C7D9"],
        edgecolor="black",
        linewidth=0.8,

```

```

        width=0.5
    )

    for bar, rate, count in zip(bars, plot_df["Recommendation_Rate"],
plot_df["Review_Count"]):
        plt.text(
            bar.get_x() + bar.get_width() / 2,
            rate + 1.2,
            f"{rate:.1f}%\n(n={count:,})",
            ha="center",
            va="bottom",
            fontsize=9
        )

    plt.ylim(0, min(100, plot_df["Recommendation_Rate"].max() + 12))
    plt.xlabel("")
    plt.ylabel("Recommendation Rate (%)")
    plt.grid(axis="y", color="#e6e6e6", linewidth=0.8)
    plt.gca().grid(False, axis="x")
    plt.gca().set_axisbelow(True)

    ax = plt.gca()
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.spines["left"].set_color("#c0c0c0")
    ax.spines["bottom"].set_color("#c0c0c0")

    finalise_plot("Appendix A.2 Recommendation Rate by Verification
Status")

# %% -----
# A.3 AIRCRAFT BRAND / FAMILY ANALYSIS
# -----
def classify_aircraft_brand(text: str) -> str:
    """Map aircraft descriptions into broad aircraft-brand families."""
    if pd.isna(text):
        return np.nan

    t = str(text).lower().strip()

    if t == "" or t == "nan":
        return np.nan

    if any(x in t for x in [
        "boeing", "b737", "b-737", "737", "b738", "b739", "b747", "747",
        "b757", "757", "b767", "767", "b777", "777", "b787", "787",
        "max 8", "max8", "max 9", "max9", "dreamliner"
    ]):
        return "Boeing"

    if any(x in t for x in [
        "airbus", "a220", "a300", "a310", "a318", "a319", "a320", "a321",
        "a330", "a340", "a350", "a380"
    ]):
        return "Airbus"

    if any(x in t for x in ["embraer", "emb", "e170", "e175", "e190",
    "e195", "erj"]):
        return "Embraer"

```

```

        if any(x in t for x in ["bombardier", "crj", "dash 8", "dash8", "q400",
                                "q300", "q200", "cseries"]):
            return "Bombardier / De Havilland"

        if any(x in t for x in ["atr 42", "atr42", "atr 72", "atr72", "atr"]):
            return "ATR"

        if any(x in t for x in ["md-", "md ", "md80", "md81", "md82", "md83",
                                "md87", "md88", "md90", "dc-"]):
            return "McDonnell Douglas"

        if any(x in t for x in ["fokker", "saab", "tupolev", "ilyushin",
                                "antonov", "comac", "yak", "sukhoi", "superjet"]):
            return "Other Manufacturer"

    return "Other / Unclear"

if "Aircraft" in appendix_df.columns:
    aircraft_df = appendix_df.dropna(subset=["Aircraft",
                                              "Recommended"]).copy()
    aircraft_df["Aircraft_Brand"] =
aircraft_df["Aircraft"].apply(classify_aircraft_brand)

    aircraft_summary = (
        aircraft_df.dropna(subset=["Aircraft_Brand"])
        .groupby("Aircraft_Brand", as_index=False)
        .agg(
            Review_Count=("Recommended", "size"),
            Recommendation_Rate=("Recommended", "mean"),
            Mean_Overall_Rating=("Overall_Rating", "mean"),
            Mean_Value_For_Money=("Value For Money", "mean")
        )
        .sort_values("Review_Count", ascending=False)
    )

    aircraft_summary["Recommendation_Rate"] =
    (aircraft_summary["Recommendation_Rate"] * 100).round(1)
    aircraft_summary["Mean_Overall_Rating"] =
    aircraft_summary["Mean_Overall_Rating"].round(2)
    aircraft_summary["Mean_Value_For_Money"] =
    aircraft_summary["Mean_Value_For_Money"].round(2)

    show_table("APPENDIX A.3 AIRCRAFT BRAND SUMMARY", aircraft_summary,
              index=False)

    aircraft_plot_df = aircraft_summary[aircraft_summary["Review_Count"] >=
100].copy()
    aircraft_plot_df = aircraft_plot_df.sort_values("Recommendation_Rate",
ascending=True)

    if len(aircraft_plot_df) > 0:
        plt.figure(figsize=(9.6, 6.2))

        bars = plt.barh(
            aircraft_plot_df["Aircraft_Brand"],
            aircraft_plot_df["Recommendation_Rate"],
            color="#6F8FAF",
            edgecolor="black",
            linewidth=0.8,
            height=0.72

```

```

    )

    for bar, rate, count in zip(bars,
aircraft_plot_df["Recommendation_Rate"], aircraft_plot_df["Review_Count"]):
        plt.text(
            rate + 0.8,
            bar.get_y() + bar.get_height() / 2,
            f"{rate:.1f}% (n={count:,})",
            va="center",
            ha="left",
            fontsize=9
        )

plt.xlim(0, min(100, aircraft_plot_df["Recommendation_Rate"].max()
+ 12))

plt.xlabel("Recommendation Rate (%)")
plt.ylabel("")
plt.grid(axis="x", color="#e6e6e6", linewidth=0.8)
plt.gca().grid(False, axis="y")
plt.gca().set_axisbelow(True)

ax = plt.gca()
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.spines["left"].set_color("#c0c0c0")
ax.spines["bottom"].set_color("#c0c0c0")

finalise_plot("Appendix A.3 Recommendation Rate by Aircraft Brand
Family")

# %% -----
# A.4 TYPE OF TRAVELLER ANALYSIS
# -----
if "Type Of Traveller" in appendix_df.columns:
    traveller_df = appendix_df.dropna(subset=["Type Of Traveller",
"Recommended"]).copy()

    traveller_summary = (
        traveller_df.groupby("Type Of Traveller", as_index=False)
        .agg(
            Review_Count=("Recommended", "size"),
            Recommendation_Rate=("Recommended", "mean"),
            Mean_Overall_Rating=("Overall_Rating", "mean"),
            Mean_Value_For_Money=("Value For Money", "mean")
        )
        .sort_values("Recommendation_Rate", ascending=False)
    )

    traveller_summary["Recommendation_Rate"] =
(traveller_summary["Recommendation_Rate"] * 100).round(1)
    traveller_summary["Mean_Overall_Rating"] =
traveller_summary["Mean_Overall_Rating"].round(2)
    traveller_summary["Mean_Value_For_Money"] =
traveller_summary["Mean_Value_For_Money"].round(2)

    show_table("APPENDIX A.4 TYPE OF TRAVELLER SUMMARY", traveller_summary,
index=False)

    traveller_plot_df = traveller_summary[traveller_summary["Review_Count"]
>= 100].copy()

```

```

traveller_plot_df =
traveller_plot_df.sort_values("Recommendation_Rate", ascending=True)

plt.figure(figsize=(10.2, 6.4))

bars = plt.barh(
    traveller_plot_df["Type Of Traveller"],
    traveller_plot_df["Recommendation_Rate"],
    color="#4C72B0",
    edgecolor="black",
    linewidth=0.8,
    height=0.72
)

for bar, rate, count in zip(bars,
traveller_plot_df["Recommendation_Rate"],
traveller_plot_df["Review_Count"]):
    plt.text(
        rate + 0.8,
        bar.get_y() + bar.get_height() / 2,
        f"{rate:.1f}% (n={count:,})",
        va="center",
        ha="left",
        fontsize=9
    )

plt.xlim(0, min(100, traveller_plot_df["Recommendation_Rate"].max() +
12))
plt.xlabel("Recommendation Rate (%)")
plt.ylabel("")
plt.grid(axis="x", color="#e6e6e6", linewidth=0.8)
plt.gca().grid(False, axis="y")
plt.gca().set_axisbelow(True)

ax = plt.gca()
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.spines["left"].set_color("#c0c0c0")
ax.spines["bottom"].set_color("#c0c0c0")

finalise_plot("Appendix A.4 Recommendation Rate by Type of Traveller")

# %% -----
# A.5 SEAT TYPE ANALYSIS (appendix version)
# -----
if "Seat Type" in appendix_df.columns:
    appendix_seat_df = appendix_df.dropna(subset=["Seat Type",
"Recommended"]).copy()
    appendix_seat_df["Seat Type"] = appendix_seat_df["Seat
Type"].astype(str).str.strip()

    seat_order = ["First Class", "Business Class", "Premium Economy",
"Economy Class"]

    appendix_seat_summary = (
        appendix_seat_df.groupby("Seat Type", as_index=False)
        .agg(
            Review_Count=("Recommended", "size"),
            Recommendation_Rate=("Recommended", "mean"),
            Mean_Overall_Rating=("Overall_Rating", "mean"),

```



```

        Mean_Value_For_Money=("Value For Money", "mean")
    )
)

appendix_seat_summary = (
    appendix_seat_summary[appendix_seat_summary["Seat
Type"].isin(seat_order)]
    .assign(
        Seat_Type_Order=lambda x: pd.Categorical(
            x["Seat Type"],
            categories=seat_order,
            ordered=True
        )
    )
    .sort_values("Seat_Type_Order")
    .drop(columns="Seat_Type_Order")
)

appendix_seat_summary["Recommendation_Rate"] =
(appendix_seat_summary["Recommendation_Rate"] * 100).round(1)
appendix_seat_summary["Mean_Overall_Rating"] =
appendix_seat_summary["Mean_Overall_Rating"].round(2)
appendix_seat_summary["Mean_Value_For_Money"] =
appendix_seat_summary["Mean_Value_For_Money"].round(2)

show_table("APPENDIX A.5 SEAT TYPE SUMMARY", appendix_seat_summary,
index=False)

# %% -----
# A.6 ROUTE ANALYSIS: top routes only
# -----
if "Route" in appendix_df.columns:
    route_df = appendix_df.dropna(subset=["Route", "Recommended"]).copy()
    route_df["Route"] = route_df["Route"].astype(str).str.strip()

    route_summary = (
        route_df.groupby("Route", as_index=False)
        .agg(
            Review_Count=("Recommended", "size"),
            Recommendation_Rate=("Recommended", "mean"),
            Mean_Overall_Rating=("Overall_Rating", "mean"),
            Mean_Value_For_Money=("Value For Money", "mean")
        )
    )

    route_summary["Recommendation_Rate"] =
(route_summary["Recommendation_Rate"] * 100).round(1)
    route_summary["Mean_Overall_Rating"] =
route_summary["Mean_Overall_Rating"].round(2)
    route_summary["Mean_Value_For_Money"] =
route_summary["Mean_Value_For_Money"].round(2)

    top_routes = (
        route_summary.sort_values("Review_Count", ascending=False)
        .head(15)
        .copy()
    )

    show_table("APPENDIX A.6 TOP 15 ROUTES BY REVIEW COUNT", top_routes,
index=False)

```

```

route_plot_df = top_routes.sort_values("Recommendation_Rate",
ascending=True).copy()

plt.figure(figsize=(11.5, 7.6))

bars = plt.barh(
    route_plot_df["Route"],
    route_plot_df["Recommendation_Rate"],
    color="#8DAA91",
    edgecolor="black",
    linewidth=0.8,
    height=0.72
)

for bar, rate, count in zip(bars, route_plot_df["Recommendation_Rate"],
route_plot_df["Review_Count"]):
    plt.text(
        rate + 0.7,
        bar.get_y() + bar.get_height() / 2,
        f"{rate:.1f}% (n={count:,})",
        va="center",
        ha="left",
        fontsize=8.5
    )

plt.xlim(0, min(100, route_plot_df["Recommendation_Rate"].max() + 12))
plt.xlabel("Recommendation Rate (%)")
plt.ylabel("")
plt.grid(axis="x", color="#e6e6e6", linewidth=0.8)
plt.gca().grid(False, axis="y")
plt.gca().set_axisbelow(True)

ax = plt.gca()
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.spines["left"].set_color("#c0c0c0")
ax.spines["bottom"].set_color("#c0c0c0")

finalise_plot("Appendix A.6 Recommendation Rate Across the 15 Most
Frequent Routes")

```

Appendix B: Manual Validation of Sentiment Classification

<i>Review Group</i>	<i>Review ID</i>	<i>Overall Rating</i>	<i>Recommended</i>	<i>VADER Score</i>	<i>VADER Label</i>	<i>RoBERTa Label</i>	<i>Positive Cues</i>	<i>Negative Cues</i>
<i>VADER Misclassification</i>	17581	4	0	0.9520	Positive	Negative	8	2
<i>VADER Misclassification</i>	4675	4	0	0.6028	Positive	Negative	5	5
<i>VADER Misclassification</i>	13254	9	0	0.9970	Positive	Neutral	8	1
<i>VADER Misclassification</i>	12207	1	0	0.9542	Positive	Negative	0	9
<i>VADER Misclassification</i>	12216	2	0	0.9307	Positive	Negative	2	7
<i>RoBERTa Misclassification</i>	8077	6	1	0.9962	Positive	Negative	10	4
<i>RoBERTa Misclassification</i>	9197	1	1	0.9976	Positive	Negative	11	1
<i>RoBERTa Misclassification</i>	16961	7	1	0.9648	Positive	Negative	5	4
<i>RoBERTa Misclassification</i>	17207	7	1	0.9891	Positive	Negative	7	2
<i>RoBERTa Misclassification</i>	6093	5	0	-0.7786	Negative	Positive	4	5
<i>Ambiguous / Mixed</i>	6929	7	1	0.9952	Positive	Neutral	13	4
<i>Ambiguous / Mixed</i>	21101	7	1	0.9808	Positive	Neutral	11	3
<i>Ambiguous / Mixed</i>	17047	9	1	0.9833	Positive	Positive	11	2
<i>Ambiguous / Mixed</i>	21679	8	1	0.9892	Positive	Positive	9	3
<i>Ambiguous / Mixed</i>	6118	1	1	0.9978	Positive	Positive	8	4

Appendix C: Illustrative Review Examples

B1. Review 17581 – VADER Misclassification (Positive vs Non-Recommendation)

Overall Rating: 4

Recommended: No

VADER: Positive

RoBERTa: Negative

Review Text:

I made a round trip with SATA Air Azores from Lisbon to Ponta Delgada. The Blue lounge in Lisbon is nothing to write home about. First of all its located on the other side of the airport from where the SATA flights depart. Second the lounge was dirty. Not a good first impression when you fly business class. The flight to Ponta Delgada continues onward to Boston and boarding was quite chaotic. Boarding was announced and we had to make 2 queues which everybody did. There is no priority boarding with this airline. After the announcement nothing happened for 30 minutes and all out of sudden the announcement was made that passengers for Ponta Delgada should be standing in the right row and passengers for Boston should be standing in the left row so everybody had to switch positions again. After scanning the boarding passes it was down on the stairs were everybody had to wait for another 20 minutes before the busses arrived. After the ride to the remote stand we had to wait again for 15 minutes on the bus because the aircraft was not ready yet. During boarding there was no welcome from the crew. It seems they were too busy chit chatting with each other. The flight departed an hour late from scheduled departure time. The aircraft was clean but looked a bit outdated from the inside, strange since this aircraft is very new to their fleet. Food was very good. However the entertainment system is not working. After lunch I wanted to relax a little and decided to recline my seat (2K) which did not work. After notifying the crew that my seat was broken I was told: "It's just a short flight!" and that was it. Not a great attitude towards a business class passenger. On the way back to Lisbon it was a better experience. The lounge at Ponta Delgada airport is small but it's just a tiny airport so no problem. There were some fresh fruits, cookies, crackers, nuts and sufficient drinks in the lounge. The flight to Lisbon on the A320 departed on time this time. A friendly welcome from the crew and I settled into my seat. No real business class seats on the A320 but a standard economy seat with the middle seat blocked. Service on this flight was good. The crew was interested if I liked my stay on the Azores. Food on this flight was not so great. We got a very dry samosa with some vegetables and potatoes which were cold and very salty. The flight did arrive on time in Lisbon and after a bus ride from the remote stand (again) I arrived at the terminal building for my next flight to Amsterdam. To sum it up: On my outbound flight to the Azores the food was good but the crew was rude and the lounge was a mess. On the inbound flight the lounge and the crew was great but the food was not.

B2. Review 8077 – RoBERTa Misclassification (Negative vs Recommendation)

Overall Rating: 6

Recommended: Yes

VADER: Positive

RoBERTa: Negative

Review Text:

Hong Kong to Shanghai. Flight was initially delayed from 1200 to 1340 but then we boarded around 1325 and sat on the aircraft not getting airborne until 1600. This was due to "bad weather" in Shanghai although probably more likely the usual chinese air force airspace delays along this route which I am given to understand are almost constant. I accept this is no fault of the airline. However it might have been better to let us sit in the terminal rather than on the aircraft for so long. They gave us our meals on the ground, these were good. The flight was good, friendly staff dealing with loud and frustrated passengers well and spoke good english. The seat was comfortable. Check in and ground staff were friendly and the aircraft was clean. Return flight to Hong Kong was cancelled. This again due to "bad weather"/chinese airspace. Problems with every airline so not Dragonair fault. However two queues to ticket desk (both massive) when we arrived, were for different things. No one told us which queue to stand in so we split up and stood one in each, eventually got a standby number, came back 2 hours later and was put on a new flight. This flight left very soon so were given priority security and baggage tags. Missed our connection in HKG but staff helped us get accommodation and a new flight in HKG. Staff at Shanghai were slow, didn't tell us which queue to be in or what was going on. However check in clerk was helpful. Again not Dragonair fault but could've been handled much better. 4 hours of queuing and then a sprint to security not fun. Flight itself was on A330, clean and had ife. Meals were good, so were crew, seats slightly dated but fine and aircraft clean. Would recommend Dragonair, they are a good airline and delays not their fault, just staff in Shanghai need more training at how to deal with this especially as it happens frequently. They are excellent though when it comes to reorganising in Hong Kong and as sister airline to Cathay Pacific make connections to them as seamless and worry free as possible. As you can make it all (international flight and domestic flight connecting) under one booking they will sort hotels and new flights if you miss connection so would recommend Dragonair especially if you are doing international flight followed by domestic flights within China

Appendix D: Manual Review Coding and Classification Notes

Case Type	Review ID	Recommended	VADER Label	RoBERTa Label	Manual Interpretation
<i>VADER Misclassification</i>	17581	No	Positive	Negative	Mixed sentiment; positive cues present but overall experience dominated by operational failures
<i>VADER Misclassification</i>	4675	No	Positive	Negative	Mixed sentiment; conflicting service experiences across journey stages
<i>VADER Misclassification</i>	13254	No	Positive	Neutral	Positive tone but weak alignment with recommendation behaviour
<i>VADER Misclassification</i>	12207	No	Positive	Negative	Strong negative experience despite presence of positive lexical cues
<i>VADER Misclassification</i>	12216	No	Positive	Negative	Positive in-flight experience outweighed by severe delays and disruption
<i>RoBERTa Misclassification</i>	8077	Yes	Positive	Negative	Negative events described but attributed to external factors; recommendation retained
<i>RoBERTa Misclassification</i>	9197	Yes	Positive	Negative	Positive overall experience despite isolated negative elements
<i>RoBERTa Misclassification</i>	16961	Yes	Positive	Negative	Service issues present but not decisive in final judgement
<i>RoBERTa Misclassification</i>	17207	Yes	Positive	Negative	Mixed evaluation; positive service offsets operational inconvenience
<i>RoBERTa Misclassification</i>	6093	No	Negative	Positive	Positive elements present but outweighed by poor service quality
<i>Ambiguous Case</i>	6929	Yes	Positive	Neutral	Mixed and context-dependent evaluation across multiple journey stages
<i>Ambiguous Case</i>	21101	Yes	Positive	Neutral	Conflicting cues with no clear dominant sentiment
<i>Ambiguous Case</i>	17047	Yes	Positive	Positive	Strong positive experience with minor inconsistencies
<i>Ambiguous Case</i>	21679	Yes	Positive	Positive	Generally positive with some service variability
<i>Ambiguous Case</i>	6118	Yes	Positive	Positive	Positive overall evaluation despite isolated complaints

Appendix E: Full Model Outputs

MODEL PERFORMANCE: Logistic Regression

=====

Accuracy: 0.9449

Precision: 0.9290

Recall: 0.9159

F1: 0.9224

ROC_AUC: 0.9795

Classification Report:

	precision	recall	f1-score	support
0.0	0.95	0.96	0.96	2801
1.0	0.93	0.92	0.92	1557
accuracy			0.94	4358
macro avg	0.94	0.94	0.94	4358
weighted avg	0.94	0.94	0.94	4358

=====

MODEL PERFORMANCE: Random Forest

Accuracy: 0.9631

Precision: 0.9532

Recall: 0.9428

F1: 0.9480

ROC_AUC: 0.9884

Classification Report:

	precision	recall	f1-score	support
0.0	0.97	0.97	0.97	2801
1.0	0.95	0.94	0.95	1557
accuracy			0.96	4358
macro avg	0.96	0.96	0.96	4358
weighted avg	0.96	0.96	0.96	4358

=====

THRESHOLD EVALUATION: Logistic Regression | threshold=0.3

Accuracy: 0.9440

Precision: 0.8919

Recall: 0.9595

F1: 0.9245

=====

THRESHOLD EVALUATION: Random Forest | threshold=0.3

Accuracy: 0.9578

Precision: 0.9225

Recall: 0.9627

F1: 0.9422

4.a.14 BASELINE MODEL COMPARISON

	Accuracy	Precision	Recall	F1	ROC_AUC
Model					
Logistic Regression (0.5)	0.9449	0.9290	0.9159	0.9224	0.9795
Random Forest (0.5)	0.9631	0.9532	0.9428	0.9480	0.9884
Logistic Regression (0.3)	0.9440	0.8919	0.9595	0.9245	-
Random Forest (0.3)	0.9578	0.9225	0.9627	0.9422	-

```

=====
4.a.15 LASSO TUNING RESULTS
Best parameters: {'model__C': 10}
Best CV ROC-AUC: 0.9823
=====

```

4.a.15 FEATURES SELECTED BY LASSO

	Feature	Coefficient
0	Value For Money	1.988729
1	Overall_Rating	1.445538
2	Food & Beverages_available	-0.687045
3	Wifi & Connectivity_available	-0.641395
4	Cabin Staff Service	0.564465
5	Seat Comfort	0.557228
6	Food & Beverages	0.507841
7	Ground Service	0.404878
8	Wifi & Connectivity	0.367257
9	Inflight Entertainment	-0.264105
10	Inflight Entertainment_available	0.250507


```
=====
=====
MODEL PERFORMANCE: LASSO Logistic Regression
Accuracy: 0.9449
Precision: 0.9290
Recall: 0.9159
F1: 0.9224
ROC_AUC: 0.9797
```

```
Classification Report:
              precision    recall  f1-score   support

     0.0         0.95      0.96      0.96      2801
     1.0         0.93      0.92      0.92      1557

 accuracy          0.94          0.94          0.94          4358
 macro avg         0.94          0.94          0.94          4358
 weighted avg      0.94          0.94          0.94          4358
```

```
=====
=====
THRESHOLD EVALUATION: LASSO Logistic Regression | threshold=0.3
Accuracy: 0.9442
Precision: 0.8920
Recall: 0.9602
F1: 0.9248
```

```
=====
=====
4.a.15 MODEL COMPARISON INCLUDING LASSO
```

	Accuracy	Precision	Recall	F1	ROC_AUC
Model					
Logistic Regression (0.5)	0.9449	0.9290	0.9159	0.9224	0.9795
Random Forest (0.5)	0.9631	0.9532	0.9428	0.9480	0.9884
LASSO Logistic Regression (0.5)	0.9449	0.9290	0.9159	0.9224	0.9797
Logistic Regression (0.3)	0.9440	0.8919	0.9595	0.9245	-
Random Forest (0.3)	0.9578	0.9225	0.9627	0.9422	-
LASSO Logistic Regression (0.3)	0.9442	0.8920	0.9602	0.9248	-

```
=====
=====
CONFUSION MATRIX: Logistic Regression (0.5)
```

	Predicted 0	Predicted 1
Actual 0	2692	109
Actual 1	131	1426

```
=====
=====
CONFUSION MATRIX: Random Forest (0.5)
```

	Predicted 0	Predicted 1
Actual 0	2729	72
Actual 1	89	1468

```
=====
=====
CONFUSION MATRIX: LASSO Logistic Regression (0.5)
```

	Predicted 0	Predicted 1
Actual 0	2692	109
Actual 1	131	1426

```
=====
=====
CONFUSION MATRIX: Logistic Regression (0.3)
```

	Predicted 0	Predicted 1
Actual 0	2620	181
Actual 1	63	1494

```
=====
=====
CONFUSION MATRIX: Random Forest (0.3)
```

	Predicted 0	Predicted 1
Actual 0	2675	126
Actual 1	58	1499

```
=====
CONFUSION MATRIX: LASSO Logistic Regression (0.3)
```

	Predicted 0	Predicted 1
Actual 0	2620	181
Actual 1	62	1495

	Predicted 0	Predicted 1
Actual 0	2620	181
Actual 1	62	1495

```
=====
4.a.17 TUNED LOGISTIC REGRESSION RESULTS
=====
```

```
Best parameters: {'model__C': 10, 'model__class_weight': 'balanced'}
Best CV ROC-AUC: 0.9827
```

```
=====
MODEL PERFORMANCE: Tuned Logistic Regression
=====
```

```
Accuracy: 0.9403
Precision: 0.9030
Recall: 0.9332
F1: 0.9179
ROC_AUC: 0.9802
```

```
Classification Report:
      precision    recall  f1-score   support

    0.0         0.96     0.94     0.95        2801
    1.0         0.90     0.93     0.92        1557

 accuracy         0.94         4358
  macro avg       0.93         0.94         0.94         4358
 weighted avg     0.94         0.94         0.94         4358
```

```
=====
THRESHOLD EVALUATION: Tuned Logistic Regression | threshold=0.3
=====
```

```
Accuracy: 0.9307
Precision: 0.8559
Recall: 0.9692
F1: 0.9090
```

4.a.17 TUNED RANDOM FOREST RESULTS

```
Best parameters: {'class_weight': 'balanced', 'max_depth': 10,
'min_samples_leaf': 3, 'n_estimators': 300}
Best CV ROC-AUC: 0.9914
```

MODEL PERFORMANCE: Tuned Random Forest

```
Accuracy: 0.9626
Precision: 0.9417
Recall: 0.9544
F1: 0.9480
ROC_AUC: 0.9912
```

Classification Report:

	precision	recall	f1-score	support
0.0	0.97	0.97	0.97	2801
1.0	0.94	0.95	0.95	1557
accuracy			0.96	4358
macro avg	0.96	0.96	0.96	4358
weighted avg	0.96	0.96	0.96	4358

THRESHOLD EVALUATION: Tuned Random Forest | threshold=0.3

```
Accuracy: 0.9553
Precision: 0.9078
Recall: 0.9737
F1: 0.9396
```

4.a.17 TUNED MODEL COMPARISON

	Accuracy	Precision	Recall	F1	ROC_AUC
Model					
Tuned Logistic Regression (0.5)	0.9403	0.9030	0.9332	0.9179	0.9802
Tuned Random Forest (0.5)	0.9626	0.9417	0.9544	0.9480	0.9912
Tuned Logistic Regression (0.3)	0.9307	0.8559	0.9692	0.9090	-
Tuned Random Forest (0.3)	0.9553	0.9078	0.9737	0.9396	-

4.a.17 BASELINE VERSUS TUNED MODELS

```
=====
```

	Accuracy	Precision	Recall	F1	ROC_AUC
Model					
Baseline Logistic Regression	0.9449	0.9290	0.9159	0.9224	0.9795
Baseline Random Forest	0.9631	0.9532	0.9428	0.9480	0.9884
Tuned Logistic Regression	0.9403	0.9030	0.9332	0.9179	0.9802
Tuned Random Forest	0.9626	0.9417	0.9544	0.9480	0.9912

CONFUSION MATRIX: Tuned Logistic Regression (0.5)

	Predicted 0	Predicted 1
Actual 0	2645	156
Actual 1	104	1453

CONFUSION MATRIX: Tuned Random Forest (0.5)

	Predicted 0	Predicted 1
Actual 0	2709	92
Actual 1	71	1486

```
=====
```

CONFUSION MATRIX: Tuned Logistic Regression (0.3)

	Predicted 0	Predicted 1
Actual 0	2547	254
Actual 1	48	1509

```
=====
```

CONFUSION MATRIX: Tuned Random Forest (0.3)

	Predicted 0	Predicted 1
Actual 0	2647	154
Actual 1	41	1516

	Predicted 0	Predicted 1
Actual 0	2647	154
Actual 1	41	1516

4.a.18 TIME-BASED TRAIN-TEST SPLIT

Training rows: 17430
 Testing rows: 4358
 Training period: 2005-03-14 00:00:00 to 2023-02-27 00:00:00
 Testing period : 2023-02-27 00:00:00 to 2023-07-27 00:00:00

MODEL PERFORMANCE: Logistic Regression (Time-Based Split)

Accuracy: 0.9617
 Precision: 0.8981
 Recall: 0.9431
 F1: 0.9201
 ROC_AUC: 0.9864

Classification Report:

	precision	recall	f1-score	support
0.0	0.98	0.97	0.97	3339
1.0	0.90	0.94	0.92	1019
accuracy			0.96	4358
macro avg	0.94	0.96	0.95	4358
weighted avg	0.96	0.96	0.96	4358

THRESHOLD EVALUATION: Logistic Regression (Time-Based Split) | threshold=0.3

Accuracy: 0.9504
 Precision: 0.8476
 Recall: 0.9607
 F1: 0.9006

MODEL PERFORMANCE: Random Forest (Time-Based Split)

Accuracy: 0.9670
 Precision: 0.9327
 Recall: 0.9254
 F1: 0.9291
 ROC_AUC: 0.9832

```

Classification Report:
              precision    recall  f1-score   support

         0.0         0.98      0.98      0.98        3339
         1.0         0.93      0.93      0.93        1019

 accuracy          0.97        4358
 macro avg         0.96        4358
 weighted avg      0.97        4358

```

```

=====
THRESHOLD EVALUATION: Random Forest (Time-Based Split) | threshold=0.3
=====

```

```

Accuracy: 0.9603
Precision: 0.8874
Recall: 0.9509
F1: 0.9180

```

```

=====
MODEL PERFORMANCE: LASSO Logistic Regression (Time-Based Split)
=====

```

```

Accuracy: 0.9617
Precision: 0.8981
Recall: 0.9431
F1: 0.9201
ROC_AUC: 0.9864

```

```

Classification Report:
              precision    recall  f1-score   support

         0.0         0.98      0.97      0.97        3339
         1.0         0.90      0.94      0.92        1019

 accuracy          0.96        4358
 macro avg         0.94        4358
 weighted avg      0.96        4358

```

```

=====
THRESHOLD EVALUATION: LASSO Logistic Regression (Time-Based Split) |
threshold=0.3
=====

```

```

Accuracy: 0.9504
Precision: 0.8476
Recall: 0.9607
F1: 0.9006

```

```

=====
4.a.18 TIME-BASED MODEL COMPARISON
=====

```

	Accuracy	Precision	Recall	F1	ROC_AUC
Model					
Logistic Regression Time (0.5)	0.9617	0.8981	0.9431	0.9201	0.9864
Random Forest Time (0.5)	0.9670	0.9327	0.9254	0.9291	0.9832
LASSO Logistic Regression Time (0.5)	0.9617	0.8981	0.9431	0.9201	0.9864
Logistic Regression Time (0.3)	0.9504	0.8476	0.9607	0.9006	-
Random Forest Time (0.3)	0.9603	0.8874	0.9509	0.9180	-
LASSO Logistic Regression Time (0.3)	0.9504	0.8476	0.9607	0.9006	-

```

=====
4.a.18 RANDOM SPLIT VERSUS TIME-BASED SPLIT
=====

```

	Accuracy	Precision	Recall	F1	ROC_AUC
Model					
Logistic Regression (Random Split)	0.9449	0.9290	0.9159	0.9224	0.9795
Random Forest (Random Split)	0.9631	0.9532	0.9428	0.9480	0.9884
LASSO Logistic Regression (Random Split)	0.9449	0.9290	0.9159	0.9224	0.9797
Logistic Regression (Time-Based Split)	0.9617	0.8981	0.9431	0.9201	0.9864
Random Forest (Time-Based Split)	0.9670	0.9327	0.9254	0.9291	0.9832
LASSO Logistic Regression (Time-Based Split)	0.9617	0.8981	0.9431	0.9201	0.9864

```

=====
CONFUSION MATRIX: Logistic Regression Time (0.5)
=====

```

	Predicted 0	Predicted 1
Actual 0	3230	109
Actual 1	58	961


```
=====
CONFUSION MATRIX: Random Forest Time (0.5)
=====
```

	Predicted 0	Predicted 1
Actual 0	3271	68
Actual 1	76	943

```
=====
CONFUSION MATRIX: LASSO Logistic Regression Time (0.5)
=====
```

	Predicted 0	Predicted 1
Actual 0	3230	109
Actual 1	58	961

```
=====
CONFUSION MATRIX: Logistic Regression Time (0.3)
=====
```

	Predicted 0	Predicted 1
Actual 0	3163	176
Actual 1	40	979

```
=====
CONFUSION MATRIX: Random Forest Time (0.3)
=====
```

	Predicted 0	Predicted 1
Actual 0	3216	123
Actual 1	50	969

```
=====
CONFUSION MATRIX: LASSO Logistic Regression Time (0.3)
=====
```

	Predicted 0	Predicted 1
Actual 0	3163	176
Actual 1	40	979

	Predicted 0	Predicted 1
Actual 0	3163	176
Actual 1	40	979

=====

MODEL PERFORMANCE: Text Logistic Regression

Accuracy: 0.9371
Precision: 0.9332
Recall: 0.8876
F1: 0.9098
ROC_AUC: 0.9771

Classification Report:

	precision	recall	f1-score	support
0.0	0.94	0.96	0.95	2801
1.0	0.93	0.89	0.91	1557
accuracy			0.94	4358
macro avg	0.94	0.93	0.93	4358
weighted avg	0.94	0.94	0.94	4358

=====

CONFUSION MATRIX: Text Logistic Regression

	Predicted 0	Predicted 1
Actual 0	2702	99
Actual 1	175	1382

=====

MODEL PERFORMANCE: Text Random Forest

Accuracy: 0.9158
Precision: 0.9053
Recall: 0.8536
F1: 0.8787
ROC_AUC: 0.9677

Classification Report:

	precision	recall	f1-score	support
0.0	0.92	0.95	0.94	2801
1.0	0.91	0.85	0.88	1557
accuracy			0.92	4358
macro avg	0.91	0.90	0.91	4358
weighted avg	0.92	0.92	0.92	4358

```
=====
=====
```

CONFUSION MATRIX: Text Random Forest

		Predicted 0	Predicted 1
Actual 0	2662		139
Actual 1	228		1329

```
=====
=====
```

MODEL PERFORMANCE: Text Linear SVM

Accuracy: 0.9406

Precision: 0.9315

Recall: 0.8998

F1: 0.9154

ROC_AUC: 0.9781

Classification Report:

	precision	recall	f1-score	support
0.0	0.95	0.96	0.95	2801
1.0	0.93	0.90	0.92	1557
accuracy			0.94	4358
macro avg	0.94	0.93	0.93	4358
weighted avg	0.94	0.94	0.94	4358

CONFUSION MATRIX: Text Linear SVM

```
=====
=====
```

		Predicted 0	Predicted 1
Actual 0	2698		103
Actual 1	156		1401

4.b.20 TEXT MODEL COMPARISON

```
=====
=====
```

	Accuracy	Precision	Recall	F1	ROC_AUC
Model					
Text Logistic Regression	0.9371	0.9332	0.8876	0.9098	0.9771
Text Random Forest	0.9158	0.9053	0.8536	0.8787	0.9677
Text Linear SVM	0.9406	0.9315	0.8998	0.9154	0.9781

4.b.20 NUMERICAL VERSUS TEXT MODEL COMPARISON

	Accuracy	Precision	Recall	F1	ROC_AUC
Model					
Numerical Logistic Regression	0.9449	0.9290	0.9159	0.9224	0.9795
Numerical Random Forest	0.9631	0.9532	0.9428	0.9480	0.9884
Text Logistic Regression	0.9371	0.9332	0.8876	0.9098	0.9771
Text Random Forest	0.9158	0.9053	0.8536	0.8787	0.9677
Text Linear SVM	0.9406	0.9315	0.8998	0.9154	0.9781

MODEL PERFORMANCE: Combined Logistic Regression

```

=====
Accuracy: 0.9578
Precision: 0.9438
Recall: 0.9377
F1: 0.9407
ROC_AUC: 0.9863

```

Classification Report:

	precision	recall	f1-score	support
0.0	0.97	0.97	0.97	2801
1.0	0.94	0.94	0.94	1557
accuracy			0.96	4358
macro avg	0.95	0.95	0.95	4358
weighted avg	0.96	0.96	0.96	4358

CONFUSION MATRIX: Combined Logistic Regression

	Predicted 0	Predicted 1
Actual 0	2714	87
Actual 1	97	1460

MODEL PERFORMANCE: Combined Linear SVM

```

=====
Accuracy: 0.9615
Precision: 0.9536
Recall: 0.9377
F1: 0.9456
ROC_AUC: 0.9874

```

```

Classification Report:
              precision    recall  f1-score   support

     0.0         0.97      0.97      0.97        2801
     1.0         0.95      0.94      0.95        1557

 accuracy         0.96         0.96         0.96         4358
 macro avg        0.96         0.96         0.96         4358
weighted avg        0.96         0.96         0.96         4358

```

CONFUSION MATRIX: Combined Linear SVM

```

=====
Predicted 0 Predicted 1
Actual 0  2730          71
Actual 1   97        1460

```

4.c.7 PERFORMANCE IMPACT OF FEATURE INTEGRATION

```

=====
Model                                     Accuracy Precision Recall  F1      ROC_AUC

Best Numerical Model
(Random Forest)                 0.9631   0.9532   0.9428  0.9480  0.9884

Best Core Text Model
(Logistic Regression)           0.9371   0.9332   0.8876  0.9098  0.9771

Text Linear SVM (Benchmark)      0.9406   0.9315   0.8998  0.9154  0.9781

Combined Logistic
Regression                       0.9578   0.9438   0.9377  0.9407  0.9863

Combined Linear SVM              0.9615   0.9536   0.9377  0.9456  0.9874

```

4.c.7 CHANGE IN PERFORMANCE AFTER INTEGRATION

```

=====
Comparison                               Accuracy_Delta Recall_Delta F1_Delta ROC_AUC_Delta

Combined Logistic
vs Best Numerical                   -0.0053      -0.0051     -0.0073    -0.0021

Combined Logistic
vs Best Core Text
Model                               0.0207         0.0501      0.0309      0.0092

Combined Logistic
vs Text SVM
Benchmark                           0.0172         0.0379      0.0253      0.0082

```

	Accuracy_Delta	Recall_Delta	F1_Delta	ROC_AUC_Delta
Comparison				
Combined SVM vs Best Numerical	-0.0016	-0.0051	-0.0024	-0.0009
Combined SVM vs Best Core Text Model	0.0243	0.0501	0.0358	0.0104
Combined SVM vs Text SVM Benchmark	0.0209	0.0379	0.0302	0.0094

=====

MODEL PERFORMANCE: Combined Random Forest

Accuracy: 0.9612

Precision: 0.9530

Recall: 0.9377

F1: 0.9453

ROC_AUC: 0.9883

Classification Report:

	precision	recall	f1-score	support
0.0	0.97	0.97	0.97	2801
1.0	0.95	0.94	0.95	1557
accuracy			0.96	4358
macro avg	0.96	0.96	0.96	4358
weighted avg	0.96	0.96	0.96	4358

CONFUSION MATRIX: Combined Random Forest

=====

	Predicted 0	Predicted 1
Actual 0	2729	72
Actual 1	97	1460